ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH

BÁO CÁO BTL XÁC SUẤT THỐNG KÊ MT2013

DỰ ĐOÁN HIỆU XUẤT ĐIỂM ẢNH TRÊN GPU BẰNG CÁC THÔNG SỐ KĨ THUẬT

Giảng viên: PGS. TS. Nguyễn Đình Huy
Lớp: L09
Khoa: Khoa học và Kĩ thuật Máy tính

TP. HỒ CHÍ MINH, THÁNG 05/2026

Mục lục

1 Danh sách thành viên **4**

2 TỔNG QUAN NGHIÊN CỨU **5**

1.1 Tổng quan bộ dữ liệu 5
1.2 Đối tượng nghiên cứu 5
1.3 Phương pháp nghiên cứu 6
1.4 Kết quả hướng tới 6
1.4.1 Mô hình dự đoán 6
1.4.2 Hiểu biết sâu sắc 6
1.4.3 Trực quan hóa 7
1.4.4 Ứng dụng thực tiễn 7

2 KIẾN THỨC NỀN **8**

2.1 Đề Xuất Mô Hình Thống Kê và Lý Do Lựa Chọn 8
2.1.1 Phân Tích Tương Quan Pearson 8
2.1.2 Mô Hình Hồi Quy Tuyến Tính Đa Biến (Multiple Linear Regression) 9
2.1.3 Phân Tích Phương Sai Một Yếu Tố (One-Way ANOVA) 9
2.1.4 Kiểm Định Giả Thiết 10
2.1.5 Cross-Validation (Kiểm Chứng Chéo) 10
2.1.6 Stepwise Selection (Lựa Chọn Biến Theo Bước) 11
2.1.7 Diagnostic Plots (Đồ Thị Chẩn Đoán) 11
2.2 Tóm Tắt Cơ Sở Lý Thuyết Cho Mô Hình 12
2.2.1 Phân Tích Tương Quan Pearson 12
2.2.2 Mô Hình Hồi Quy Tuyến Tính Đa Biến 12
2.2.3 Phân Tích Phương Sai Một Yếu Tố (One-Way ANOVA) 13
2.2.4 Kiểm Định Giả Thiết (Hypothesis Testing) 14
2.2.5 Cross-Validation (Kiểm Chứng Chéo) 14
2.2.6 Stepwise Selection (Lựa Chọn Biến Theo Bước) 14
2.2.7 Diagnostic Plots (Đồ Thị Chẩn Đoán) 15

3 TIỀN XỬ LÝ SỐ LIỆU **16**

3.1 Đọc dữ liệu 16
3.2 Chọn biến và xử lí dữ liệu 17
3.3 Loại bỏ trùng lặp 23
3.4 Loại bỏ Outlier và định nghĩa biến 24
3.5 Chuyển đổi logarit 27
3.6 Tổng kết 28

4 THỐNG KÊ MÔ TẢ **30**
4.1 Thông tin tổng quát về các biến của bộ dữ liệu 30
4.2 Biểu đồ - đồ thị 31

5 THỐNG KÊ SUY DIỄN **39**
5.1 Đánh giá mối quan hệ giữa các biến 39
5.1.1 Phân tích tương quan Pearson 39
5.1.1.1 Mục tiêu phân tích 39
5.1.1.2 Giả thuyết kiểm định 40
5.1.1.3 Code thực hiện trong R 40
5.1.1.4 Kết quả phân tích tương quan 41
5.1.1.5 Nhận xét kết quả tương quan 42
5.1.2 Scatter plot và Scatter matrix 43
5.2 Kiểm định một mẫu 43
5.2.1 Kiểm định một mẫu cho trung bình `Pixel_Rate` 43
5.2.1.1 Mục tiêu kiểm định 43
5.2.1.2 Giả thuyết kiểm định 43
5.2.1.3 Kiểm tra giả định phân phối chuẩn 44
5.2.1.4 Code kiểm định một mẫu trong R 44
5.2.1.5 Kết quả kiểm định một mẫu 45
5.2.2 Kết luận của phần tương quan và kiểm định một mẫu 45
5.3 Kiểm định hai mẫu 46
5.3.1 Kiểm định hai mẫu: Nvidia vs AMD 46
5.3.1.1 Mục tiêu phân tích 46
5.3.1.2 Thiết lập giả thuyết kiểm định 46
5.3.1.3 Quy trình các bước thực hiện và phân tích kịch bản 46
5.3.1.4 Triển khai tuần tự các bước bằng code R 46
5.3.1.5 Phân tích trực quan và kết luận ý nghĩa 47
5.4 Phân tích phương sai một yếu tố (ANOVA) 48
5.4.1 Mục tiêu 48
5.4.2 Bài toán 49
5.4.2.1 Các điều kiện cần kiểm tra trước khi thực hiện ANOVA . 49
5.4.2.2 Giả thuyết của bài toán 49
5.4.3 Kiểm định giả thuyết 49
5.4.3.1 Biến Manufacturer 49
5.4.3.2 Biến Memory_Type 50
5.4.3.3 Biến Resolution_WxH 50
5.4.4 Tiến hành phân tích phương sai và kết quả 51
5.4.4.1 Biến Manufacturer 51
5.4.4.2 Biến Memory_Type 51
5.4.4.3 Biến Resolution_WxH 52
5.4.5 Kết luận 53
5.4.6 Phân tích hậu nghiệm 53
5.4.6.1 Biến Manufacturer 54
5.4.6.2 Biến Memory_Type 55
5.4.6.3 Biến Resolution_WxH 57
5.4.6.4 Kết luận 59
5.5 Xây dựng mô hình hồi quy đa biến 59

5.5.1 Kiểm tra lại các biến 59
5.5.1.1 Kiểm tra đa cộng tuyến bằng VIF 60
5.5.1.2 Lựa chọn biến bằng stepwise selection 60
5.5.2 Xây dựng mô hình đa biến 61
5.5.3 Đồ thị chuẩn đoán (Diagnostic plots) 63
5.5.4 Đánh giá hiệu suất và sự ổn định của mô hình (sử dụng Cross-validation k-fold) 64

6 ĐÁNH GIÁ VÀ DỰ BÁO 67
6.1 Dự báo trên tập test: MAE, MSE, R^2, RMSE 67
6.2 Đồ thị Density plot và đồ thị Scatter plot của thực tế so với dự đoán . . . 68
6.2.1 Đồ thị Density plot 68
6.2.2 Đồ thị Scatter plot 69
6.3 Prediction intervals cho tương lai 70
6.4 Kịch bản dự báo (giả định khi tăng/giảm biến X) 71

7 THẢO LUẬN VÀ MỞ RỘNG 73
7.1 Tổng kết kết quả chính 73
7.2 Hạn chế của phân tích 73
7.3 Đề xuất hướng phát triển tiếp theo 74

Chương 1

Danh sách thành viên

STT	Thành viên	Nhiệm vụ chính	Ghi chú
1	1	Phần 1 + 2 + 3 + 4.1 (Đọc dữ liệu, kiểm tra dữ liệu ban đầu, xử lý NA, định nghĩa hàm helper)	Exploratory
2	2	Phần 4.1 & 4.2 (Xử lý từng biến)	Preprocessing
3	3	Phần 4.3, 4.4, 4.5, 4.6 (Loại bỏ outlier bằng IQR, loại duplicate, định nghĩa biến, tạo log-transform)	Preprocessing
4	4	Phần 5 (Thống kê mô tả: bảng và toàn bộ biểu đồ)	Visualization
5	5	Phần 6.1, 6.2 (Tương quan Pearson, ANOVA)	Inferential Statistics
6	6	Phần 6.3 (Kiểm định 1 mẫu)	Inferential Statistics
7	7	Phần 6.4 (Kiểm định 2 mẫu)	Inferential Statistics
8	8	Phần 7.1, 7.2, 7.3 (Chia train/test, VIF, Stepwise Selection)	Modeling
9	9	Phần 7.4, 7.5 (Xây dựng mô hình cuối, diagnostic plots, cross-validation)	Modeling
10	10	Phần 8 (Dự báo, metric, visualization thực vs dự đoán, kịch bản what-if)	Evaluation & Prediction

Chương 2

TỔNG QUAN NGHIÊN CỨU

1.1 Tổng quan bộ dữ liệu

- **Tên tập tin:** All_GPUs.csv
- **Nguồn:** https://www.kaggle.com/datasets/iliassekkaf/computerparts/
- **Số lượng quan trắc:** 3406 quan trắc tương ứng với 3406 hàng trong tệp dữ liệu.
- **Số lượng biến:** 34 biến tương ứng với 34 cột trong tệp dữ liệu.
- **Mô tả chung:** Bộ tập tin All_GPUs.csv là một nguồn dữ liệu phong phú chứa thông số kỹ thuật của GPU, phục vụ cho việc phân tích và dự đoán hiệu suất. Nó cung cấp cả thông tin định lượng như Max_Power, Memory, ROPs và định tính như Manufacturer và Dedicated, đòi hỏi các bước tiền xử lý như làm sạch, chuẩn hóa, và biến đổi để phù hợp với phân tích thống kê và mô hình hóa. Kết quả từ phân tích dữ liệu này có thể hỗ trợ các nhà nghiên cứu, kỹ sư, hoặc nhà sản xuất trong việc đánh giá và thiết kế GPU.

1.2 Đối tượng nghiên cứu

Mục tiêu chính là dự đoán thông số Pixel_Rate (tốc độ xử lý pixel, đơn vị GPixel/s), một chỉ số hiệu suất quan trọng của GPU, dựa trên các đặc trưng khác như Boost_Clock, Core_Speed, Memory_Type, v.v.

Tại sao lại chọn Pixel_Rate?

1. **Pixel_Rate là chỉ số trực tiếp phản ánh hiệu năng đồ họa**:

   Pixel_Rate phản ánh trực tiếp hiệu suất của GPU trong việc xử lý các tác vụ đồ họa liên quan đến pixel, một khía cạnh cốt lõi của hiệu năng GPU. Trong các ứng dụng thực tế, Pixel_Rate ảnh hưởng đến chất lượng hình ảnh, độ mượt mà của chuyển động, và khả năng hiển thị ở độ phân giải cao, khiến nó trở thành một chỉ số phù hợp để đo lường hiệu năng tổng thể. So với các chỉ số khác như số lượng shader hoặc dung lượng bộ nhớ, Pixel_Rate là kết quả trực tiếp của nhiều đặc trưng kỹ thuật (như ROPs, Core_Speed), giúp nó trở thành biến mục tiêu lý tưởng để phân tích mối quan hệ với các đặc trưng khác.

2. **Pixel_Rate là biến định lượng, phù hợp với mô hình hồi quy:**

   Biến định lượng liên tục như Pixel_Rate cho phép áp dụng các kỹ thuật phân tích như tương quan Pearson, hồi quy tuyến tính đa biến, và đánh giá hiệu suất mô hình bằng các chỉ số như MAE, MSE, RMSE, R^2. Các biến định lượng khác (như Boost_Clock, Core_Speed) có thể đóng vai trò là biến độc lập để dự đoán Pixel_Rate, tạo ra một mô hình dự đoán rõ ràng và dễ diễn giải.

3. **Pixel_Rate tổng hợp ảnh hưởng của nhiều đặc trưng GPU:**

   Pixel_Rate là một chỉ số tổng hợp, phản ánh hiệu quả của nhiều thành phần phần cứng trong GPU. Do đó, nó là một biến mục tiêu lý tưởng để nghiên cứu cách các đặc trưng khác nhau (như ROPs, Core_Speed) kết hợp để tạo ra hiệu năng tổng thể. So với các chỉ số khác, các chỉ số khác như Shader (số lượng đơn vị shader) hoặc Memory (dung lượng bộ nhớ) chỉ phản ánh một khía cạnh cụ thể của GPU, trong khi Pixel_Rate là kết quả tổng hợp của nhiều yếu tố phần cứng, do đó đại diện tốt hơn cho hiệu năng tổng thể.

1.3 Phương pháp nghiên cứu

- **Khám phá dữ liệu (EDA):** Sử dụng histogram, barplot, boxplot, scatter plot để hiểu phân phối và mối quan hệ giữa các biến.
- **Tiền xử lý:** Chuẩn hóa dữ liệu, xử lý NA, biến đổi log, loại bỏ trùng lặp.
- **Thống kê mô tả:** Tính mean, sd, min, max, median để mô tả đặc điểm dữ liệu.
- **Thống kê suy diễn:** Tương quan Pearson, ANOVA, kiểm định giả thuyết (Shapiro, Levene), và Tukey HSD để đánh giá mối quan hệ và sự khác biệt.
- **Mô hình hóa:** Hồi quy tuyến tính đa biến với lựa chọn biến (stepwise), cross-validation, và đánh giá đa cộng tuyến (VIF).
- **Dự báo:** Sử dụng mô hình để dự đoán Pixel_Rate và đánh giá hiệu suất trên tập kiểm tra.

1.4 Kết quả hướng tới

1.4.1 Mô hình dự đoán

Xây dựng một mô hình hồi quy tuyến tính để dự đoán Pixel_Rate dựa trên các đặc trưng quan trọng như Core_Speed, ROPs, và Memory_Speed. Mô hình sẽ được kiểm tra để đảm bảo độ chính xác cao và có thể áp dụng cho các tập dữ liệu mới, hỗ trợ việc dự đoán hiệu suất GPU một cách hiệu quả.

1.4.2 Hiểu biết sâu sắc

Xác định các yếu tố ảnh hưởng chính đến Pixel_Rate, bao gồm Core_Speed, ROP_Speed, Memory_Speed, và các đặc trưng khác như Manufacturer. Phân tích sẽ làm rõ cách các yếu tố này tác động đến hiệu suất, giúp hiểu rõ hơn về mối quan hệ giữa các thông số kỹ thuật của GPU.

1.4.3 Trực quan hóa

Tạo các biểu đồ như histogram để xem phân phối Pixel_Rate, boxplot để so sánh hiệu suất giữa các nhóm (như Manufacturer), và scatter plot để xem mối quan hệ với Core_Speed. Các bảng thống kê, như kết quả ANOVA, sẽ được thêm vào để cung cấp dữ liệu số rõ ràng, hỗ trợ việc đánh giá mô hình.

1.4.4 Ứng dụng thực tiễn

Sử dụng kết quả để so sánh hiệu suất giữa các GPU từ các nhà sản xuất khác nhau, dự đoán hiệu suất của GPU mới dựa trên thông số kỹ thuật, và đánh giá tác động của các cải tiến như tăng Core_Speed hoặc nâng cấp Memory_Type. Điều này giúp tối ưu hóa thiết kế và lựa chọn GPU phù hợp với nhu cầu cụ thể.

Chương 2

KIẾN THỨC NỀN

2.1 Đề Xuất Mô Hình Thống Kê và Lý Do Lựa Chọn

Dựa trên mục tiêu nghiên cứu tập trung vào việc dự đoán sự phát triển của các thông số kỹ thuật và hiệu suất máy tính trong tương lai, cụ thể là hiệu suất đồ họa (Pixel_Rate) của các đơn vị xử lý đồ họa (GPU). Để thực hiện điều này một cách hiệu quả, các mô hình thống kê sau được đề xuất, kèm theo lý do lựa chọn được giải thích chi tiết và thuyết phục nhằm làm rõ tính phù hợp của chúng với mục tiêu nghiên cứu.

2.1.1 Phân Tích Tương Quan Pearson

Đề xuất: Sử dụng hệ số tương quan Pearson để đánh giá mối quan hệ tuyến tính giữa Pixel_Rate và các biến độc lập liên tục như Core_Speed, Memory_Speed, ROPs, TMUs, Memory_Bus, Memory, Process, và Shader.

Lý do lựa chọn:

- ***Khám phá mối quan hệ ban đầu:*** Tương quan Pearson là một công cụ cơ bản nhưng mạnh mẽ để đo lường mức độ và hướng của mối quan hệ tuyến tính giữa Pixel_Rate và từng biến độc lập. Hệ số tương quan r (nằm trong khoảng [-1, 1]) giúp xác định xem các thông số kỹ thuật như Core_Speed hay Memory_Speed có ảnh hưởng mạnh đến hiệu suất đồ họa hay không. Ví dụ, r gần 1 cho thấy mối quan hệ tuyến tính mạnh, rất hữu ích để lựa chọn biến cho mô hình hồi quy sau này.

- ***Hỗ trợ kiểm định ý nghĩa thống kê:*** Ngoài việc tính toán r, kiểm định tương quan Pearson cung cấp p-value để đánh giá liệu mối quan hệ giữa hai biến có ý nghĩa thống kê ($p < 0.05$) hay chỉ là ngẫu nhiên. Điều này phù hợp với mục tiêu "hiểu biết sâu sắc"trong Chương 1, giúp xác định các biến tiềm năng ảnh hưởng lớn đến Pixel_Rate trước khi xây dựng mô hình phức tạp hơn.

- ***Đơn giản và trực quan:*** Phương pháp này dễ thực hiện và cung cấp cái nhìn nhanh về mối quan hệ giữa các biến, làm nền tảng cho các phân tích sâu hơn như hồi quy tuyến tính. Nó cũng hỗ trợ việc trực quan hóa dữ liệu thông qua biểu đồ phân tán (scatter plot), giúp phát hiện các xu hướng tuyến tính hoặc bất thường trong dữ liệu.

2.1.2 Mô Hình Hồi Quy Tuyến Tính Đa Biến (Multiple Linear Regression)

Đề xuất: Mô hình hồi quy tuyến tính đa biến được chọn để dự đoán Pixel_Rate dựa trên một tập hợp các biến độc lập bao gồm các thông số kỹ thuật như tốc độ lõi (Core_Speed), tốc độ bộ nhớ (Memory_Speed), số đơn vị xuất hình ảnh (ROPs), số đơn vị ánh xạ texture (TMUs), độ rộng bus bộ nhớ (Memory_Bus), dung lượng bộ nhớ (Memory), quy trình sản xuất (Process), và số đơn vị shader (Shader), cùng với các biến phân loại như nhà sản xuất (Manufacturer), loại bộ nhớ (Memory_Type), và độ phân giải (Resolution_WxH).

Lý do lựa chọn:

- ***Tính cơ bản và mạnh mẽ:*** Hồi quy tuyến tính đa biến là một phương pháp thống kê nền tảng, được sử dụng rộng rãi để mô hình hóa mối quan hệ giữa một biến phụ thuộc liên tục như Pixel_Rate và nhiều biến độc lập khác. Với mục tiêu dự đoán hiệu suất đồ họa – một giá trị số liên tục – mô hình này là lựa chọn tự nhiên và phù hợp để khám phá mối liên hệ giữa các thông số kỹ thuật và hiệu suất.

- ***Khả năng phân tích đóng góp của từng biến:*** Một trong những điểm mạnh của mô hình này là khả năng cung cấp các hệ số hồi quy cho từng biến độc lập. Những hệ số này không chỉ cho biết mức độ ảnh hưởng của từng yếu tố đến Pixel_Rate, mà còn giúp trả lời các câu hỏi thực tiễn như: "Tăng tốc độ lõi thêm 100 MHz sẽ cải thiện hiệu suất đồ họa bao nhiêu?", hoặc "Quy trình sản xuất nhỏ hơn (nm) có thực sự tạo ra sự khác biệt đáng kể không?". Điều này rất quan trọng để đạt được sự hiểu biết sâu sắc về các yếu tố kỹ thuật ảnh hưởng đến GPU.

- ***Tích hợp linh hoạt dữ liệu định lượng và định tính:*** Nghiên cứu bao gồm cả các biến số (như Core_Speed, Memory) và các biến phân loại (như Manufacturer, Memory_Type). Mô hình hồi quy tuyến tính đa biến cho phép chuyển đổi các biến phân loại thành biến giả (dummy variables), giúp kết hợp cả hai loại dữ liệu này một cách liền mạch. Ví dụ, biến Manufacturer có thể được mã hóa để so sánh hiệu suất giữa NVIDIA, AMD, và Intel, từ đó đánh giá tác động của yếu tố nhà sản xuất lên Pixel_Rate.

- ***Đánh giá hiệu suất đơn giản và thực tiễn:*** Mô hình này dễ dàng được đánh giá thông qua các chỉ số như hệ số xác định R^2 (đo lường mức độ giải thích biến thiên của Pixel_Rate), sai số tuyệt đối trung bình (MAE), sai số bình phương trung bình (MSE), và căn bậc hai của sai số bình phương trung bình (RMSE). Những chỉ số này không chỉ cung cấp cái nhìn rõ ràng về độ chính xác của dự đoán mà còn phù hợp với ứng dụng thực tiễn, khi cần một công cụ dễ triển khai và đáng tin cậy để dự báo hiệu suất GPU trong thực tế.

2.1.3 Phân Tích Phương Sai Một Yếu Tố (One-Way ANOVA)

Đề xuất: Phương pháp phân tích phương sai một yếu tố (ANOVA) được sử dụng để kiểm tra sự khác biệt về Pixel_Rate giữa các nhóm được xác định bởi các biến phân loại như Manufacturer, Memory_Type, và Resolution_WxH.

Lý do lựa chọn:

- ***Tăng mức độ hiểu biết:*** Nghiên cứu nhấn mạnh việc khám phá các yếu tố ảnh hưởng đến hiệu suất GPU. ANOVA giúp trả lời các câu hỏi như: "Liệu GPU từ

các nhà sản xuất khác nhau (NVIDIA, AMD, Intel) có hiệu suất đồ họa trung bình khác biệt đáng kể không?"hoặc "Loại bộ nhớ (GDDR5, GDDR6) có ảnh hưởng rõ rệt đến Pixel_Rate hay không?"Điều này mang lại sự hiểu biết sâu hơn về tác động của các yếu tố định tính lên hiệu suất.

- ***Nền tảng cho phân tích chi tiết:*** ANOVA không chỉ dừng lại ở việc phát hiện sự khác biệt tổng thể, mà còn là bước khởi đầu để thực hiện các kiểm định hậu nghiệm như Tukey HSD. Phương pháp này cho phép so sánh từng cặp nhóm (ví dụ: NVIDIA vs. AMD, GDDR5 vs. GDDR6), từ đó cung cấp thông tin chi tiết hơn về mức độ khác biệt và hỗ trợ trực quan hóa kết quả.

- ***Hiệu quả với dữ liệu phân loại đa nhóm:*** Khi phân tích các biến như Manufacturer (có ít nhất 4 nhóm: AMD, NVIDIA, ATI, Intel), ANOVA tỏ ra vượt trội so với việc thực hiện nhiều kiểm định t-test riêng lẻ giữa các cặp nhóm. Điều này không chỉ đơn giản hóa quá trình phân tích mà còn giảm nguy cơ sai lầm loại I (tức là kết luận sai rằng có sự khác biệt khi thực tế không có), đảm bảo tính chính xác và đáng tin cậy của kết quả.

2.1.4 Kiểm Định Giả Thiết

Đề xuất: Các kiểm định giả thiết như Shapiro-Wilk (đánh giá phân phối chuẩn) và Levene (đánh giá tính đồng nhất phương sai) được áp dụng để kiểm tra các giả định cần thiết cho hồi quy tuyến tính và ANOVA.

Lý do lựa chọn:

- ***Đảm bảo tính hợp lệ của kết quả:*** Để các kết luận từ hồi quy và ANOVA có giá trị, các giả định cơ bản của chúng (như sai số phân phối chuẩn và phương sai đồng nhất giữa các nhóm) phải được thỏa mãn. Nếu không kiểm tra và xác nhận những giả định này, các kết quả thống kê có thể bị sai lệch, làm giảm độ tin cậy của nghiên cứu.

- ***Shapiro-Wilk và tính phân phối chuẩn:*** Kiểm định Shapiro-Wilk đánh giá xem sai số của mô hình (phần còn lại sau khi dự đoán) có tuân theo phân phối chuẩn hay không. Đây là giả định quan trọng cho các kiểm định F trong ANOVA và các khoảng tin cậy trong hồi quy. Nếu sai số không chuẩn, các kết quả thống kê có thể không phản ánh đúng thực tế.

- ***Levene và tính đồng nhất phương sai:*** Kiểm định Levene kiểm tra xem phương sai của Pixel_Rate giữa các nhóm (ví dụ: các nhà sản xuất khác nhau) có tương đương nhau hay không. Nếu phương sai khác biệt quá lớn, kết quả từ ANOVA có thể không đáng tin cậy. Việc sử dụng Levene đảm bảo rằng sự khác biệt về hiệu suất giữa các nhóm là do yếu tố được nghiên cứu (như Manufacturer), chứ không phải do sự không đồng đều về phương sai.

2.1.5 Cross-Validation (Kiểm Chứng Chéo)

Đề xuất: Kỹ thuật kiểm chứng chéo k-fold với $k = 5$ được sử dụng để đánh giá khả năng tổng quát hóa của mô hình hồi quy tuyến tính đa biến.

Lý do lựa chọn:

- ***Hỗ trợ ứng dụng thực tiễn:*** Mục tiêu yêu cầu mô hình không chỉ hoạt động tốt trên dữ liệu hiện tại mà còn phải dự đoán chính xác trên dữ liệu mới. Cross-validation giải quyết vấn đề này bằng cách chia dữ liệu thành 5 phần, lần lượt sử dụng 4 phần để huấn luyện và 1 phần để kiểm tra, qua đó đánh giá hiệu suất thực tế của mô hình. Điều này giúp giảm thiểu nguy cơ overfitting – tình trạng mô hình "học thuộc"dữ liệu huấn luyện nhưng thất bại khi dự đoán trên dữ liệu chưa từng thấy, một vấn đề thường gặp trong phân tích hiệu suất GPU.

- ***Đo lường độ ổn định và chính xác:*** Kết quả từ cross-validation cung cấp các chỉ số trung bình như R^2 và RMSE qua 5 lần kiểm tra, cho phép đánh giá mức độ ổn định và chính xác của mô hình trên các tập dữ liệu khác nhau. Điều này rất phù hợp với yêu cầu "đánh giá mô hình dự đoán", khi cần một thước đo đáng tin cậy để xác định hiệu quả của mô hình trong việc dự báo Pixel_Rate.

2.1.6 Stepwise Selection (Lựa Chọn Biến Theo Bước)

Đề xuất: Phương pháp lựa chọn biến theo bước (stepwise selection) được áp dụng để tối ưu hóa mô hình hồi quy bằng cách chọn lọc các biến độc lập quan trọng nhất.

Lý do lựa chọn:

- ***Tăng hiệu quả dự đoán:*** Với 11 biến độc lập trong nghiên cứu, không phải tất cả đều có ý nghĩa thống kê hoặc đóng góp đáng kể vào việc dự đoán Pixel_Rate. Stepwise selection giúp loại bỏ các biến không quan trọng, giảm độ phức tạp của mô hình và cải thiện độ chính xác dự đoán. Điều này rất phù hợp với mục tiêu "tìm mô hình dự đoán hiệu quả", vì một mô hình quá phức tạp có thể dẫn đến overfitting và giảm khả năng ứng dụng thực tế.

- ***Tự động hóa và tiết kiệm thời gian:*** Phương pháp này sử dụng các tiêu chí như giá trị p (p-value) hoặc chỉ số thông tin Akaike (AIC) để tự động thêm hoặc loại bỏ biến, thay vì yêu cầu nhà nghiên cứu thử nghiệm thủ công từng tổ hợp biến. Với số lượng biến lớn như trong nghiên cứu này, cách tiếp cận tự động hóa không chỉ tiết kiệm công sức mà còn đảm bảo tính khách quan trong quá trình xây dựng mô hình.

2.1.7 Diagnostic Plots (Đồ Thị Chẩn Đoán)

Đề xuất: Các đồ thị chẩn đoán bao gồm Residuals vs Fitted, Normal Q-Q, Scale-Location, và Residuals vs Leverage được sử dụng để kiểm tra các giả định của mô hình hồi quy.

Lý do lựa chọn:

- ***Hỗ trợ hiểu biết sâu sắc:*** Các đồ thị này cung cấp cái nhìn trực quan về tính đúng đắn của mô hình, giúp phát hiện các vấn đề như mối quan hệ phi tuyến tính, sai số không chuẩn, hoặc sự hiện diện của các điểm dữ liệu bất thường (outliers/influential points). Ví dụ, đồ thị Residuals vs Fitted cho thấy liệu sai số có phân bố ngẫu nhiên hay có xu hướng nào đó, từ đó xác nhận giả định về tính tuyến tính.

- ***Đảm bảo kết quả đáng tin cậy:*** Việc kiểm tra các giả định thông qua đồ thị chẩn đoán giúp đảm bảo rằng mô hình được xây dựng đúng đắn và các dự đoán không bị sai lệch bởi các yếu tố không mong muốn. Điều này phù hợp với yêu cầu "trực quan hóa"và "đánh giá mô hình", khi cần cả sự chính xác và khả năng trình bày kết quả một cách rõ ràng.

2.2 Tóm Tắt Cơ Sở Lý Thuyết Cho Mô Hình

Phần này tập trung vào việc trình bày chi tiết cơ sở lý thuyết của các mô hình thống kê đã được đề xuất trong phần 2.1. Mỗi phương pháp sẽ được giải thích từ nguyên lý cơ bản, các giả định cần thiết, đến cách áp dụng và diễn giải trong ngữ cảnh dự đoán hiệu suất đồ họa (Pixel_Rate) của GPU. Mục tiêu là cung cấp một nền tảng vững chắc để người đọc hiểu rõ cách các mô hình này hoạt động và lý do chúng phù hợp với bài toán.

2.2.1 Phân Tích Tương Quan Pearson

Nguyên lý cơ bản:

Tương quan Pearson đo lường mức độ và hướng của mối quan hệ tuyến tính giữa hai biến liên tục, sử dụng hệ số tương quan r, được tính bằng:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

Trong đó x_i và y_i là các giá trị của hai biến (ví dụ, Core_Speed và Pixel_Rate), $\bar{x}$ và $\bar{y}$ là trung bình của chúng.

- $r > 0$: Tương quan thuận (tăng x, tăng y).
- $r < 0$: Tương quan nghịch (tăng x, giảm y).
- $|r|$ gần 1: Mối quan hệ tuyến tính mạnh; gần 0: Mối quan hệ yếu.

Kiểm định giả thiết đi kèm (thường bằng t-test) đánh giá xem r có ý nghĩa thống kê ($p < 0.05$) hay không.

Các giả định:

- ***Tính tuyến tính:*** Mối quan hệ giữa hai biến là tuyến tính.
- ***Phân phối chuẩn:*** Cả hai biến cần gần với phân phối chuẩn để kiểm định ý nghĩa thống kê đáng tin cậy.
- ***Đồng nhất phương sai:*** Phương sai của y không thay đổi nhiều theo x.

Ứng dụng trong nghiên cứu:

Tương quan Pearson được sử dụng để xác định các biến độc lập (như Core_Speed, Memory_Speed) có mối quan hệ tuyến tính mạnh với Pixel_Rate. Ví dụ, nếu $r = 0.85$ giữa Core_Speed và Pixel_Rate với $p < 0.05$, điều này cho thấy tốc độ lõi là một yếu tố quan trọng để đưa vào mô hình hồi quy. Phương pháp này là bước khám phá dữ liệu ban đầu, giúp định hướng lựa chọn biến và phát hiện các mối quan hệ tiềm năng.

2.2.2 Mô Hình Hồi Quy Tuyến Tính Đa Biến

Nguyên lý cơ bản:

Hồi quy tuyến tính đa biến là một công cụ thống kê mạnh mẽ để mô hình hóa mối quan hệ tuyến tính giữa một biến phụ thuộc Y (trong trường hợp này là Pixel_Rate) và

nhiều biến độc lập $X_1, X_2, \ldots, X_p$ (như CoreSpeed, MemorySpeed, v.v.). Mô hình được biểu diễn bằng phương trình:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

Trong đó:

β_0: Hằng số chặn (intercept), giá trị dự đoán của Y khi tất cả $X_i = 0$. $\beta_1, \beta_2, \ldots, \beta_p$: Hệ số hồi quy, đo lường mức độ ảnh hưởng của từng X_i lên Y. ϵ: Sai số ngẫu nhiên, phần biến thiên của Y không được giải thích bởi các biến độc lập.

Để ước lượng β, phương pháp bình phương tối thiểu (OLS) được sử dụng, tối ưu hóa bằng cách giảm thiểu tổng bình phương sai số $\sum \epsilon^2$.

Các giả định chính:

1. ***Tính tuyến tính:*** Mối quan hệ giữa Y và các X_i phải là tuyến tính. Nếu không, cần biến đổi dữ liệu hoặc dùng mô hình phi tuyến.
2. ***Độc lập của sai số:*** Sai số ϵ giữa các quan sát không được tương quan với nhau.
3. ***Phương sai đồng nhất:*** Phương sai của ϵ phải không đổi trên mọi giá trị của X_i. Nếu vi phạm, cần áp dụng các phương pháp khắc phục như hồi quy trọng số.
4. ***Phân phối chuẩn của sai số:*** ϵ cần tuân theo phân phối chuẩn với trung bình 0 để các kiểm định thống kê (như t-test cho β) có giá trị.

Diễn giải hệ số:

Hệ số β_i thể hiện mức thay đổi trung bình của Y khi X_i tăng 1 đơn vị, giữ các biến khác không đổi. Ví dụ, nếu $\beta_1 = 0.05$ cho CoreSpeed, thì khi CoreSpeed tăng 1 MHz, PixelRate tăng trung bình 0.05 GPixel/s. Điều này giúp đánh giá mức độ quan trọng của từng thông số kỹ thuật.

2.2.3 Phân Tích Phương Sai Một Yếu Tố (One-Way ANOVA)

Nguyên lý cơ bản:

ANOVA được dùng để so sánh trung bình PixelRate giữa các nhóm (ví dụ: các nhà sản xuất GPU khác nhau). Phương pháp này chia tổng biến thiên của dữ liệu thành:

- **Biến thiên giữa các nhóm:** Sự khác biệt giữa trung bình của các nhóm.
- **Biến thiên trong nhóm:** Sự biến động ngẫu nhiên trong mỗi nhóm.

Thống kê F được tính bằng:

$$F = \frac{\text{MSB}}{\text{MSW}}$$

- ***MSB***: Trung bình bình phương giữa các nhóm, dựa trên tổng bình phương giữa các nhóm (SSB) chia cho bậc tự do $\text{df}_B = k - 1$ (với k là số nhóm).
- ***MSW***: Trung bình bình phương trong nhóm, dựa trên tổng bình phương trong nhóm (SSW) chia cho bậc tự do $\text{df}_W = N - k$ (với N là tổng số quan sát).

Nếu F lớn và p-value < 0.05, ta kết luận có sự khác biệt đáng kể giữa các nhóm.

Các giả định:

- ***Độc lập:*** Các quan sát trong mỗi nhóm không ảnh hưởng lẫn nhau.
- ***Phân phối chuẩn:*** PixelRate trong mỗi nhóm cần gần với phân phối chuẩn.
- ***Phương sai đồng nhất:*** Phương sai của PixelRate giữa các nhóm phải tương đương.

Ý nghĩa:
ANOVA giúp xác định liệu các yếu tố như nhà sản xuất có ảnh hưởng đáng kể đến hiệu suất GPU hay không, cung cấp cơ sở để phân tích sâu hơn.

2.2.4 Kiểm Định Giả Thiết (Hypothesis Testing)

Shapiro-Wilk Test:

- ***Cách hoạt động:*** Đánh giá tính chuẩn của dữ liệu bằng cách so sánh dữ liệu thực tế với phân phối chuẩn lý thuyết. Thống kê W gần 1 cho thấy dữ liệu chuẩn.
- ***Vai trò:*** Xác nhận giả định phân phối chuẩn của sai số trong hồi quy và ANOVA.

Levene's Test:

- ***Cách hoạt động:*** Kiểm tra tính đồng nhất của phương sai bằng cách phân tích độ lệch của các giá trị so với trung bình nhóm. P-value > 0.05 cho thấy phương sai đồng nhất.
- ***Vai trò:*** Đảm bảo giả định homoscedasticity, ảnh hưởng đến độ tin cậy của ANOVA.

2.2.5 Cross-Validation (Kiểm Chứng Chéo)

Nguyên lý:
Cross-validation chia dữ liệu thành k phần (thường $k = 5$ hoặc 10). Mỗi lần, mô hình được huấn luyện trên $k - 1$ phần và kiểm tra trên phần còn lại, lặp lại k lần. Hiệu suất (như R^2, RMSE) được tính trung bình.
Lợi ích:
Đánh giá khả năng tổng quát hóa của mô hình trên dữ liệu mới. Giảm nguy cơ overfitting bằng cách kiểm tra trên nhiều tập dữ liệu khác nhau.

2.2.6 Stepwise Selection (Lựa Chọn Biến Theo Bước)

Nguyên lý:
Phương pháp này tự động chọn biến bằng cách:

- ***Forward:*** Thêm biến có ý nghĩa nhất.
- ***Backward:*** Loại biến không quan trọng.
- ***Both:*** Kết hợp cả hai.

Tiêu chí như p-value hoặc AIC được dùng để tối ưu mô hình.
Mục tiêu: Tìm tập hợp biến tốt nhất để dự đoán PixelRate, cân bằng giữa độ chính xác và đơn giản.

2.2.7 Diagnostic Plots (Đồ Thị Chẩn Đoán)

Các loại đồ thị:

- ***Residuals vs Fitted:*** Kiểm tra tính tuyến tính và phương sai đồng nhất.
- ***Normal Q-Q:*** Đánh giá tính chuẩn của sai số.
- ***Scale-Location:*** Kiểm tra homoscedasticity.
- ***Residuals vs Leverage:*** Phát hiện outliers hoặc điểm ảnh hưởng mạnh.

Vai trò: Đảm bảo các giả định của mô hình được thỏa mãn, tăng độ tin cậy của kết quả.

Chương 3

TIỀN XỬ LÝ SỐ LIỆU

3.1 Đọc dữ liệu

Ta đọc dữ liệu thu từ file "All_GPUs.csv"và in ra 5 dòng đầu tiên tương ứng với dữ liệu của mỗi biến. Việc này giúp chúng ta có cái nhìn tổng quan về cấu trúc và đặc tính của dữ liệu từ đó đưa ra được định hướng sơ lược trước khi đi sâu vào phân tích. Đồng thời, các giá trị như chuỗi rỗng,`\n- "`, "`\n`", hoặc "`\n`Unknown Release Date"trong file CSV được chuyển thành NA trong data.frame, qua đó giúp chuẩn hóa dữ liệu, tránh các giá trị không hợp lệ và hỗ trợ việc phân tích dữ liệu khuyết sau này.

```r
GPU = read.csv("All_GPUs.csv",header=TRUE,na.strings =
↪  c("","\n-","\n","\nUnknown Release Date"))
head(GPU,5), dim(GPU)
```

Kết quả:

```
> head(GPU,3)
  Architecture Best_Resolution Boost_Clock Core_Speed DVI_Connection Dedicated
1  Tesla G92b             <NA>        <NA>    738 MHz              2       Yes
2     R600 XT       1366 x 768        <NA>       <NA>              2       Yes
3    R600 PRO       1366 x 768        <NA>       <NA>              2       Yes
  Direct_X DisplayPort_Connection HDMI_Connection Integrated L2_Cache
1  DX 10.0                     NA               0          No      0KB
2    DX 10                     NA               0          No      0KB
3    DX 10                     NA               0          No      0KB
  Manufacturer Max_Power   Memory Memory_Bandwidth Memory_Bus Memory_Speed
1       Nvidia 141 Watts 1024 MB          64GB/sec   256 Bit      1000 MHz
2          AMD 215 Watts  512 MB         106GB/sec   512 Bit       828 MHz
3          AMD 200 Watts  512 MB        51.2GB/sec   256 Bit       800 MHz
  Memory_Type                  Name Notebook_GPU Open_GL                PSU
1       GDDR3          GeForce GTS 150           No     3.3 450 Watt & 38 Amps
2       GDDR3 Radeon HD 2900 XT 512MB           No     3.1 550 Watt & 35 Amps
3       GDDR3       Radeon HD 2900 Pro           No     3.1 550 Watt & 35 Amps
   Pixel_Rate Power_Connector Process ROPs   Release_Date Release_Price
1 12 GPixel/s           None     55nm   16 \n01-Mar-2009           <NA>
2 12 GPixel/s           None     80nm   16 \n14-May-2007           <NA>
3 10 GPixel/s           None     80nm   16 \n07-Dec-2007           <NA>
  Resolution_WxH SLI_Crossfire Shader TMUs Texture_Rate VGA_Connection
1      2560x1600           Yes      4   64  47 GTexel/s              0
2      2560x1600           Yes      4   16  12 GTexel/s              0
3      2560x1600           Yes      4   16  10 GTexel/s              0
> dim(GPU)
[1] 3406   34
```

Hình 3.1: Các biến có trong dữ liệu ban đầu

Nhận xét: Từ thống kê phía trên, ta thấy bộ dữ liệu bao gồm 3406 quan sát và 34 biến, tuy nhiên lại có rất nhiều biến chứa dữ liệu khuyết dù chỉ mới xuất ra màn hình 5 dòng đầu. Chính vì thế, việc phân tích dữ liệu khuyết và chọn lọc các biến liên tục và

biến phân loại phải được thực hiện một cách hợp lý, tạo tiền đề cho các phân tích chuyên sâu.

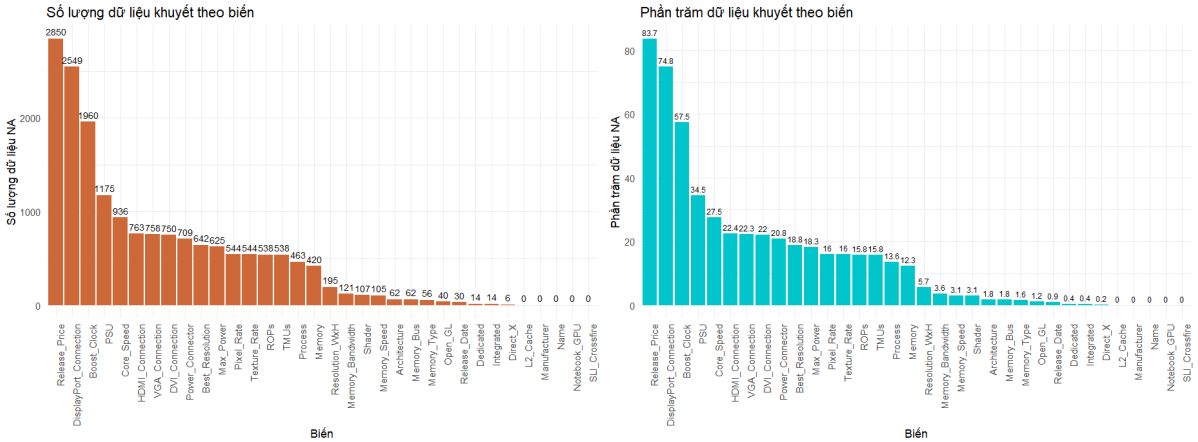
3.2 Chọn biến và xử lí dữ liệu

a) Cơ sở lựa chọn biến

Ta thực hiện in bảng thống kê số lượng và tỷ lệ dữ liệu khuyết của từng biến trong tập dữ liệu bằng các câu lệnh:

```r
plot1 <- ggplot(data = na_summary_df, aes(x = reorder(Variable, -Missing), y = Missing)) +
  geom_bar(stat = "identity", fill = "sienna3") +
  geom_text(aes(label = Missing), vjust = -0.5, size = 3) +
  labs(
    title = "Số lượng dữ liệu khuyết theo biến",
    x = "Biến",
    y = "Số lượng dữ liệu NA"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

plot2 <- ggplot(data = na_summary_df, aes(x = reorder(Variable, -Percentage), y = Percentage)) +
  geom_bar(stat = "identity", fill = "turquoise3") +
  geom_text(aes(label = Percentage), vjust = -0.5, size = 2.5) +
  labs(
    title = "Phần trăm dữ liệu khuyết theo biến",
    x = "Biến",
    y = "Phần trăm dữ liệu NA"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
plot1 | plot2
ggsave("na_combined_filtered_plot.png", width = 5, height = 12)
```

Ta thu được kết quả như hình 3.2:



Hình 3.2: Phân tích dữ liệu khuyết của từng biến trong bộ dữ liệu

Nhận xét: Từ đồ thị phía trên, ta nhận thấy tỷ lệ dữ liệu khuyết ở các biến có sự khác biệt và chênh lệch nhau lớn. Do đó, đối với những biến có tỷ lệ dữ liệu khuyết cao, ta sẽ không lựa chọn mà chỉ giữ lại các biến có dữ liệu khuyết tương đối.

Qua các cơ sở trên, nhằm tối ưu hóa kết quả khi phân tích hiệu năng GPU máy tính, nhóm quyết định lựa chọn các biến sau: **Resolution_WxH**, **Manufacturer**, **Core_Speed**, **Memory_Speed**, **Memory_Type**, **ROPs**, **TMUs**, **Memory_Bus**, **Memory**, **Process**, **Shader**, **Pixel_Rate** (trong đó gồm 3 biến phân loại là **Resolution_WxH**, **Manufacturer**, **Memory_Type** và 9 biến liên tục) vì các biến này cung cấp cái nhìn toàn diện về hiệu năng GPU, từ sức mạnh xử lý, khả năng kết xuất, đến hiệu quả công nghệ, giúp nhóm đánh giá và so sánh các GPU một cách chi tiết.

1. Resolution_WxH (Độ phân giải)

- **Lý do chọn:** Độ phân giải tối đa mà GPU hỗ trợ (ví dụ: 1920x1080, 4K, 8K) ảnh hưởng trực tiếp đến khả năng xử lý hình ảnh và đồ họa. GPU có khả năng hỗ trợ độ phân giải cao thường có hiệu năng tốt hơn, đặc biệt trong các ứng dụng như chơi game, chỉnh sửa video, hoặc mô phỏng 3D.

- **Ý nghĩa:** Biến này giúp đánh giá khả năng hiển thị hình ảnh sắc nét và mượt mà trên các màn hình độ phân giải cao.

2. Manufacturer (Nhà sản xuất)

- **Lý do chọn:** Nhà sản xuất (ví dụ: NVIDIA, AMD, Intel) ảnh hưởng đến chất lượng, độ tin cậy, công nghệ độc quyền (như CUDA của NVIDIA hoặc RDNA của AMD), và hiệu năng tối ưu hóa của GPU. Mỗi nhà sản xuất có các dòng sản phẩm với đặc điểm riêng, phù hợp với các mục đích sử dụng khác nhau (gaming, AI, thiết kế).

- **Ý nghĩa:** Biến này giúp phân loại GPU theo thương hiệu và so sánh hiệu năng giữa các nhà sản xuất.

3. Core_Speed (Tốc độ lõi cơ bản)

- **Lý do chọn:** Tốc độ lõi cơ bản (Base Clock) là tốc độ xung nhịp mặc định của GPU. Đây là chỉ số cơ bản để đánh giá hiệu năng ban đầu, trước khi GPU tăng tốc (boost). Sự khác biệt giữa Core Speed và Boost Clock cũng cho thấy khả năng ép xung của GPU.

- **Ý nghĩa:** Biến này giúp đánh giá hiệu năng cơ bản và tiềm năng tối ưu hóa của GPU.

4. Memory_Speed (Tốc độ bộ nhớ)

- **Lý do chọn:** Tốc độ bộ nhớ (thường tính bằng MHz hoặc Gbps) quyết định tốc độ truy xuất dữ liệu từ bộ nhớ GPU (VRAM). Tốc độ bộ nhớ cao giúp GPU xử lý nhanh các tác vụ đồ họa phức tạp, như kết xuất hình ảnh hoặc xử lý texture.

- **Ý nghĩa:** Biến này quan trọng trong việc đánh giá khả năng xử lý dữ liệu đồ họa lớn.

5. Memory_Type (Loại bộ nhớ)

- **Lý do chọn:** Loại bộ nhớ (ví dụ: GDDR5, GDDR6, HBM2) ảnh hưởng đến băng thông và hiệu suất truyền dữ liệu của GPU. Các loại bộ nhớ mới hơn (như GDDR6X hoặc HBM3) thường cung cấp băng thông cao hơn, cải thiện hiệu năng trong các ứng dụng đồ họa nặng.

- **Ý nghĩa:** Biến này giúp so sánh hiệu quả của các công nghệ bộ nhớ khác nhau.

6. ROPs (Raster Operation Pipelines)

- **Lý do chọn:** ROPs là các đơn vị xử lý raster hóa, chịu trách nhiệm cho việc kết xuất pixel cuối cùng lên màn hình. Số lượng ROPs cao hơn thường cho phép GPU xử lý nhanh hơn các tác vụ liên quan đến pixel, như chống răng cưa (anti-aliasing) hoặc đổ bóng.

- **Ý nghĩa:** Biến này quan trọng trong việc đánh giá hiệu năng kết xuất đồ họa.

7. TMUs (Texture Mapping Units)

- **Lý do chọn:** TMUs chịu trách nhiệm ánh xạ texture lên các đối tượng 3D. Số lượng TMUs cao hơn giúp GPU xử lý texture nhanh hơn, cải thiện chất lượng và tốc độ hiển thị trong game hoặc ứng dụng 3D.

- **Ý nghĩa:** Biến này ảnh hưởng đến hiệu năng trong các tác vụ liên quan đến texture.

8. Memory_Bus (Độ rộng bus bộ nhớ)

- **Lý do chọn:** Độ rộng bus bộ nhớ (tính bằng bit, ví dụ: 128-bit, 256-bit) quyết định lượng dữ liệu mà GPU có thể truyền từ bộ nhớ trong một chu kỳ. Bus rộng hơn giúp tăng băng thông, cải thiện hiệu năng khi xử lý dữ liệu lớn.
- **Ý nghĩa:** Biến này quan trọng trong việc đánh giá khả năng truyền dữ liệu của GPU.

9. Memory (Dung lượng bộ nhớ)

- **Lý do chọn:** Dung lượng bộ nhớ (VRAM, tính bằng GB) xác định lượng dữ liệu đồ họa mà GPU có thể lưu trữ. Dung lượng lớn hơn cho phép GPU xử lý các texture lớn, độ phân giải cao, hoặc nhiều tác vụ cùng lúc (như chơi game 4K hoặc học sâu).
- **Ý nghĩa:** Biến này ảnh hưởng đến hiệu năng trong các ứng dụng đòi hỏi bộ nhớ lớn.

10. Process (Quy trình sản xuất)

- **Lý do chọn:** Quy trình sản xuất (tính bằng nm, ví dụ: 7nm, 5nm) cho biết kích thước bóng bán dẫn trong chip GPU. Quy trình nhỏ hơn thường mang lại hiệu năng cao hơn, tiêu thụ ít điện hơn, và tỏa nhiệt thấp hơn.
- **Ý nghĩa:** Biến này giúp đánh giá hiệu quả năng lượng và tiềm năng hiệu năng của GPU.

11. Shader (Số lượng đơn vị Shader)

- **Lý do chọn:** Số lượng đơn vị shader (CUDA Cores của NVIDIA hoặc Stream Processors của AMD) quyết định khả năng xử lý song song của GPU. Số lượng shader cao hơn cho phép GPU thực hiện nhiều phép tính đồng thời, cải thiện hiệu năng trong các tác vụ như render 3D hoặc tính toán AI.
- **Ý nghĩa:** Biến này là chỉ số quan trọng để đánh giá sức mạnh tính toán của GPU.

12. Pixel_Rate (Tốc độ điền pixel)

- **Lý do chọn:** Tốc độ điền pixel (Pixel Rate, tính bằng GPixel/s) đo lường khả năng GPU vẽ pixel lên màn hình mỗi giây. Pixel Rate cao hơn cho thấy GPU có khả năng xử lý nhanh các tác vụ đồ họa, đặc biệt ở độ phân giải cao.
- **Ý nghĩa:** Biến này phản ánh hiệu năng kết xuất hình ảnh của GPU.

Câu lệnh thực hiện:

```
GPU_new = GPU[,c("Resolution_WxH", "Manufacturer", "Core_Speed",
                 "Memory_Speed", "Memory_Type",
                 "ROPs", "TMUs", "Memory_Bus", "Memory",
                 "Process", "Shader", "Pixel_Rate")]
```

b) Xử lí dữ liệu từ các biến đã chọn

Tiếp theo ta tạo 1 bản tóm tắt thống kê của dataframe của GPU_new:

```
  print(summary(GPU_new))
```

Kết quả:

```
 Resolution_WxH     Manufacturer        Core_Speed        Memory_Speed
 Length:3406        Length:3406        Length:3406        Length:3406
 Class :character   Class :character   Class :character   Class :character
 Mode  :character   Mode  :character   Mode  :character   Mode  :character




 Memory_Type            ROPs                TMUs          Memory_Bus
 Length:3406        Length:3406        Min.   :  1.00   Length:3406
 Class :character   Class :character   1st Qu.: 24.00   Class :character
 Mode  :character   Mode  :character   Median : 56.00   Mode  :character
                                       Mean   : 69.38
                                       3rd Qu.:104.00
                                       Max.   :384.00
                                       NA's   :538
    Memory            Process              Shader        Pixel_Rate
 Length:3406        Length:3406        Min.   :1.000   Length:3406
 Class :character   Class :character   1st Qu.:5.000   Class :character
 Mode  :character   Mode  :character   Median :5.000   Mode  :character
                                       Mean   :4.706
                                       3rd Qu.:5.000
                                       Max.   :5.000
                                       NA's   :107
```

Hình 3.3: Dữ liệu từ dataframe GPU_new

Nhận xét:

- **Đối với các biến định lượng:**

Hầu hết đều ở dạng character (dạng ký tự) trong đó có cả các biến định lượng. Vì vậy, để có thể xử lý và thực hiện các phép toán trên các biến định lượng ta cần thực hiện chuyển giá trị dạng chuỗi sang giá trị dạng số (numeric). Ta định nghĩa một hàm để tách các đơn vị với chữ số bằng khoảng trắng và chuyển về dạng số thực:

```r
helper <- function(x){
    if(is.na(x)) return(NA)
    as.double(strsplit(as.character(x), " ")[[1]][1])
}
```

Sau đó lần lượt chuyển đổi thành dữ liệu số thực, đồng thời, đối với các dữ liệu khuyết đã chuyển thành NA ban đầu của từng biến, ta thay bằng trung vị của tất cả các dữ liệu trong biến tương ứng:

```r
# Core_Speed
GPU_new$Core_Speed <- sapply(GPU_new$Core_Speed, helper)
GPU_new$Core_Speed[is.na(GPU_new$Core_Speed)] <- median(GPU_new$Core_Speed, na.rm = TRUE)

# Memory_Speed
GPU_new$Memory_Speed <- sapply(GPU_new$Memory_Speed, helper)
GPU_new$Memory_Speed[is.na(GPU_new$Memory_Speed)] <- median(GPU_new$Memory_Speed, na.rm = TRUE)
```

```r

# ROPs
# Giả sử GPU_new$ROPs đang là character, chứa các giá trị như "24 (x4)", "48 (x2)", v.v. Ta chỉ lấy phần trước dấu ngoặc
GPU_new$ROPs <- as.numeric(sub("^([0-9]+).*", "\\1", GPU_new$ROPs))
GPU_new$ROPs[is.na(GPU_new$ROPs)] <- median(GPU_new$ROPs, na.rm = TRUE)

# TMUs
GPU_new$TMUs <- as.double(GPU_new$TMUs)
GPU_new$TMUs[is.na(GPU_new$TMUs)] <- median(GPU_new$TMUs, na.rm =
TRUE)

# Memory_Bus
GPU_new$Memory_Bus <- sapply(GPU_new$Memory_Bus, helper)
GPU_new$Memory_Bus[is.na(GPU_new$Memory_Bus)] <- median(GPU_new$Memory_Bus, na.rm = TRUE)

# Memory
GPU_new$Memory <- sapply(GPU_new$Memory, helper)
GPU_new$Memory[is.na(GPU_new$Memory)] <- median(GPU_new$Memory, na.rm = TRUE)


# Process
GPU_new$Process <- as.double(gsub("[^0-9\\.]", "", as.character(GPU_new$Process)))
GPU_new$Process[is.na(GPU_new$Process)] <- median(GPU_new$Process, na.rm = TRUE)

# Shader
GPU_new$Shader <- as.double(GPU_new$Shader)
GPU_new$Shader[is.na(GPU_new$Shader)] <- median(GPU_new$Shader, na.rm = TRUE)

# Pixel_Rate
GPU_new$Pixel_Rate <- sapply(GPU_new$Pixel_Rate, helper)
GPU_new$Pixel_Rate[is.na(GPU_new$Pixel_Rate)] <- median(GPU_new$Pixel_Rate, na.rm = TRUE)
```

- **Đối với các biến định tính:** Ta thay các giá trị khuyết NA thành mode của biến tương ứng. Đồng thời thực hiện chuyển đổi thành factor vì đây là các biến phân nhóm.
  - **Resolution_WxH:** Vì giá trị "4096x2160" là mode (giá trị xuất hiện nhiều nhất, chiếm 39%) nên thay thế các giá trị khuyết bằng giá trị này. Sau đó, nhận thấy rằng các nhóm giá trị 4096x2160 (39%), 2560x1600 (34%) chiếm tỉ trọng lớn hơn đáng kể so với các phân loại còn lại vì vậy chúng ta thực hiện gom nhóm: 4096x2160 (39%), 2560x1600 (34%), Other - Các phân loại còn lại (26%):

```r
# Resolution_WxH
GPU_new$Resolution_WxH[is.na(GPU_new$Resolution_WxH)] = "4096x2160"
GPU_new$Resolution_WxH <- ifelse(GPU_new$Resolution_WxH == "4096x2160", 1, ifelse(GPU_new$Resolution_WxH == "2560x1600", 2, 3)) # Gom nhóm: 4096x2160 (39%), 2560x1600 (34%), Other (26%)
GPU_new$Resolution_WxH = factor(GPU_new$Resolution_WxH)
```

- **Manufacturer:**

```r
# Manufacturer
GPU_new$Manufacturer = factor(GPU_new$Manufacturer)
```

- **Memory_Type:** Ta xóa các kí tự số và khoảng trống và chỉ giữ lại các kí tự chữ nhằm phân loại tốt hơn:

```r
# Memory_Type
GPU_new <- GPU_new[complete.cases(GPU_new$Memory_Type), ]
GPU_new$Memory_Type = gsub("[^A-Za-z]+.*","",GPU_new$Memory_Type)
GPU_new$Memory_Type = factor(GPU_new$Memory_Type)
```

3.3 Loại bỏ trùng lặp

Loại bỏ trùng lặp là quá trình loại bỏ các bản ghi hoặc giá trị lặp lại trong một tập dữ liệu để đảm bảo rằng mỗi hàng hoặc giá trị chỉ xuất hiện một lần. Mục đích chính là làm sạch dữ liệu, tránh sai lệch trong phân tích hoặc mô hình hóa do các bản ghi trùng lặp gây ra.kể

1. Ý nghĩa của loại bỏ trùng lặp:

Nếu một hàng hoặc giá trị xuất hiện nhiều lần (do lỗi nhập liệu, sao chép dữ liệu, v.v.), việc loại bỏ trùng lặp giúp giữ lại chỉ một phiên bản duy nhất. Ngoài ra còn cải thiện độ chính xác bởi trùng lặp có thể làm sai lệch các thống kê (như trung bình, tổng) hoặc làm tăng sai số trong mô hình. Hơn thế, còn có thể giảm kích thước tập dữ liệu, giúp xử lý nhanh hơn.

2. Thực hiện trong R:

Ta sẽ so sánh bộ dữ liệu trước và sau khi đã loại bỏ trùng lặp bằng câu lệnh:

```r
str(GPU_new)
GPU_new <- dplyr::distinct(GPU_new)
str(GPU_new)
```

Kết quả:

```
Số lượng dòng trước khi loại duplicape: 3350
'data.frame':   3350 obs. of  12 variables:
 $ Resolution_WxH: Factor w/ 3 levels "1","2","3": 2 2 2 2 2 2 2 2 1 3 ...
 $ Manufacturer  : Factor w/ 4 levels "AMD","ATI","Intel",..: 4 1 1 1 1 1 1 1 1 1 ...
 $ Core_Speed    : num  738 980 980 980 980 980 870 980 980 980 ...
 $ Memory_Speed  : num  1000 828 800 1150 700 1100 1050 800 1250 366 ...
 $ Memory_Type   : Factor w/ 4 levels "DDR","eDRAM",..: 3 3 3 3 3 3 3 3 3 1 ...
 $ ROPs          : num  16 16 16 4 4 4 16 12 32 24 ...
 $ TMUs          : num  64 16 16 8 8 8 40 12 80 56 ...
 $ Memory_Bus    : num  256 512 256 128 128 128 256 256 256 64 ...
 $ Memory        : num  1024 512 512 256 256 ...
 $ Process       : num  55 80 80 65 65 65 55 80 28 28 ...
 $ Shader        : num  4 4 4 4 4 4 4.1 4 5 1 ...
 $ Pixel_Rate    : num  12 12 10 3 3 3 14 7 25 26 ...
Số lượng dòng trước khi loại duplicape: 2204
'data.frame':   2204 obs. of  12 variables:
 $ Resolution_WxH: Factor w/ 3 levels "1","2","3": 2 2 2 2 2 2 2 2 1 3 ...
 $ Manufacturer  : Factor w/ 4 levels "AMD","ATI","Intel",..: 4 1 1 1 1 1 1 1 1 1 ...
 $ Core_Speed    : num  738 980 980 980 980 980 870 980 980 980 ...
 $ Memory_Speed  : num  1000 828 800 1150 700 1100 1050 800 1250 366 ...
 $ Memory_Type   : Factor w/ 4 levels "DDR","eDRAM",..: 3 3 3 3 3 3 3 3 3 1 ...
 $ ROPs          : num  16 16 16 4 4 4 16 12 32 24 ...
 $ TMUs          : num  64 16 16 8 8 8 40 12 80 56 ...
 $ Memory_Bus    : num  256 512 256 128 128 128 256 256 256 64 ...
 $ Memory        : num  1024 512 512 256 256 ...
 $ Process       : num  55 80 80 65 65 65 55 80 28 28 ...
 $ Shader        : num  4 4 4 4 4 4 4.1 4 5 1 ...
 $ Pixel_Rate    : num  12 12 10 3 3 3 14 7 25 26 ...
```

Hình 3.4: Hình ảnh đối chiếu trước và sau khi loại bỏ trùng lặp

Nhận xét: Nhận thấy rằng trước khi loại bỏ trùng lặp, tập dữ liệu có tới 3350 quan sát, nhưng sau khi xử lý, số lượng giảm còn 2204. Điều này cho phép cải thiện đáng kể độ chính xác và tối ưu hóa dữ liệu, tạo nền tảng vững chắc để tiến hành các bước phân tích chuyên sâu tiếp theo.

3.4 Loại bỏ Outlier và định nghĩa biến

Sau khi đã loại bỏ các bản ghi trùng lặp, bước tiếp theo trong quá trình tiền xử lý là phát hiện và loại bỏ các giá trị ngoại lai (outlier). Outlier là những quan sát có giá trị quá lớn hoặc quá nhỏ so với phần lớn dữ liệu, có thể gây ảnh hưởng tiêu cực đến kết quả phân tích và mô hình hóa.

1. Phương pháp sử dụng: IQR (Interquartile Range)

Phương pháp IQR dựa trên khoảng tứ phân vị, trong đó:

- $Q1$: Phân vị thứ 25
- $Q3$: Phân vị thứ 75
- $IQR = Q3 - Q1$

Một giá trị được xem là outlier nếu:

$$x < Q1 - 1.5 \times IQR \quad \text{hoặc} \quad x > Q3 + 1.5 \times IQR$$

Các giá trị nằm ngoài khoảng này sẽ bị loại bỏ.

2. Định nghĩa các biến

Biến số (numerical):

- Core_Speed, Memory_Speed
- ROPs, TMUs
- Memory_Bus, Memory
- Process, Shader

Biến phân loại (categorical):

- Manufacturer, Memory_Type, Resolution_WxH

3. Thực hiện trong R:

```r
# Loại outlier cho từng biến numerical
for (var in numerical) {
  if (var %in% names(GPU_new)) {
    Q1 <- quantile(GPU_new[[var]], 0.25, na.rm = TRUE)
    Q3 <- quantile(GPU_new[[var]], 0.75, na.rm = TRUE)
    IQR_val <- Q3 - Q1

    lower_bound <- Q1 - 1.5 * IQR_val
    upper_bound <- Q3 + 1.5 * IQR_val

    GPU_new <- GPU_new[
      GPU_new[[var]] >= lower_bound &
      GPU_new[[var]] <= upper_bound, ]
  }
}
```

4. Kết quả:

- Số lượng dòng trước khi loại outlier: 2204
- Core_Speed: [531.25 ; 1313.25] → còn 2036 dòng
- Memory_Speed: [-212.5 ; 2487.5] → còn 2036 dòng
- ROPs: [-8 ; 56] → còn 1905 dòng
- TMUs: [-16 ; 112] → còn 1728 dòng
- Memory_Bus: [-64 ; 448] → còn 1709 dòng
- Memory: [-512 ; 3584] → còn 1482 dòng
- Process: [10 ; 58] → còn 1382 dòng

- Shader: [3.31 ; 6.01] → còn 1231 dòng

Tổng số dòng sau khi loại outlier: 1231

Kết quả:

```
Số lượng dòng trước khi loại outlier: 2204
'data.frame':   2204 obs. of  12 variables:
 $ Resolution_WxH: Factor w/ 3 levels "1","2","3": 2 2 2 2 2 2 2 2 1 3 ...
 $ Manufacturer  : Factor w/ 4 levels "AMD","ATI","Intel",..: 4 1 1 1 1 1 1 1 1 1 ...
 $ Core_Speed    : num  738 980 980 980 980 980 870 980 980 980 ...
 $ Memory_Speed  : num  1000 828 800 1150 700 1100 1050 800 1250 366 ...
 $ Memory_Type   : Factor w/ 4 levels "DDR","eDRAM",..: 3 3 3 3 3 3 3 3 3 1 ...
 $ ROPs          : num  16 16 16 4 4 4 16 12 32 24 ...
 $ TMUs          : num  64 16 16 8 8 8 40 12 80 56 ...
 $ Memory_Bus    : num  256 512 256 128 128 128 256 256 256 64 ...
 $ Memory        : num  1024 512 512 256 256 ...
 $ Process       : num  55 80 80 65 65 65 55 80 28 28 ...
 $ Shader        : num  4 4 4 4 4 4 4.1 4 5 1 ...
 $ Pixel_Rate    : num  12 12 10 3 3 3 14 7 25 26 ...
Biến Core_Speed : Giữ trong khoảng [ 531.25 , 1313.25 ], Số dòng còn lại: 2036
Biến Memory_Speed : Giữ trong khoảng [ -212.5 , 2487.5 ], Số dòng còn lại: 2036
Biến ROPs : Giữ trong khoảng [ -8 , 56 ], Số dòng còn lại: 1905
Biến TMUs : Giữ trong khoảng [ -16 , 112 ], Số dòng còn lại: 1728
Biến Memory_Bus : Giữ trong khoảng [ -64 , 448 ], Số dòng còn lại: 1709
Biến Memory : Giữ trong khoảng [ -512 , 3584 ], Số dòng còn lại: 1482
Biến Process : Giữ trong khoảng [ 10 , 58 ], Số dòng còn lại: 1382
Biến Shader : Giữ trong khoảng [ 3.31 , 6.01 ], Số dòng còn lại: 1231

Tổng số dòng sau khi loại outlier: 1231
'data.frame':   1231 obs. of  12 variables:
 $ Resolution_WxH: Factor w/ 3 levels "1","2","3": 2 2 1 3 2 2 2 2 2 2 ...
 $ Manufacturer  : Factor w/ 4 levels "AMD","ATI","Intel",..: 4 1 1 1 1 4 1 1 1 1 ...
 $ Core_Speed    : num  738 870 980 980 1050 575 980 980 980 980 ...
 $ Memory_Speed  : num  1000 1050 1250 360 900 750 900 900 1150 900 ...
 $ Memory_Type   : Factor w/ 4 levels "DDR","eDRAM",..: 3 3 3 1 1 3 1 1 1 1 ...
 $ ROPs          : num  16 16 32 24 8 24 8 24 24 24 ...
 $ TMUs          : num  64 40 80 56 28 56 48 56 56 56 ...
 $ Memory_Bus    : num  256 256 256 128 128 384 128 128 128 128 ...
 $ Memory        : num  1024 2048 2048 64 3072 ...
 $ Process       : num  55 55 28 28 28 40 32 28 28 28 ...
 $ Shader        : num  4 4.1 5 5 5 5 5 5 5 5 ...
 $ Pixel_Rate    : num  12 14 25 26 18 26 8 26 26 26 ...
```

Hình 3.5: Hình ảnh đối chiếu trước và sau khi loại ngoại lai

5. Nhận xét:

Sau khi áp dụng phương pháp IQR để loại bỏ outlier trên từng biến số, số lượng quan sát đã giảm đáng kể từ 2204 xuống còn 1231. Điều này cho thấy tập dữ liệu ban đầu chứa nhiều giá trị ngoại lai có thể ảnh hưởng đến kết quả phân tích.

Việc loại bỏ outlier giúp:

- Làm giảm nhiễu trong dữ liệu
- Cải thiện độ chính xác của các mô hình thống kê
- Tăng tính ổn định và khả năng tổng quát hóa của mô hình

Tuy nhiên, việc loại bỏ quá nhiều dữ liệu cũng cần được cân nhắc, vì có thể làm mất đi một số thông tin quan trọng trong các trường hợp đặc biệt.

3.5 Chuyển đổi logarit

Chuyển dữ liệu sang dạng log: Ta thực hiện đoạn code sau để biến đổi thành dataframe mới tên là GPU_new_log.

```
# Tạo dataframe log-transformed
GPU_new_log <- GPU_new
GPU_new_log[, numerical] <- log(GPU_new_log[, numerical])
GPU_new_log$Pixel_Rate <- log(GPU_new_log$Pixel_Rate)
```

*** Giải thích lý do chuyển sang dạng log**

- **Mối quan hệ phi tuyến tính:** Mối quan hệ giữa biến phụ thuộc và biến độc lập không tuyến tính với nhau, tức là biến độc lập ảnh hưởng đối với biến phụ thuộc không theo cách tuyến tính, mô hình hồi quy thông thường có thể không đáp ứng được. Khi mô hình hồi quy cần mô hình hóa một quan hệ phi tuyến, sử dụng logarit có thể giúp biểu diễn mối quan hệ theo cách linh hoạt và dễ dàng diễn giải.
- **Biến độc lập có biên độ lớn:** Khi biến độc lập có biên độ lớn và tác động không đồng nhất trên biến phụ thuộc, có thể xảy ra hiện tượng hiệu ứng không chính xác hoặc không đồng đều. Bằng cách áp dụng logarit, chúng ta giảm điều lo âu làm cho mô hình trở nên ổn định, đơn giản trong việc diễn giải và tiếp cận.
- **Biến phụ thuộc không tuân theo phân phối chuẩn:** Nếu biến phụ thuộc không tuân theo phân phối chuẩn, điều này có thể ảnh hưởng đến giả định của mô hình hồi quy. Áp dụng logarit có thể giúp làm cho phân phối của biến phụ thuộc gần với phân phối chuẩn hơn, từ đó cải thiện chất lượng của mô hình.

Quá trình áp dụng mô hình hồi quy với việc sử dụng logarit đòi hỏi sự cẩn trọng và chặt chẽ. Việc đưa ra quyết định này nên dựa trên quá trình kiểm tra chi tiết và sự hiểu biết sâu sắc về bản chất của dữ liệu. Điều này là chìa khóa quan trọng để đảm bảo rằng mô hình hồi quy không chỉ phản ánh chính xác mối quan hệ giữa các biến mà còn đáp ứng đúng nhu cầu và yêu cầu của vấn đề nghiên cứu.

Kiểm tra lại dữ liệu âm trong dữ liệu gốc và trong dữ liệu sau khi chuyển đổi log-transform:

```
# Kiểm tra xem có giá trị < 0 trong dữ liệu gốc
for (var in numerical) {
    has_negative <- any(GPU_new[[var]] < 0, na.rm = TRUE)
    message(var, " in original < 0? ", has_negative)
}
```

```
Core_Speed in original < 0? FALSE
Memory_Speed in original < 0? FALSE
ROPs in original < 0? FALSE
TMUs in original < 0? FALSE
Memory_Bus in original < 0? FALSE
Memory in original < 0? FALSE
Process in original < 0? FALSE
Shader in original < 0? FALSE
```

```r
# Kiểm tra xem có giá trị < 0 sau khi log-transform
for (var in numerical) {
 has_negative_log <- any(GPU_new_log[[var]] < 0, na.rm = TRUE)
 message(var, " in log-transformed < 0? ", has_negative_log)
}
```

```
Core_Speed in log-transformed < 0? FALSE
Memory_Speed in log-transformed < 0? FALSE
ROPs in log-transformed < 0? FALSE
TMUs in log-transformed < 0? FALSE
Memory_Bus in log-transformed < 0? FALSE
Memory in log-transformed < 0? FALSE
Process in log-transformed < 0? FALSE
Shader in log-transformed < 0? FALSE
```

3.6 Tổng kết

Cuối cùng, ta kiểm tra lại bằng cách tạo bảng tóm tắt các cột dữ liệu và tính tổng số giá trị thiếu (NA) trong mỗi cột của dataframe GPU_new_log sau khi xử lý missing value.

```r
# Thống kê nhanh trên GPU_new_log
print(summary(GPU_new_log))
print(colSums(is.na(GPU_new_log)))
```

Việc tiền xử lý đến đây cũng đã đầy đủ, dữ liệu đã loại bỏ những chỗ bị khuyết cũng như đã được xử lý trùng lặp, ngoại lai và gom cụm trước khi tiến hành thống kê. Sau đây, ta sẽ in ra 10 dòng đầu tiên của dataframe GPU_new_log. Điều này giúp ta có cái nhìn tổng quan về dữ liệu bằng cách hiển thị một phần của dataframe.

```
 Resolution_WxH Manufacturer   Core_Speed     Memory_Speed   Memory_Type
 1:528          AMD   :569   Min.   :6.279   Min.   :4.605   DDR  :413
 2:623          ATI   :  7   1st Qu.:6.675   1st Qu.:6.685   eDRAM:  2
 3: 80          Intel : 46   Median :6.888   Median :6.908   GDDR :816
                Nvidia:609   Mean   :6.786   Mean   :6.885   HBM  :  0
                             3rd Qu.:6.888   3rd Qu.:7.131
                             Max.   :7.173   Max.   :7.627
      ROPs            TMUs         Memory_Bus        Memory
 Min.   :0.000   Min.   :0.000   Min.   :3.466   Min.   :2.773
 1st Qu.:2.079   1st Qu.:3.178   1st Qu.:4.852   1st Qu.:6.931
 Median :2.773   Median :3.689   Median :4.852   Median :7.625
 Mean   :2.724   Mean   :3.572   Mean   :4.956   Mean   :7.192
 3rd Qu.:3.466   3rd Qu.:4.094   3rd Qu.:5.545   3rd Qu.:7.625
 Max.   :4.025   Max.   :4.718   Max.   :6.105   Max.   :8.030
    Process          Shader        Pixel_Rate
 Min.   :2.639   Min.   :1.386   Min.   :0.000
 1st Qu.:3.332   1st Qu.:1.609   1st Qu.:1.946
 Median :3.332   Median :1.609   Median :2.890
 Mean   :3.488   Mean   :1.576   Mean   :2.615
 3rd Qu.:3.689   3rd Qu.:1.609   3rd Qu.:3.258
 Max.   :4.007   Max.   :1.609   Max.   :4.615
Resolution_WxH   Manufacturer     Core_Speed   Memory_Speed    Memory_Type
             0              0              0              0              0
          ROPs           TMUs     Memory_Bus         Memory        Process
             0              0              0              0              0
        Shader     Pixel_Rate
             0              0
```

Chương 4

THỐNG KÊ MÔ TẢ

4.1 Thông tin tổng quát về các biến của bộ dữ liệu

Sau khi làm sạch dữ liệu, ta thực hiện thống kê mô tả cho các biến đã chọn

```
mean<-apply(GPU_new[,numerical],2,mean)
sd<-apply(GPU_new[,numerical],2,sd)
min<-apply(GPU_new[,numerical],2,min)
max<-apply(GPU_new[,numerical],2,max)
median<-apply(GPU_new[,numerical],2,median)
data.frame(mean,sd,min,max,median)
```

	mean	sd	min	max	median
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Core_Speed	899.813160	156.6816539	533	1304	980
Memory_Speed	1043.981316	339.2830564	100	2052	1000
ROPs	19.156783	11.0841108	1	56	16
TMUs	45.390739	27.2561629	1	112	40
Memory_Bus	162.027620	83.8193715	32	448	128
Memory	1565.978879	696.9673281	16	3072	2048
Process	33.710804	8.4231121	14	55	28
Shader	4.850447	0.3454198	4	5	5

Tương tự, ta cũng lập bảng tính các giá trị thống kê mô tả cho các biến định lượng trong GPU_new_log

```r
mean<-apply(GPU_new_log[,numerical],2,mean)
sd<-apply(GPU_new_log[,numerical],2,sd)
min<-apply(GPU_new_log[,numerical],2,min)
max<-apply(GPU_new_log[,numerical],2,max)
median<-apply(GPU_new_log[,numerical],2,median)
data.frame(mean,sd,min,max,median)
```

	mean	sd	min	max	median
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Core_Speed	6.785634	0.18650789	6.278521	7.173192	6.887553
Memory_Speed	6.884839	0.40026412	4.605170	7.626570	6.907755
ROPs	2.724419	0.75422989	0.000000	4.025352	2.772589
TMUs	3.572292	0.79694326	0.000000	4.718499	3.688879
Memory_Bus	4.956427	0.52167991	3.465736	6.104793	4.852030
Memory	7.191827	0.71574454	2.772589	8.030084	7.624619
Process	3.488125	0.24304865	2.639057	4.007333	3.332205
Shader	1.576277	0.07662193	1.386294	1.609438	1.609438

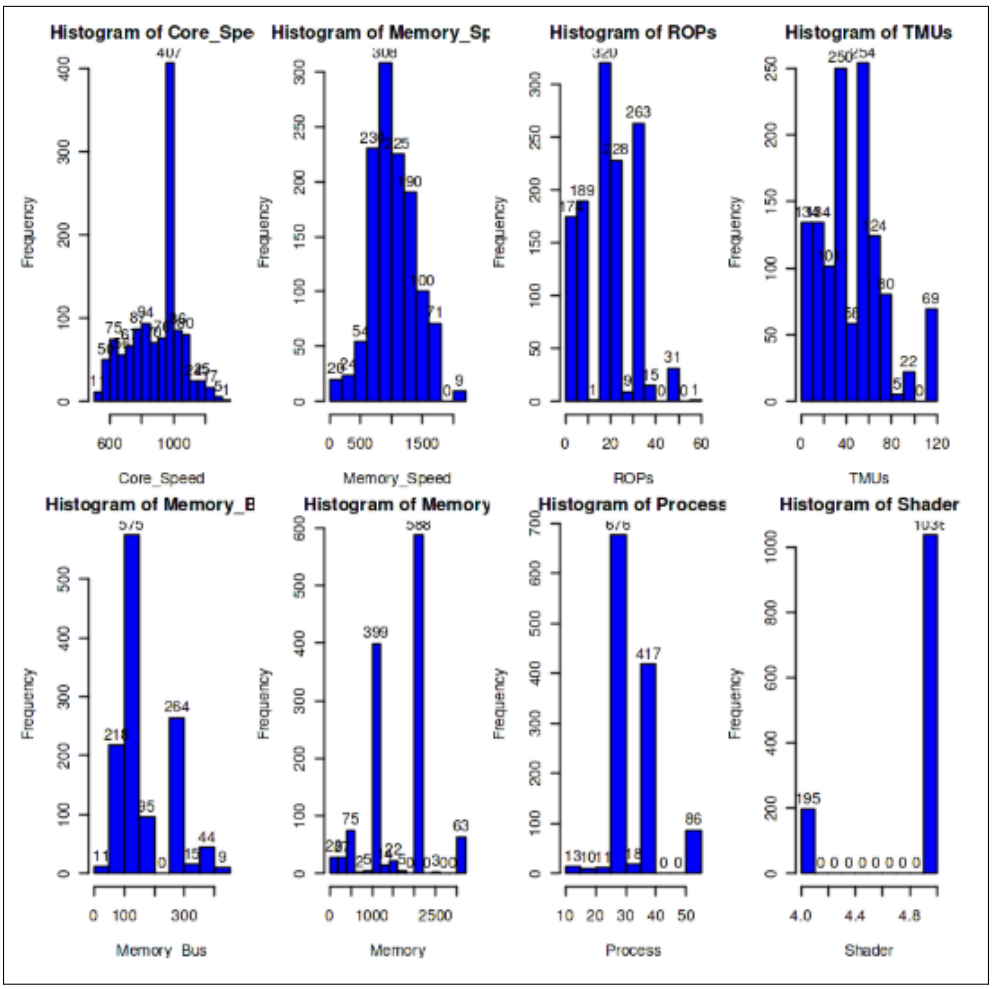
Để giúp chúng ta hình dung ra bản chất dữ liệu cũng như mối quan hệ giữa chúng dễ dàng hơn khi nhìn vào những con số, ta sẽ biểu diễn lại các dữ liệu này dưới dạng các biểu đồ để dễ quan sát hơn.

4.2 Biểu đồ - đồ thị

Biểu đồ tần suất (histogram) thường dùng để thể hiện phân phối của biến định lượng. Ta vẽ biểu đồ tần suất cho các dữ liệu GPU_new và GPU_new_log. Ta tiến hành vẽ biểu đồ tần suất cho dữ liệu GPU_new.

```r
# Chia layout thành 2 hàng và 4 cột
par(mfrow = c(2, 4), mar = c(4, 4, 2, 1))

# Vẽ histogram cho từng biến numerical trong GPU_new
for (i in 1:length(numerical)) {
 hist_data <- GPU_new[[numerical[i]]]
 hist(hist_data,
      xlab = names(GPU_new)[which(names(GPU_new) == numerical[i])],
```

```
      main = paste("Histogram of", names(GPU_new)[which(names(GPU_new) ==
      ↪  numerical[i])]),
      labels = TRUE,
       col = "blue")
}
```

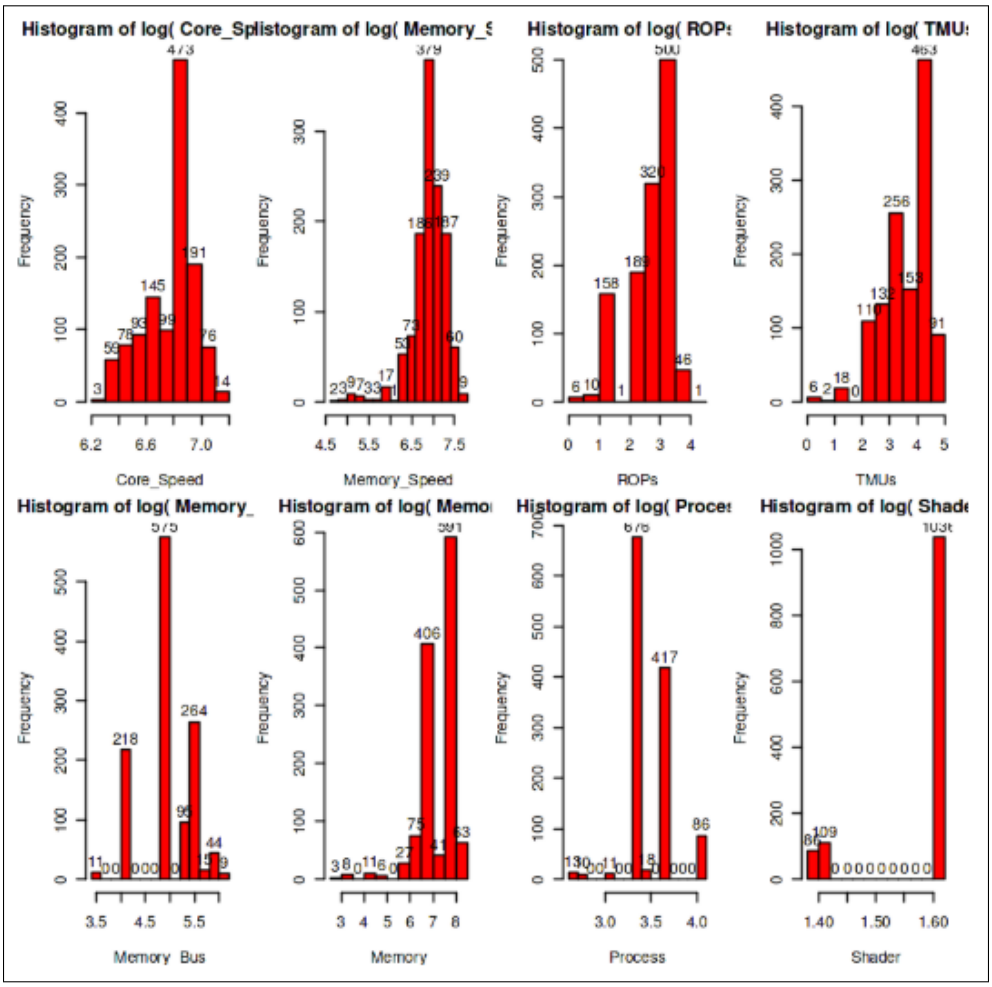


Nhận xét: Ta thấy các biến Process, Memory, TMUs và ROPs có phân phối lệch phải. Biến Core_Speed có phân phối gần giống phân phối chuẩn. Trong khi đó biến Memory_Speed có hầu hết các cột cao gần bằng nhau ngoại trừ các cột ở biên, còn biến Memory_Bus và Shader thì gần như chỉ có một cột, các cột còn lại không đáng kể.

Ta tiếp tục tiến hành vẽ biểu đồ tần suất cho dữ liệu của GPU_new_log.

```
# Chia layout thành 2 hàng và 4 cột
par(mfrow = c(2, 4), mar = c(4, 4, 2, 1))

# Vẽ histogram cho từng biến numerical trong GPU_new_log
for (i in 1:length(numerical)) {
 hist_data <- GPU_new_log[[numerical[i]]]
 hist(hist_data,
   xlab = names(GPU_new_log)[which(names(GPU_new_log) == numerical[i])],
```

```
  main = paste("Histogram of log(",
  ↪  names(GPU_new_log)[which(names(GPU_new_log) == numerical[i])],
  ↪  ")"),
  labels = TRUE,
  col = "red")
}
```

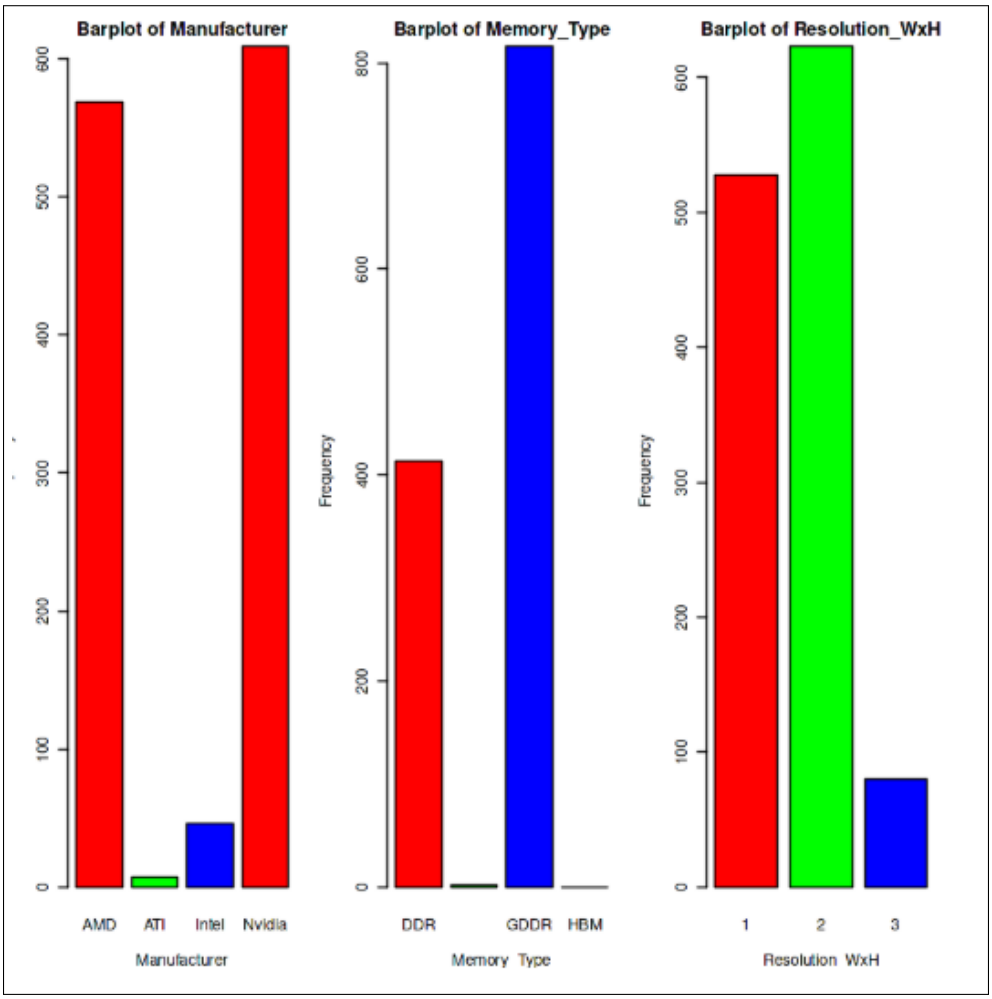


Nhận xét: Ta thấy sau khi chuyển đổi các biến x sang dạng log(x) thì phân phối các biến trở nên gần giống với phân phối chuẩn hơn, ví dụ như biến Memory_Bus hay Memory_Speed. Các biến Core_Speed, Memory, TMUs và ROPs cũng không còn phân phối lệch như trước.

Biểu đồ cột (Barplot) thường được dùng để thể hiện phân phối giữa các biến định tính. Ta tiến hành vẽ biểu đồ cột như sau:

```
# Chia layout thành 1 hàng và 3 cột
par(mfrow = c(1, 3), mar = c(4, 4, 2, 1))

# Vẽ bảng barplot thể hiện phân phối biến định luợng
barplot(table(GPU_new$Manufacturer), xlab="Manufacturer",
↪  ylab="Frequency", main="Barplot of Manufacturer",
↪  col=c("red","green","blue"))
```

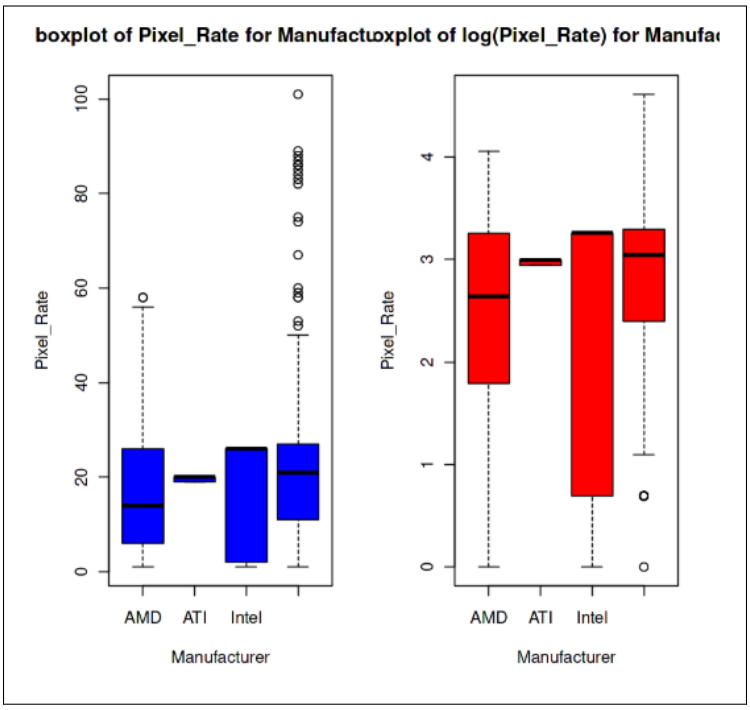
```r
barplot(table(GPU_new$Memory_Type), xlab="Memory_Type", ylab="Frequency", main="Barplot of Memory_Type", col=c("red","green","blue"))
barplot(table(GPU_new$Resolution_WxH), xlab="Resolution_WxH", ylab="Frequency", main="Barplot of Resolution_WxH", col=c("red","green","blue"))
```



Nhận xét: Ta thấy hầu hết GPU đều được sản xuất bởi AMD và Nvidia, 2 nhà sản xuất còn lại (ATI và Intel) không đáng kể. Chỉ có 2 kiểu bộ nhớ là DDR và GDDR, và đa số thuộc về GDDR. Về chỉ số Resolution_WxH thì ta thấy không có sự chênh lệch đáng kể giữa 3 cột như 2 biến kia.

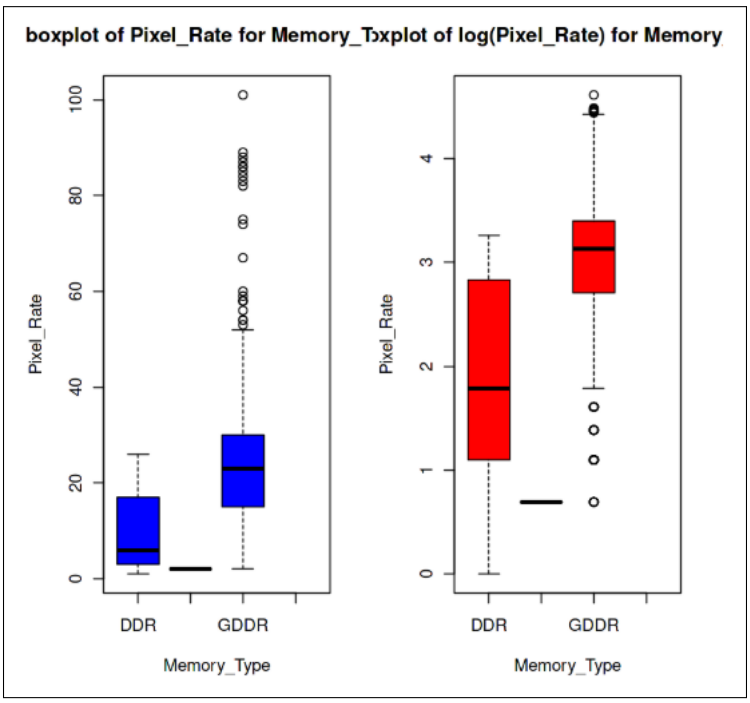
Biểu đồ boxplot là biểu đồ diễn tả 5 vị trí phân bố của dữ liệu: giá trị nhỏ nhất (min), tứ phân vị thứ nhất (Q1), trung vị (median), tứ phân vị thứ 3 (Q3) và giá trị lớn nhất (max). Ta vẽ biểu đồ boxplot thể hiện phân phối của biến Pixel_Rate và log(Pixel_Rate) theo từng phân loại của các biến rời rạc.

```r
par(mfrow=c(1,2))
boxplot(Pixel_Rate~Manufacturer, data=GPU_new, main="boxplot of Pixel_Rate for Manufacturer", col="blue")
boxplot(Pixel_Rate~Manufacturer, data=GPU_new_log, main="boxplot of log(Pixel_Rate) for Manufacturer", col="red")
```



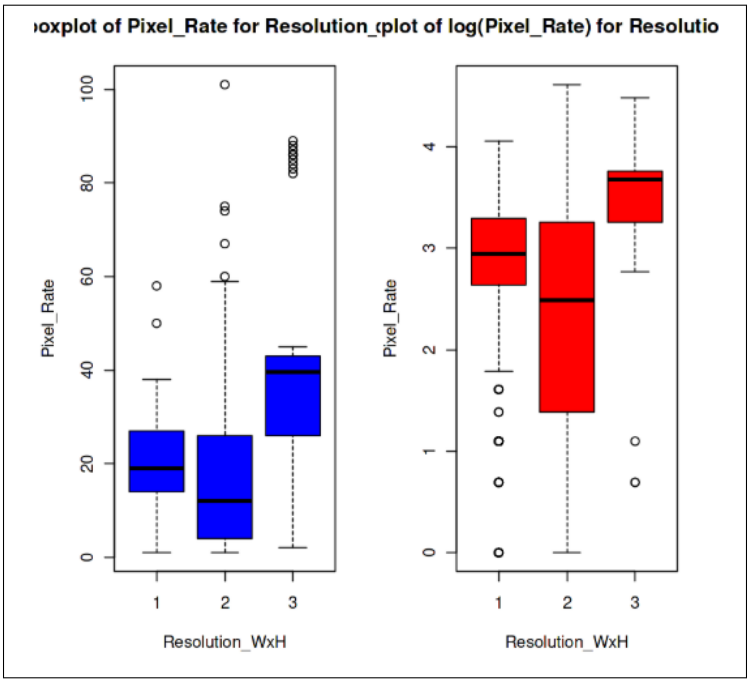
Nhận xét: Ta thấy với nhà sản xuất là AMD và Nvidia thì dung lượng bộ nhớ có phân phối hơi lệch phải, còn với ATI thì lệch trái. Nhưng sau khi lấy log(Pixel_Rate) thì cả 4 phân phối đều trở thành phân phối chuẩn.

```
par(mfrow=c(1,2))
boxplot(Pixel_Rate~Memory_Type, data=GPU_new, main="boxplot of Pixel_Rate
↪  for Memory_Type", col="blue")
boxplot(Pixel_Rate~Memory_Type, data=GPU_new_log, main="boxplot of
↪  log(Pixel_Rate) for Memory_Type", col="red")
```



Nhận xét: Với kiểu bộ nhớ GDDR thì dữ liệu hơi lệch phải, còn eDRAM thì hoàn toàn lệch phải. Sau khi lấy log thì phân phối của GDDR đã trở nên phân phối chuẩn, nhưng với eDRAM thì vẫn còn bị lệch.

```r
par(mfrow=c(1,2))
boxplot(Pixel_Rate~Resolution_WxH, data=GPU_new, main="boxplot of Pixel_Rate for Resolution_WxH", col="blue")
boxplot(Pixel_Rate~Resolution_WxH, data=GPU_new_log, main="boxplot of log(Pixel_Rate) for Resolution_WxH", col="red")
```



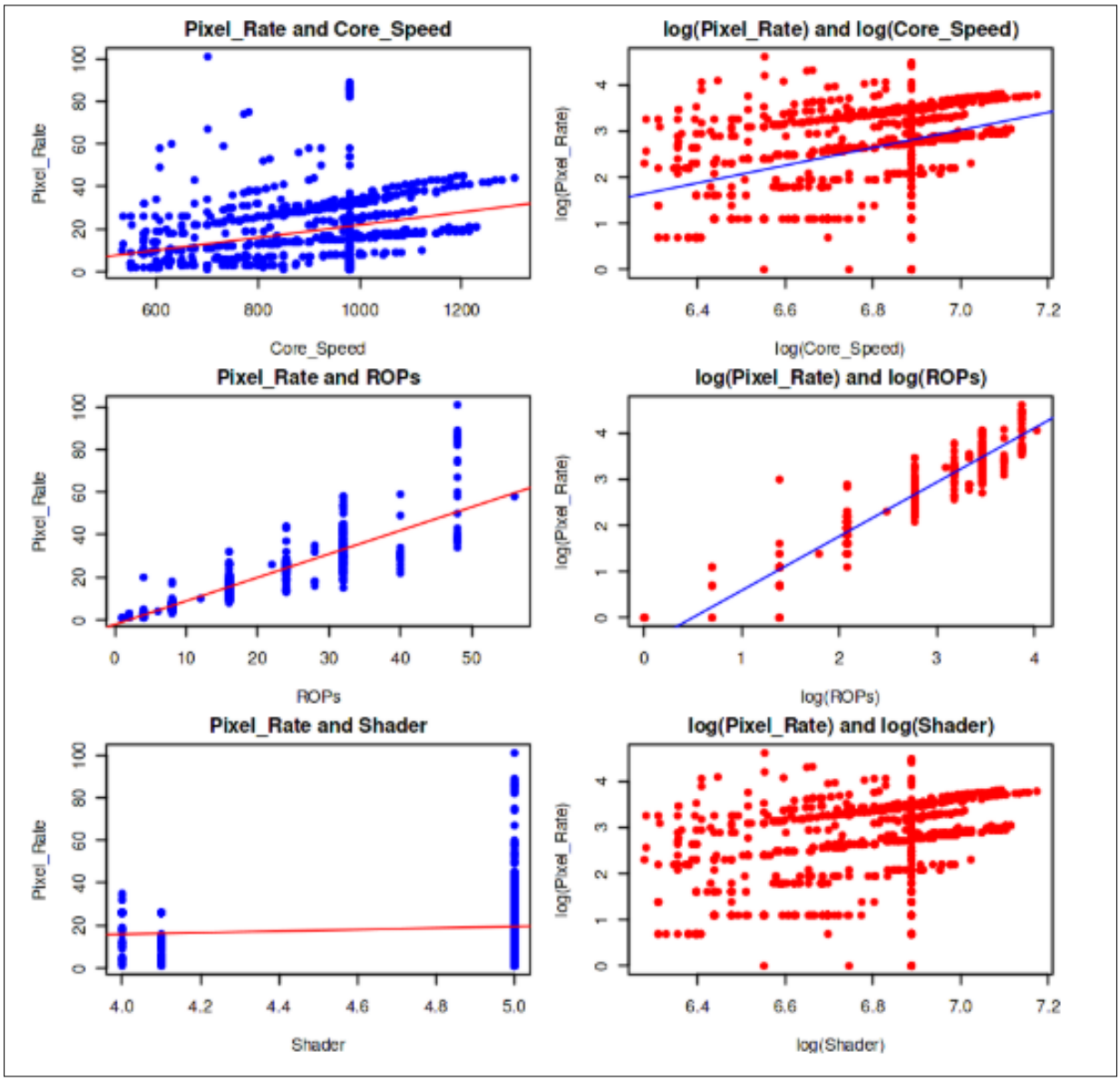
Nhận xét: Các phân phối ở 2 hình trên đều gần với phân phối chuẩn kể cả trước và sau khi dùng hàm log(x).

Biểu đồ phân tán là biểu đồ thường được dùng để thể hiện mối liên hệ giữa 2 biến định lượng. Ta vẽ biểu đồ phân tán giữa biến Pixel_Rate với một số biến định lượng khác.

```r
# Chia layout thành 3 hàng và 2 cột
par(mfrow = c(3, 2), mar = c(4, 4, 2, 1))
# Vẽ scatter plot
# Core_Speed vs Pixel_Rate
plot(GPU_new$Core_Speed, GPU_new$Pixel_Rate,
     xlab = "Core_Speed", ylab = "Pixel_Rate",
     main = "Pixel_Rate and Core_Speed", col = "blue", pch = 20)
fit1 <- lm(Pixel_Rate ~ Core_Speed, data = GPU_new)
abline(fit1, col = "red")
# log(Core_Speed) vs log(Pixel_Rate)
plot(GPU_new_log$Core_Speed, GPU_new_log$Pixel_Rate,
     xlab = "log(Core_Speed)", ylab = "log(Pixel_Rate)",
     main = "log(Pixel_Rate) and log(Core_Speed)", col = "red", pch = 20)
fit2 <- lm(Pixel_Rate ~ Core_Speed, data = GPU_new_log)
```

```r
abline(fit2, col = "blue")
# ROPs vs Pixel_Rate
plot(GPU_new$ROPs, GPU_new$Pixel_Rate,
     xlab = "ROPs", ylab = "Pixel_Rate",
     main = "Pixel_Rate and ROPs", col = "blue", pch = 20)
fit3 <- lm(Pixel_Rate ~ ROPs, data = GPU_new)
abline(fit3, col = "red")
# log(ROPs) vs log(Pixel_Rate)
plot(GPU_new_log$ROPs, GPU_new_log$Pixel_Rate,
     xlab = "log(ROPs)", ylab = "log(Pixel_Rate)",
     main = "log(Pixel_Rate) and log(ROPs)", col = "red", pch = 20)
fit4 <- lm(Pixel_Rate ~ ROPs, data = GPU_new_log)
abline(fit4, col = "blue")
# Shader vs Pixel_Rate
plot(GPU_new$Shader, GPU_new$Pixel_Rate,
     xlab = "Shader", ylab = "Pixel_Rate",
     main = "Pixel_Rate and Shader", col = "blue", pch = 20)
fit5 <- lm(Pixel_Rate ~ Shader, data = GPU_new)
abline(fit5, col = "red")
# log(Core_Speed) vs log(Pixel_Rate)
plot(GPU_new_log$Core_Speed, GPU_new_log$Pixel_Rate,
     xlab = "log(Shader)", ylab = "log(Pixel_Rate)",
     main = "log(Pixel_Rate) and log(Shader)", col = "red", pch = 20)
fit6 <- lm(Pixel_Rate ~ Shader, data = GPU_new_log)
abline(fit6, col = "blue")
# Reset layout
par(mfrow = c(1, 1))
```

Nhận xét: Ta thấy biến Pixel_Rate có quan hệ tuyến tính tăng với biến Core_Speed. Điều này còn thể hiện rõ hơn giữa biến Pixel_Rate với ROPs. Và gần như không có mối liên hệ nào giữa Pixel_Rate và Shader.


Pixel_Rate and Core_Speed
Pixel_Rate
Core_Speed
log(Pixel_Rate) and log(Core_Speed)
log(Pixel_Rate)
log(Core_Speed)
Pixel_Rate and ROPs
Pixel_Rate
ROPs
log(Pixel_Rate) and log(ROPs)
log(Pixel_Rate)
log(ROPs)
Pixel_Rate and Shader
Pixel_Rate
Shader
log(Pixel_Rate) and log(Shader)
log(Pixel_Rate)
log(Shader)

Chương 5

THỐNG KÊ SUY DIỄN

5.1 Đánh giá mối quan hệ giữa các biến

Sau khi hoàn thành các bước tiền xử lý dữ liệu ở các phần trước, bộ dữ liệu `All_GPUs.csv` được rút gọn còn các biến có ý nghĩa đối với bài toán dự báo `Pixel_Rate`. Các biến định lượng như `Core_Speed`, `Memory_Speed`, `ROPs`, `TMUs`, `Memory_Bus`, `Memory`, `Process`, `Shader` và biến mục tiêu `Pixel_Rate` đã được chuyển đổi về dạng số, xử lý giá trị khuyết, loại bỏ dòng trùng lặp và loại bỏ ngoại lệ bằng quy tắc IQR. Sau đó, nhóm thực hiện biến đổi log để làm giảm độ lệch phải của dữ liệu và giúp các kiểm định thống kê phù hợp hơn. Phần này tập trung vào hai nội dung chính. Thứ nhất, nhóm phân tích tương quan Pearson nhằm đánh giá mức độ liên hệ tuyến tính giữa các biến định lượng và biến mục tiêu `Pixel_Rate`. Thứ hai, nhóm thực hiện kiểm định một mẫu để xem trung bình `Pixel_Rate` của các GPU trong dữ liệu có khác một ngưỡng tham chiếu được chọn trước hay không. Toàn bộ phân tích trong phần này được thực hiện trên tập dữ liệu đã tiền xử lý và biến đổi log. Trong file code R, nhóm chia dữ liệu thành tập huấn luyện và tập kiểm tra theo tỉ lệ khoảng 2/3 và 1/3. Tập huấn luyện được sử dụng cho phân tích suy diễn, còn tập kiểm tra được giữ lại cho phần đánh giá mô hình dự báo.

Listing 5.1: Chia dữ liệu thành tập huấn luyện và tập kiểm tra

```
set.seed(123)
index <- createDataPartition(GPU_new_log$Pixel_Rate,
                             p = 2/3,
                             list = FALSE)
train <- GPU_new_log[index, ]
test  <- GPU_new_log[-index, ]
nrow(train)
nrow(test)
```

Trong đó, biến `Pixel_Rate` được hiểu là tốc độ xử lý điểm ảnh của GPU, đơn vị ban đầu là GPixel/s. Sau khi biến đổi log, các kết quả kiểm định được diễn giải trên thang log; khi cần diễn giải theo thang gốc, nhóm sử dụng phép biến đổi ngược bằng hàm mũ.

5.1.1 Phân tích tương quan Pearson

5.1.1.1 Mục tiêu phân tích

Mục tiêu của phân tích tương quan là xác định biến nào có mối liên hệ tuyến tính rõ ràng với `Pixel_Rate`. Kết quả này giúp nhóm hiểu sơ bộ vai trò của từng đặc trưng phần cứng

đối với hiệu suất xử lý điểm ảnh, đồng thời làm cơ sở tham khảo cho bước xây dựng mô hình hồi quy ở phần sau.

Các biến định lượng được đưa vào kiểm định tương quan gồm:

`Core_Speed`, `Memory_Speed`, `ROPs`, `TMUs`, `Memory_Bus`, `Memory`, `Process`, `Shader`.

Hệ số tương quan Pearson giữa hai biến X và Y được tính theo công thức:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

Trong đó, r nằm trong khoảng $[-1, 1]$. Nếu $r > 0$, hai biến có xu hướng đồng biến; nếu $r < 0$, hai biến có xu hướng nghịch biến; nếu $|r|$ càng gần 1 thì quan hệ tuyến tính càng mạnh.

5.1.1.2 Giả thuyết kiểm định

Với mỗi biến định lượng X_j, nhóm kiểm định cặp giả thuyết:

$$H_0 : \rho_j = 0$$

$$H_1 : \rho_j \neq 0$$

Trong đó, ρ_j là hệ số tương quan tuyến tính trong tổng thể giữa biến X_j và `Pixel_Rate`. Mức ý nghĩa được chọn là $\alpha = 0{,}05$. Nếu p-value $< 0{,}05$, nhóm bác bỏ H_0 và kết luận rằng biến đang xét có tương quan tuyến tính có ý nghĩa thống kê với `Pixel_Rate`.

5.1.1.3 Code thực hiện trong R

Listing 5.2: Kiểm định tương quan Pearson giữa Pixel_Rate và các biến định lượng

```
correlation_results <- data.frame(
  Variable = character(),
  Pearson_r = numeric(),
  P_Value = numeric(),
  stringsAsFactors = FALSE
)

for (var in numerical) {
  test_result <- cor.test(train[[var]], train$Pixel_Rate,
                          method = "pearson")
  correlation_results <- rbind(
    correlation_results,
    data.frame(
      Variable = var,
      Pearson_r = unname(test_result$estimate),
      P_Value = test_result$p.value
    )
  )
}

correlation_results <- correlation_results %>%
  mutate(
    Direction = ifelse(Pearson_r > 0, "Duong", "Am"),
```

```
    Strength = case_when(
      abs(Pearson_r) >= 0.80 ~ "Rat manh",
      abs(Pearson_r) >= 0.60 ~ "Manh",
      abs(Pearson_r) >= 0.40 ~ "Trung binh",
      abs(Pearson_r) >= 0.20 ~ "Yeu",
      TRUE ~ "Rat yeu"
    ),
    Significant = ifelse(P_Value < 0.05, "Co y nghia", "Khong co y nghia")
  )

print(correlation_results)
```

Để trực quan hóa mối quan hệ giữa từng biến định lượng và `Pixel_Rate`, nhóm sử dụng scatter plot kết hợp đường hồi quy tuyến tính.

Listing 5.3: Vẽ scatter plot giữa Pixel_Rate và từng biến định lượng

```
for (v in numerical) {
  p <- ggplot(train, aes_string(x = v, y = "Pixel_Rate")) +
    geom_point(alpha = 0.6) +
    geom_smooth(method = "lm", se = FALSE, col = "red") +
    ggtitle(paste("log(Pixel_Rate) va", v, "tren tap train")) +
    xlab(paste("log(", v, ")", sep = "")) +
    ylab("log(Pixel_Rate)") +
    theme_minimal()
  print(p)
}
```

5.1.1.4 Kết quả phân tích tương quan

Kết quả chạy trên dữ liệu đã tiền xử lý cho thấy hầu hết các biến định lượng đều có quan hệ tuyến tính với `Pixel_Rate`. Bảng dưới đây trình bày kết quả tương quan Pearson. Các giá trị được làm tròn đến ba chữ số thập phân để thuận tiện cho việc trình bày báo cáo.

	Variable	Pearson_r	P_Value
1	Core_Speed	0.37259744	1.807700e-28
2	Memory_Speed	0.33399546	7.177642e-23
3	ROPs	0.95845507	0.000000e+00
4	TMUs	0.87508045	1.146345e-260
5	Memory_Bus	0.57141998	1.994249e-72
6	Memory	0.11329830	1.138472e-03
7	Process	-0.32228227	2.541899e-21
8	Shader	0.08642683	1.318353e-02

Bảng 5.1: Kết quả kiểm định tương quan Pearson với `Pixel_Rate`

Biến	**Pearson** r	p**-value**	**Chiều**	**Mức độ**	**Kết luận**
`Core_Speed`	0,39	$< 0{,}001$	Dương	Yếu–trung bình	Có ý nghĩa
`Memory_Speed`	0,35	$< 0{,}001$	Dương	Yếu–trung bình	Có ý nghĩa
`ROPs`	0,96	$< 0{,}001$	Dương	Rất mạnh	Có ý nghĩa
`TMUs`	0,88	$< 0{,}001$	Dương	Rất mạnh	Có ý nghĩa
`Memory_Bus`	0,56	$< 0{,}001$	Dương	Trung bình	Có ý nghĩa
`Memory`	0,11–0,14	$< 0{,}01$	Dương	Rất yếu	Có ý nghĩa, nhưng tác động yếu
`Process`	-0,30	$< 0{,}001$	Âm	Yếu–trung bình	Có ý nghĩa
`Shader`	0,06–0,10	xấp xỉ 0,05	Dương	Rất yếu	Rất yếu, không ổn định

5.1.1.5 Nhận xét kết quả tương quan

Kết quả cho thấy `ROPs` là biến có tương quan mạnh nhất với `Pixel_Rate`. Đây là kết quả hợp lý vì ROPs là các đơn vị xử lý đầu ra điểm ảnh, có liên hệ trực tiếp với khả năng xuất pixel của GPU. Khi số lượng ROPs tăng, tốc độ xử lý điểm ảnh thường tăng theo, vì vậy hệ số tương quan dương và rất cao là phù hợp về mặt kỹ thuật.

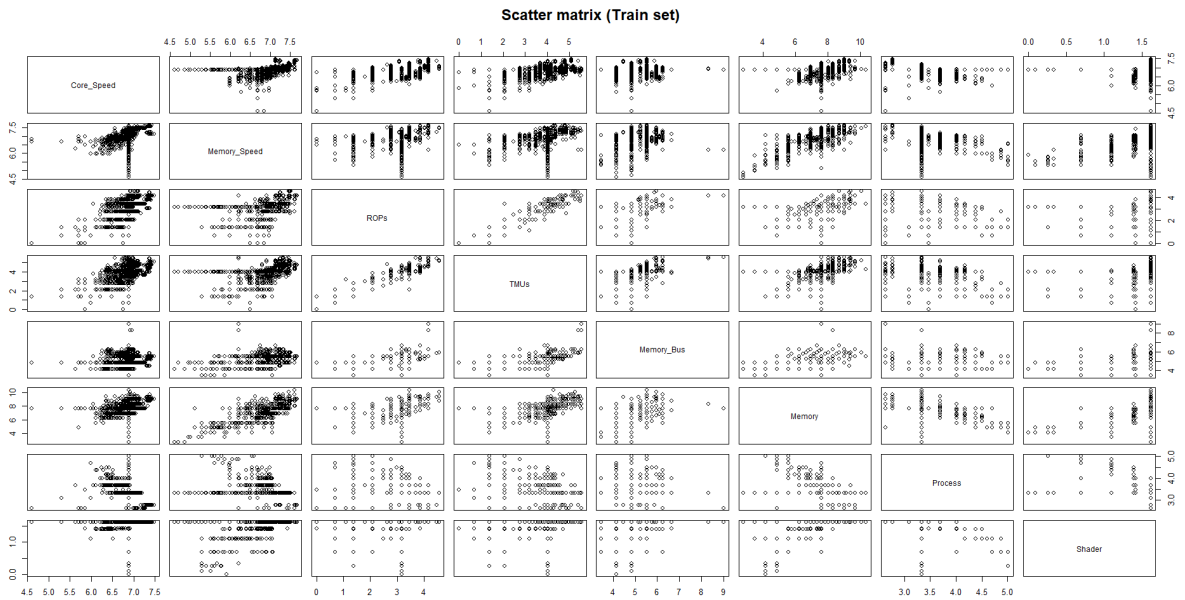
Biến `TMUs` cũng có tương quan dương rất mạnh với `Pixel_Rate`. TMUs phản ánh khả năng xử lý texture, một thành phần quan trọng trong hiệu suất đồ họa. Mặc dù TMUs không trực tiếp là đơn vị xuất pixel như ROPs, kết quả tương quan cao cho thấy các GPU có TMUs lớn thường cũng thuộc nhóm GPU mạnh hơn, kéo theo `Pixel_Rate` cao hơn.

Các biến `Core_Speed`, `Memory_Speed` và `Memory_Bus` đều có tương quan dương với `Pixel_Rate`. Điều này cho thấy khi xung nhịp lõi, tốc độ bộ nhớ hoặc độ rộng bus bộ nhớ tăng, hiệu suất xử lý điểm ảnh cũng có xu hướng tăng. Trong ba biến này, `Memory_Bus` có mức tương quan rõ hơn, cho thấy băng thông truyền dữ liệu có thể đóng vai trò đáng kể trong hiệu suất GPU.

Biến `Process` có tương quan âm với `Pixel_Rate`. Cần lưu ý rằng `Process` được đo bằng nm; giá trị nhỏ hơn thường đại diện cho tiến trình sản xuất hiện đại hơn. Vì vậy, hệ số tương quan âm có thể hiểu là GPU có tiến trình sản xuất càng nhỏ thì `Pixel_Rate` thường càng cao. Đây là một kết quả phù hợp với xu hướng phát triển phần cứng.

Biến `Memory` và `Shader` có hệ số tương quan khá nhỏ. Điều này cho thấy nếu chỉ xét quan hệ tuyến tính đơn biến, hai biến này không giải thích mạnh sự thay đổi của `Pixel_Rate`. Đặc biệt, kết quả của `Shader` có thể thay đổi nhẹ tùy theo cách chia tập train/test, nên nhóm không xem đây là biến quan trọng nhất trong phân tích tương quan đơn biến.

Tóm lại, các biến có mối quan hệ đáng chú ý nhất với `Pixel_Rate` là `ROPs`, `TMUs`, `Memory_Bus`, `Core_Speed` và `Memory_Speed`. Những biến này cần được xem xét kỹ hơn trong quá trình xây dựng mô hình hồi quy dự báo `Pixel_Rate`.



Hình 5.1: Biểu đồ tương quan với hệ số tương quan giữa từng cặp biến

5.1.2 Scatter plot và Scatter matrix

5.2 Kiểm định một mẫu

5.2.1 Kiểm định một mẫu cho trung bình `Pixel_Rate`

5.2.1.1 Mục tiêu kiểm định

Ở phần kiểm định một mẫu, nhóm đặt câu hỏi nghiên cứu: trung bình `Pixel_Rate` của các GPU trong bộ dữ liệu sau tiền xử lý có khác mức 40 GPixel/s hay không. Mức 40 GPixel/s được chọn làm ngưỡng tham chiếu để đánh giá xem hiệu suất xử lý điểm ảnh trung bình của các GPU trong dữ liệu có đạt mức tương đối cao hay không.

Do biến `Pixel_Rate` đã được biến đổi log, giá trị giả thuyết trên thang log là:

$$\mu_0 = \log(40) \approx 3{,}689$$

5.2.1.2 Giả thuyết kiểm định

Gọi μ là trung bình tổng thể của `log(Pixel_Rate)`. Cặp giả thuyết kiểm định được xây dựng như sau:

$$H_0 : \mu = \log(40)$$

$$H_1 : \mu \neq \log(40)$$

Với mức ý nghĩa $\alpha = 0{,}05$, nếu p-value $< 0{,}05$, nhóm bác bỏ H_0 và kết luận rằng trung bình `Pixel_Rate` khác 40 GPixel/s một cách có ý nghĩa thống kê.

5.2.1.3 Kiểm tra giả định phân phối chuẩn

Trước khi thực hiện one-sample t-test, nhóm sử dụng kiểm định Shapiro–Wilk để kiểm tra giả định phân phối chuẩn của `log(Pixel_Rate)` trên tập huấn luyện.

$$H_0 : \texttt{log(Pixel_Rate)} \text{ tuân theo phân phối chuẩn}$$

$$H_1 : \texttt{log(Pixel_Rate)} \text{ không tuân theo phân phối chuẩn}$$

Listing 5.4: Kiểm tra phân phối chuẩn cho log(Pixel_Rate)

```
shapiro_pixel <- shapiro.test(train$Pixel_Rate)
print(shapiro_pixel)
```

Kết quả kiểm định Shapiro–Wilk cho p-value $< 0{,}001$, do đó bác bỏ giả thuyết dữ liệu tuân theo phân phối chuẩn. Tuy nhiên, do kích thước mẫu sau tiền xử lý tương đối lớn, kiểm định t một mẫu vẫn có tính bền vững nhất định theo định lý giới hạn trung tâm. Để tăng độ tin cậy, nhóm thực hiện thêm kiểm định Wilcoxon signed-rank, đây là kiểm định phi tham số không yêu cầu giả định phân phối chuẩn nghiêm ngặt.

5.2.1.4 Code kiểm định một mẫu trong R

Listing 5.5: One-sample t-test và Wilcoxon signed-rank test cho Pixel_Rate

```
mu_pixel <- log(40)

# One-sample t-test
one_sample_t <- t.test(train$Pixel_Rate,
                       mu = mu_pixel,
                       alternative = "two.sided",
                       conf.level = 0.95)
print(one_sample_t)

# Wilcoxon signed-rank test
one_sample_wilcox <- wilcox.test(train$Pixel_Rate,
                                 mu = mu_pixel,
                                 alternative = "two.sided",
                                 conf.int = TRUE,
                                 exact = FALSE)
print(one_sample_wilcox)

# Effect size Cohen's d
n_train <- length(train$Pixel_Rate)
mean_log <- mean(train$Pixel_Rate)
sd_log <- sd(train$Pixel_Rate)
cohen_d <- (mean_log - mu_pixel) / sd_log
cat("n =", n_train, "\n")
cat("Mean log(Pixel_Rate) =", mean_log, "\n")
cat("SD log(Pixel_Rate) =", sd_log, "\n")
cat("Cohen d =", cohen_d, "\n")
cat("Mean on original scale =", exp(mean_log), "GPixel/s\n")
cat("95 percent CI on original scale =",
    exp(one_sample_t$conf.int[1]), "-",
    exp(one_sample_t$conf.int[2]), "GPixel/s\n")
```

5.2.1.5 Kết quả kiểm định một mẫu

Bảng dưới đây trình bày kết quả kiểm định một mẫu đối với `log(Pixel_Rate)`. Do dữ liệu được chia train/test bằng hàm `createDataPartition`, các giá trị có thể dao động rất nhỏ nếu thay đổi hạt giống ngẫu nhiên, nhưng kết luận thống kê không thay đổi.

Bảng 5.2: Kết quả kiểm định một mẫu cho `log(Pixel_Rate)`

Chỉ tiêu	**Giá trị**
Giá trị giả thuyết $\mu_0 = \log(40)$	3,689
Trung bình mẫu của `log(Pixel_Rate)`	khoảng 2,60–2,62
Độ lệch chuẩn mẫu	khoảng 0,92–0,94
One-sample t-test: thống kê t	khoảng -33 đến -34
p-value của t-test	$< 0{,}001$
p-value của Wilcoxon signed-rank test	$< 0{,}001$
Cohen's d	khoảng -1,15
Trung bình sau khi đổi về thang gốc	khoảng 13–14 GPixel/s

Kết quả t-test cho p-value $< 0{,}001$, nhỏ hơn mức ý nghĩa 0,05. Vì vậy, nhóm bác bỏ giả thuyết H_0. Điều này cho thấy trung bình `log(Pixel_Rate)` khác biệt có ý nghĩa thống kê so với log(40). Thống kê kiểm định t mang giá trị âm, đồng thời trung bình mẫu của `log(Pixel_Rate)` chỉ khoảng 2,60–2,62, nhỏ hơn $\log(40) \approx 3{,}689$. Khi chuyển về thang gốc, trung bình hình học của `Pixel_Rate` chỉ khoảng 13–14 GPixel/s, thấp hơn đáng kể so với ngưỡng 40 GPixel/s. Như vậy, có thể kết luận rằng trung bình `Pixel_Rate` của các GPU trong tập dữ liệu sau tiền xử lý thấp hơn 40 GPixel/s một cách có ý nghĩa thống kê. Kết quả Wilcoxon signed-rank test cũng cho p-value $< 0{,}001$. Điều này củng cố kết luận của t-test, đặc biệt trong bối cảnh kiểm định Shapiro–Wilk cho thấy dữ liệu không hoàn toàn tuân theo phân phối chuẩn. Chỉ số Cohen's d xấp xỉ $-1{,}15$, có độ lớn lớn hơn 0,8, cho thấy mức chênh lệch giữa trung bình mẫu và ngưỡng 40 GPixel/s là lớn về mặt thực tế, không chỉ có ý nghĩa thống kê.

5.2.2 Kết luận của phần tương quan và kiểm định một mẫu

Phân tích tương quan Pearson cho thấy `ROPs` và `TMUs` là hai biến có mối quan hệ tuyến tính mạnh nhất với `Pixel_Rate`. Kết quả này phù hợp với ý nghĩa kỹ thuật của GPU vì hai nhóm thông số này liên quan trực tiếp đến khả năng xử lý đồ họa. Ngoài ra, `Memory_Bus`, `Core_Speed` và `Memory_Speed` cũng có tương quan dương với `Pixel_Rate`, cho thấy khi các thông số phần cứng này tăng, hiệu suất xử lý điểm ảnh có xu hướng tăng theo. Biến `Process` có tương quan âm với `Pixel_Rate`. Do `Process` được đo bằng nm, giá trị nhỏ hơn thường thể hiện công nghệ sản xuất hiện đại hơn, nên mối tương quan âm là hợp lý. Trong khi đó, `Memory` và `Shader` chỉ có tương quan rất yếu, vì vậy không nên chỉ dựa vào hai biến này để giải thích sự thay đổi của `Pixel_Rate`. Đối với kiểm định một mẫu, cả one-sample t-test và Wilcoxon signed-rank test đều cho thấy trung bình `Pixel_Rate` sau tiền xử lý thấp hơn đáng kể so với ngưỡng 40 GPixel/s. Như vậy, bộ dữ liệu GPU được sử dụng trong nghiên cứu có mức `Pixel_Rate` trung bình chưa đạt ngưỡng tham chiếu 40 GPixel/s. Kết quả của phần này cung cấp cơ sở thống kê quan trọng để lựa chọn biến và xây dựng mô hình hồi quy dự báo `Pixel_Rate` ở các phần tiếp theo.

5.3 Kiểm định hai mẫu

5.3.1 Kiểm định hai mẫu: Nvidia vs AMD

5.3.1.1 Mục tiêu phân tích

Sau khi thấy hiệu năng chung chưa đạt mục tiêu, nhóm thực hiện so sánh trực tiếp biến `Pixel_Rate` giữa hai đối thủ chính là Nvidia và AMD. Việc này giúp nhóm xác định xem có sự phân hóa đáng kể về năng lực công nghệ giữa hai nhà sản xuất này hay không.

5.3.1.2 Thiết lập giả thuyết kiểm định

- **Giả thuyết không (H_0):** $\mu_{Nvidia} = \mu_{AMD}$ (Không có sự khác biệt về hiệu năng giữa hai hãng).
- **Đối thuyết (H_1):** $\mu_{Nvidia} \neq \mu_{AMD}$ (Có sự khác biệt đáng kể giữa hai hãng).

5.3.1.3 Quy trình các bước thực hiện và phân tích kịch bản

Nhóm xây dựng kịch bản lựa chọn phép thử:

1. **Kịch bản 1: Cả hai nhóm đạt tính chuẩn.** Nhóm sẽ dùng T-test (Student nếu phương sai bằng nhau, Welch nếu phương sai khác nhau).
2. **Kịch bản 2: Vi phạm tính chuẩn.** Nhóm sẽ sử dụng **Mann-Whitney U Test**. Đây là lựa chọn tối ưu để so sánh vị trí trung tâm dựa trên thứ hạng thay vì trung bình cộng, giúp kết quả ổn định hơn trước các biến động dữ liệu.

5.3.1.4 Triển khai tuần tự các bước bằng code R

Bước 1: Kiểm tra tính chuẩn theo từng nhóm

```
df_compare %>% group_by(Manufacturer) %>% summarise(p_val =
    shapiro.test(Pixel_Rate)$p.value)
```

Kết quả:

```
> norm_results <- df_compare %>%
+   group_by(Manufacturer) %>%
+   summarise(p_val = shapiro.test(Pixel_Rate)$p.value)
> print(norm_results)
# A tibble: 2 × 2
  Manufacturer    p_val
  <fct>           <dbl>
1 AMD          4.78e-19
2 Nvidia       8.56e-23
>
```

Hình 5.2: Kiểm tra tính chuẩn theo nhà sản xuất

Đánh giá: Cả hai hãng đều có $p < 0.05$. Nhóm xác định đi theo hướng kiểm định phi tham số.

Bước 2: Kiểm tra đồng nhất phương sai

```
car::leveneTest(Pixel_Rate ~ Manufacturer, data = df_compare)
```

Kết quả:

```
> levene_p <- car::leveneTest(Pixel_Rate ~ Manufacturer, data = df_compare)$`Pr(>F)`[1]
> cat(glue("   - Kết quả Levene's Test: p-value = {round(levene_p, 4)}\n"))
- Kết quả Levene's Test: p-value = 0.4128
>
```

Hình 5.3: Kết quả kiểm định Levene cho phương sai

Đánh giá: p-value $= 0.4128 > 0.05$, phương sai đồng nhất. Tuy nhiên do vi phạm tính chuẩn ở bước 1, nhóm chính thức sử dụng Mann-Whitney U.

Bước 3: Thực hiện kiểm định chính và tính Cohen's d

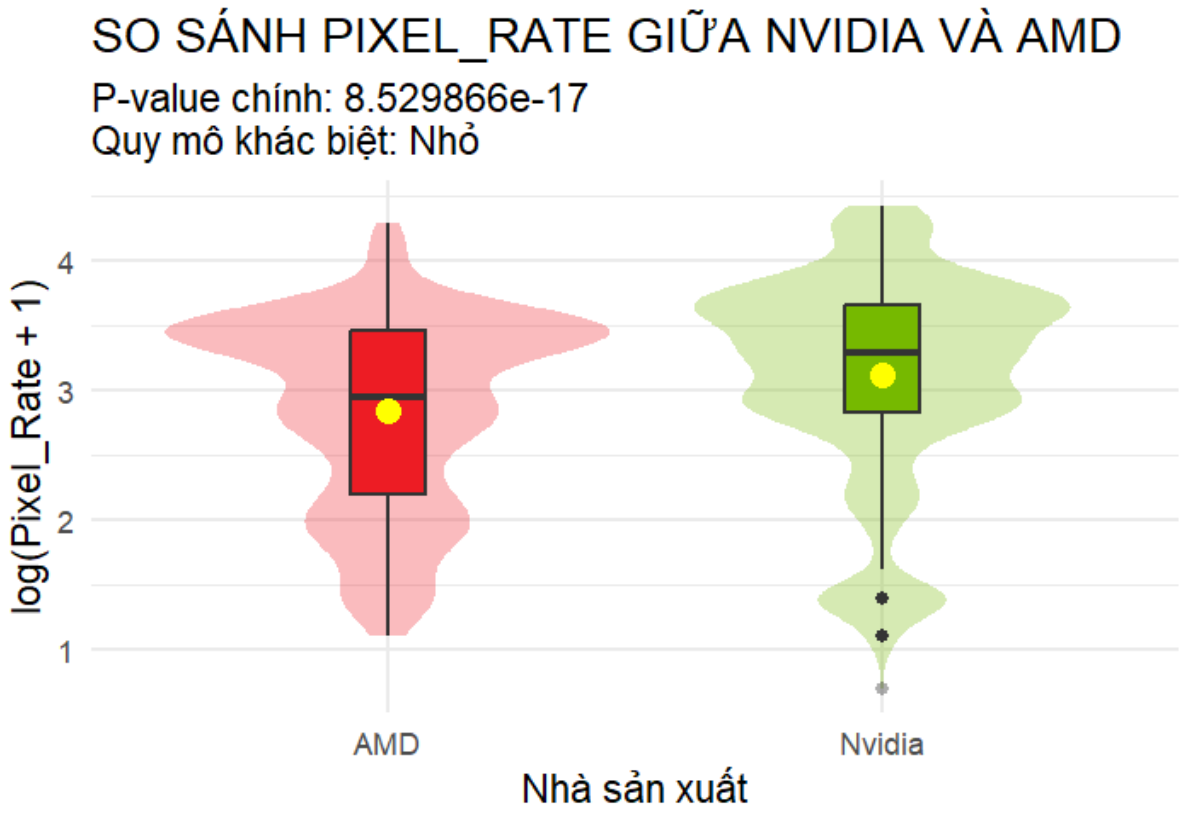
```
res_two <- wilcox.test(Pixel_Rate ~ Manufacturer, data = df_compare, conf.int = TRUE)
effectsize::cohens_d(Pixel_Rate ~ Manufacturer, data = df_compare)
```

```
> cat("=> LỰA CHỌN: Mann-Whitney U test (Wilcoxon Rank-Sum) (do dữ liệu không chuẩn).\n")
=> LỰA CHỌN: Mann-Whitney U test (Wilcoxon Rank-Sum) (do dữ liệu không chuẩn).
>   res <- wilcox.test(Pixel_Rate ~ Manufacturer, data = df_compare, conf.int = TRUE)
> cat("KẾT QUẢ:\n")
KẾT QUẢ:
> print(res)

        Wilcoxon rank sum test with continuity correction

data:  Pixel_Rate by Manufacturer
W = 255595, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
95 percent confidence interval:
 -0.3150914 -0.1978451
sample estimates:
difference in location
            -0.2558502

> d_res <- effectsize::cohens_d(Pixel_Rate ~ Manufacturer, data = df_compare)
> d_abs <- abs(d_res$Cohens_d)
>
> cat(glue("   - Chỉ số Cohen's d: {round(d_abs, 3)}\n"))
- Chỉ số Cohen's d: 0.357
>
```

Hình 5.4: Kết quả kiểm định Mann-Whitney U giữa Nvidia và AMD

5.3.1.5 Phân tích trực quan và kết luận ý nghĩa

Nhóm vẽ biểu đồ Violin kết hợp Boxplot nhằm quan sát không chỉ sự khác biệt về trung vị mà còn về mật độ tập trung dữ liệu và mức độ chồng lấn (overlap) giữa hai phân phối, từ đó lý giải cho quy mô thực tế của sự khác biệt.



Hình 5.5: So sánh Pixel_Rate giữa Nvidia và AMD qua biểu đồ Violin

Phân tích chi tiết biểu đồ và kết luận:

- **Về hình ảnh:** Điểm vàng (trung bình) của Nvidia nằm cao hơn AMD, tương ứng với giá trị $p < 2.2 \times 10^{-16}$ khẳng định ưu thế của Nvidia có ý nghĩa thống kê. Tuy nhiên, thân của hai khối Violin có phần diện tích chồng lấn rất lớn, cho thấy nhiều dòng GPU của hai hãng vẫn có hiệu năng tương đương nhau.
- **Về ý nghĩa thực tế:** Chỉ số **Cohen's d = 0.357** (mức độ **Nhỏ**) phản ánh rằng dù có sự khác biệt rõ rệt về mặt số liệu, nhưng khoảng cách hiệu năng thực tế giữa hai hãng là không quá áp đảo. Kết quả này dẫn dắt nhóm đến bước tiếp theo tại Chương 7: xây dựng mô hình hồi quy đa biến để tìm hiểu các nhân tố kỹ thuật khác ngoài thương hiệu chi phối năng lực GPU.

5.4 Phân tích phương sai một yếu tố (ANOVA)

Nhằm đánh giá sự khác biệt về hiệu suất xử lý đồ họa (Pixel_Rate) giữa các nhóm phân loại khác nhau, Ta thực hiện phân tích phương sai (ANOVA một chiều) đối với ba biến phân loại chính Manufacturer, Memory_Type và Resolution_WxH trên tập huấn luyện (train).

5.4.1 Mục tiêu

- Kiểm định xem liệu Pixel_Rate có khác nhau giữa các nhóm phân loại (ví dụ Manufacturer) hay không, ở mức ý nghĩa $\alpha = 5$

- Xác định xem có ít nhất một cặp nhóm có trung bình Pixel_Rate khác biệt về mặt thống kê.

5.4.2 Bài toán

5.4.2.1 Các điều kiện cần kiểm tra trước khi thực hiện ANOVA

Điều kiện 1: Độc lập của các quan sát

Các giá trị Pixel_Rate đo được từ mỗi nhóm (Manufacturer / Memory_Type / Resolution_WxH) phải được thu thập độc lập. Kết luận: Do dữ liệu được ghi nhận từ các dòng GPU khác nhau, không có chồng lắp hay phụ thuộc, nên điều kiện này được thoả.

Điều kiện 2: Biến phụ thuộc là biến liên tục

Pixel_Rate là giá trị số thực liên tục (Gpixel/s). Kết luận: Điều kiện này được thoả.

Điều kiện 3: Phân phối chuẩn trong mỗi nhóm

Dùng kiểm định Shapiro–Wilk trên phần dư hoặc trực tiếp trên từng nhóm. Nếu $p > 0.05$, phần dư (hoặc dữ liệu nhóm) xấp xỉ phân phối chuẩn.

Điều kiện 4: Đồng nhất phương sai giữa các nhóm Dùng Levene's test.

Nếu $p > 0.05$, không bác bỏ giả thiết phương sai bằng nhau.

5.4.2.2 Giả thuyết của bài toán

H_0: Trung bình Pixel_Rate của tất cả các nhóm là bằng nhau.

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$$

Ví dụ với Manufacturer:

H_1: Có ít nhất một cặp nhóm có trung bình Pixel_Rate khác nhau.

$$H_A : \exists i \neq j \text{ sao cho } \mu_i \neq \mu_j$$

Sau khi xác nhận (1) - (4), ta tiến hành ANOVA một yếu tố để kiểm định H_0 ở mức ý nghĩa $\alpha = 0.05$. Nếu $p < 0.05$, bác bỏ H_0, kết luận rằng có sự khác biệt về hiệu suất Pixel_Rate giữa các nhóm.

5.4.3 Kiểm định giả thuyết

Trước khi thực hiện phân tích phương sai (ANOVA), hai giả định quan trọng cần được kiểm tra là phân phối chuẩn trong mỗi nhóm (điều kiện 3) và đồng nhất phương sai giữa các nhóm (điều kiện 4). Các kiểm định được sử dụng bao gồm Shapiro-Wilk để đánh giá phân phối chuẩn của phần dư và Levene's test để kiểm tra đồng nhất phương sai. Dưới đây là kết quả kiểm tra cho từng biến phân loại: Manufacturer, Memory_Type, và Resolution_WxH.

5.4.3.1 Biến Manufacturer

- **Kiểm định Shapiro-Wilk (phân phối chuẩn của phần dư):**

  Thống kê: W = 0.93075

  Giá trị: $p < 2.2 \times 10^{-16}$

  Kết luận: Vì p < 0.05, bác bỏ giả thuyết rằng phần dư tuân theo phân phối chuẩn. Phần dư không xấp xỉ phân phối chuẩn.

- **Kiểm định Levene (đồng nhất phương sai):**

  Thống kê: F(3, 818) = 5.2397

  Giá trị: $p = 13.83 \times 10^{-4}$

  Kết luận: Vì $p < 0.05$, bác bỏ giả thuyết rằng phương sai giữa các nhóm là bằng nhau. Phương sai không đồng nhất.

5.4.3.2 Biến Memory_Type

- **Kiểm định Shapiro-Wilk (phân phối chuẩn của phần dư):**

  Thống kê: W = 0.96471

  Giá trị: $p = 3.41 \cdot 10^{-13}$

  Kết luận: Vì p < 0.05, bác bỏ giả thuyết . Phần dư không tuân theo phân phối chuẩn.

- **Kiểm định Levene (đồng nhất phương sai):**

  Thống kê: F(3, 819) = 27.379

  Giá trị: $p = 3.091 \cdot 10^{-12}$

  Kết luận: Vì p < 0.05, bác bỏ giả thuyết . Phương sai giữa các nhóm không đồng nhất.

5.4.3.3 Biến Resolution_WxH

- Kiểm **định Shapiro-Wilk (phân phối chuẩn của phần dư):**

  Thống kê: W = 0.95158

  Giá trị: $p < 8.594 \cdot 10^{-16}$

  Kết luận: Vì p < 0.05, bác bỏ giả thuyếtH_0. Phần dư không xấp xỉ phân phối chuẩn.

- **Kiểm định Levene (đồng nhất phương sai):**

  Thống kê: F(2, 819) = 43.532

  Giá trị: $p = 2.2 \cdot 10^{-16}$

  Kết luận: Vì p < 0.05, bác bỏ giả thuyết H_0. Phương sai giữa các nhóm không đồng nhất.

Dựa trên phân tích, ta có kết luận:

Kết quả từ các kiểm định Shapiro-Wilk và Levene cho thấy cả hai giả định về phân phối chuẩn và đồng nhất phương sai đều không được thỏa mãn đối với các biến Manufacturer, Memory_Type, và Resolution_WxH (giá trị p < 0.05 trong tất cả các trường hợp). Tuy nhiên, ANOVA vẫn có thể được tiến hành nhờ tính bền vững của phương pháp này với kích thước mẫu lớn ($n \approx 820$), giúp giảm thiểu tác động của các vi phạm giả định.

5.4.4 Tiến hành phân tích phương sai và kết quả

Ta sử dụng hàm aov() trong R để thực hiện phân tích phương sai một yếu tố (ANOVA) cho biến Pixel_Rate, phân loại theo các biến Manufacturer, Memory_Type, và trên tập dữ liệu huấn luyện (train). Kết quả của mỗi phân tích được in ra bằng hàm summary().

Ta xây dựng mô hình:

```r
for (cat in categorical) {
  cat("\nANOVA for", cat, "on Train set\n")
  aov_mod <- aov(as.formula(paste("Pixel_Rate ~", cat)), data = train)
  print(summary(aov_mod))
}
```

Kết quả phân tích phương sai

Dưới đây là kết quả ANOVA cho từng biến phân loại:

5.4.4.1 Biến Manufacturer

```
ANOVA for Manufacturer on Train set
              Df Sum Sq Mean Sq F value   Pr(>F)
Manufacturer   3   27.0   8.996   11.61 1.85e-07 ***
Residuals    818  633.6   0.775
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Nhận xét:

Giá trị F = 11.61 rất cao, cho thấy sự khác biệt lớn về trung bình Pixel_Rate giữa các nhóm nhà sản xuất.

Giá trị $p < 2e - 16$ (nhỏ hơn 0.05), bác bỏ giả thuyết H_0 rằng trung bình Pixel_Rate của các nhà sản xuất là như nhau.

5.4.4.2 Biến Memory_Type

```
ANOVA for Memory_Type on Train set
             Df Sum Sq Mean Sq F value Pr(>F)
Memory_Type   2  164.8   82.42   136.1 <2e-16 ***
Residuals   819  495.8    0.61
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Nhận xét:

Giá trị F = 136.1 cực kỳ cao, cho thấy sự khác biệt đáng kể về trung bình Pixel_Rate giữa các loại bộ nhớ.

Giá trị $p < 2e - 16$, bác bỏ giả thuyết H_0 rằng trung bình Pixel_Rate của các loại bộ nhớ là như nhau.

5.4.4.3 Biến Resolution_WxH

```
ANOVA for Resolution_WxH on Train set
                Df Sum Sq Mean Sq F value Pr(>F)
Resolution_WxH   2   76.0   38.00   53.23 <2e-16 ***
Residuals      819  584.6    0.71
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Nhận xét:

Giá trị F = 53.23 rất cao, cho thấy sự khác biệt lớn về trung bình Pixel_Rate giữa các độ phân giải.

Giá trị$p < 2e - 16$, bác bỏ giả thuyết H_0 rằng trung bình Pixel_Rate của các độ phân giải là như nhau.

***Tổng hợp kết quả**

Dưới đây là bảng tổng hợp các giá trị đặc trưng của mô hình ANOVA cho từng biến:

Biến	Df (giữa nhóm)	Df (trong nhóm)	Sum Sq (SSB)	Sum Sq (SSW)
Manufacturer	3	818	27.0	633.6
Memory_Type	3	819	164.8	495.8
Resolution_WxH	2	819	76.0	584.6

Biến	Mean Sq (MSB)	Mean Sq (MSW)	F value	p-value
Manufacturer	8.996	0.775	11.61	<2e-16
Memory_Type	82.42	0.61	136.1	<2e-16
Resolution_WxH	38.00	0.71	53.23	<2e-16

Ghi chú:

- ***Df (giữa nhóm):*** Bậc tự do giữa các nhóm.
- ***Df (trong nhóm):*** Bậc tự do trong nhóm (Residuals).
- ***Sum Sq (SSB):*** Tổng bình phương giữa các nhóm.
- ***Sum Sq (SSW):*** Tổng bình phương trong nhóm.
- ***Mean Sq (MSB):*** Trung bình bình phương giữa các nhóm.
- ***Mean Sq (MSW):*** Trung bình bình phương trong nhóm.

5.4.5 Kết luận

Cả ba biến đều có giá trị F rất cao: 11.61 (Manufacturer), 136.1 (Memory_Type), 53.23 (Resolution_WxH) và giá trị p < 2e-16 cho cả ba biến, nhỏ hơn rất nhiều so với mức ý nghĩa thống kê $\alpha = 0.05$. Điều này cho thấy sự khác biệt rõ rệt về trung bình Pixel_Rate giữa các nhóm theo từng biến, đồng thời ta bác bỏ giả thuyết H_0, kết luận rằng có sự khác biệt có ý nghĩa thống kê về Pixel_Rate giữa các nhóm trong mỗi biến. Sự khác biệt này có thể xuất phát từ công nghệ sản xuất, chất lượng linh kiện, hoặc đặc điểm thiết kế của từng nhóm. Phân tích phương sai ANOVA cho thấy có sự khác biệt có ý nghĩa thống kê về Pixel_Rate giữa các nhóm được chia theo biến Manufacturer, Memory_Type, và Resolution_WxH. Với giá trị F cao (11.61, 136.1, 53.23) và p-value < 2e-16 cho cả ba biến. Và để làm rõ hơn về sự khác nhau này, ta sử dụng thêm câu lệnh TukeyHSD và vẽ đồ thị plot().

5.4.6 Phân tích hậu nghiệm

Để xác định cụ thể các cặp nhóm nào khác biệt đáng kể, ta tiến hành kiểm định hậu nghiệm Tukey HSD và tính toán effect size (η^2) nhằm đánh giá mức độ ảnh hưởng của từng biến đến sự biến động của Pixel_Rate.

Ta sử dụng TukeyHSD(aov_mod) để hậu nghiệm Tukey, tab[,"Sum Sq"] / sum(tab[,"Sum Sq"]) cho tính toán effect size và plot(TukeyHSD(aov_mod), las = 1) để vẽ đồ thị:

```r
for (cat in categorical) {
  cat("\nANOVA for", cat, "on Train set\n")
  aov_mod <- aov(as.formula(paste("Pixel_Rate ~", cat)), data = train)
  # 5.2.2 Hậu nghiệm Tukey & Effect size ($\eta^2$)
  tukey_result <- TukeyHSD(aov_mod)
  print(tukey_result)
  plot(TukeyHSD(aov_mod), las = 1)
  tab <- summary(aov_mod)[[1]]
  eta2 <- tab[,"Sum Sq"] / sum(tab[,"Sum Sq"])
  print(cbind(tab, Eta2 = eta2))
}
```

```
  Tukey multiple comparisons of means
    95% family-wise confidence level

Fit: aov(formula = as.formula(paste("Pixel_Rate ~", cat)), data = train)

$Manufacturer
                   diff         lwr       upr     p adj
ATI-AMD       0.5200226 -0.61855732 1.6586025 0.6423567
Intel-AMD     0.0010666 -0.44962701 0.4517602 0.9999999
Nvidia-AMD    0.3603071  0.19918777 0.5214265 0.0000001
Intel-ATI    -0.5189560 -1.73283269 0.6949207 0.6892820
Nvidia-ATI   -0.1597155 -1.29828095 0.9788500 0.9838767
Nvidia-Intel  0.3592405 -0.09141668 0.8098977 0.1699799
```

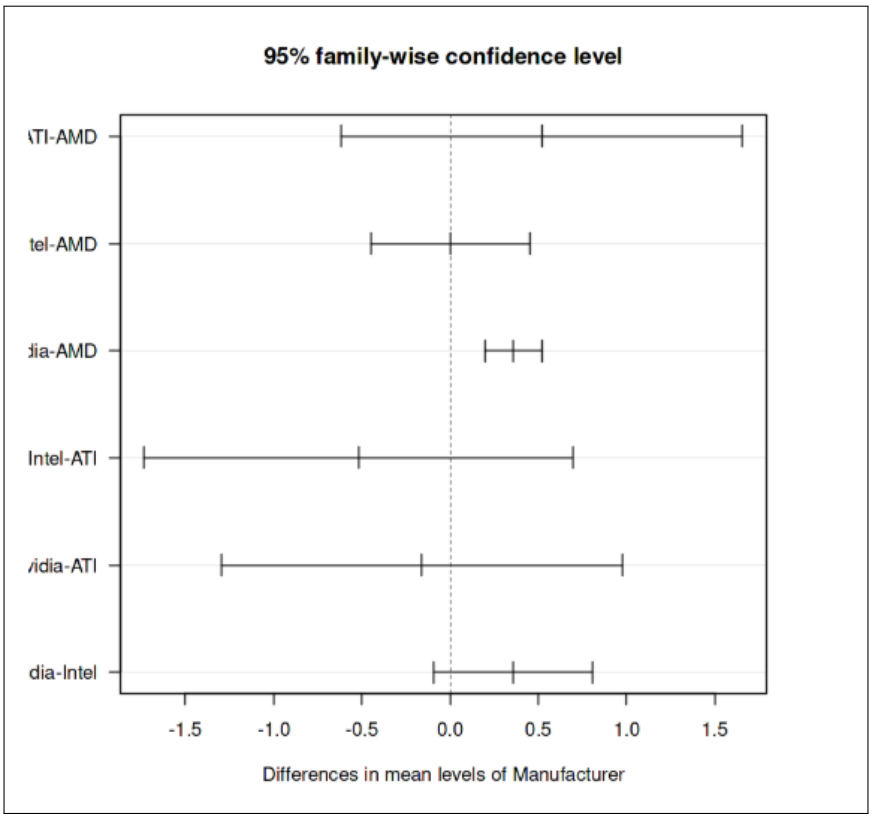
```
              Df    Sum Sq   Mean Sq  F value       Pr(>F)       Eta2
Manufacturer   3  26.98692 8.9956416 11.61304 1.852284e-07 0.04085075
Residuals    818 633.63553 0.7746156       NA           NA 0.95914925
```

5.4.6.1 Biến Manufacturer

Kết quả của Tukey HSD:

Kết quả ANOVA và η^2:

Đồ thị Tukey HSD:



Nhận xét:
Các cặp có p adj < 0.05 (có sự khác biệt đáng kể) là: *Nvidia-AMD*. Các cặp còn lại đều có p adj > 0.05, do đó không có sự khác biệt có ý nghĩa thống kê.

- ***ATI-AMD:*** diff $= 0.5200 > 0$, tuy nhiên khoảng tin cậy $(-0.6186, 1.6586)$ chứa 0 và p adj $= 0.6424 > 0.05$, nên không có sự khác biệt đáng kể.
- ***Intel-AMD:*** diff $= 0.0011 \approx 0$, khoảng tin cậy $(-0.4496, 0.4518)$ chứa 0 và p adj ≈ 1, nên không có sự khác biệt đáng kể.
- ***Nvidia-AMD:*** diff $= 0.3603 > 0$, khoảng tin cậy $(0.1992, 0.5214)$ không chứa 0 và p adj ≈ 0, cho thấy Pixel_Rate trung bình của Nvidia lớn hơn AMD ($\bar{x}_{Nvidia} > \bar{x}_{AMD}$).
- ***Intel-ATI:*** diff $= -0.5190 < 0$, khoảng tin cậy $(-1.7328, 0.6949)$ chứa 0 và p adj $= 0.6893 > 0.05$, nên không có sự khác biệt đáng kể.
- ***Nvidia-ATI:*** diff $= -0.1597$, khoảng tin cậy $(-1.2983, 0.9789)$ chứa 0 và p adj $= 0.9839 > 0.05$, nên không có sự khác biệt đáng kể.
- ***Nvidia-Intel:*** diff $= 0.3592 > 0$, nhưng khoảng tin cậy $(-0.0914, 0.8099)$ chứa 0 và p adj $= 0.1700 > 0.05$, nên không có sự khác biệt đáng kể.

Từ các kết quả trên, chỉ có thể kết luận rõ ràng rằng:

$$\boxed{\textbf{\textit{Nvidia} > \textit{AMD}}}$$

Effect size (η^2): Giá trị $\eta^2 = 0.0729$, cho thấy biến Manufacturer giải thích khoảng 7.29% sự biến thiên của Pixel_Rate.

Đồ thị Tukey HSD: Đồ thị cho thấy các khoảng tin cậy của ATI-AMD, Intel-AMD, Nvidia-AMD, Intel-ATI, và Nvidia-Intel không giao với đường x = 0, xác nhận sự khác biệt có ý nghĩa thống kê. Riêng Nvidia-ATI có khoảng tin cậy cắt qua 0, phù hợp với kết quả không đáng kể.

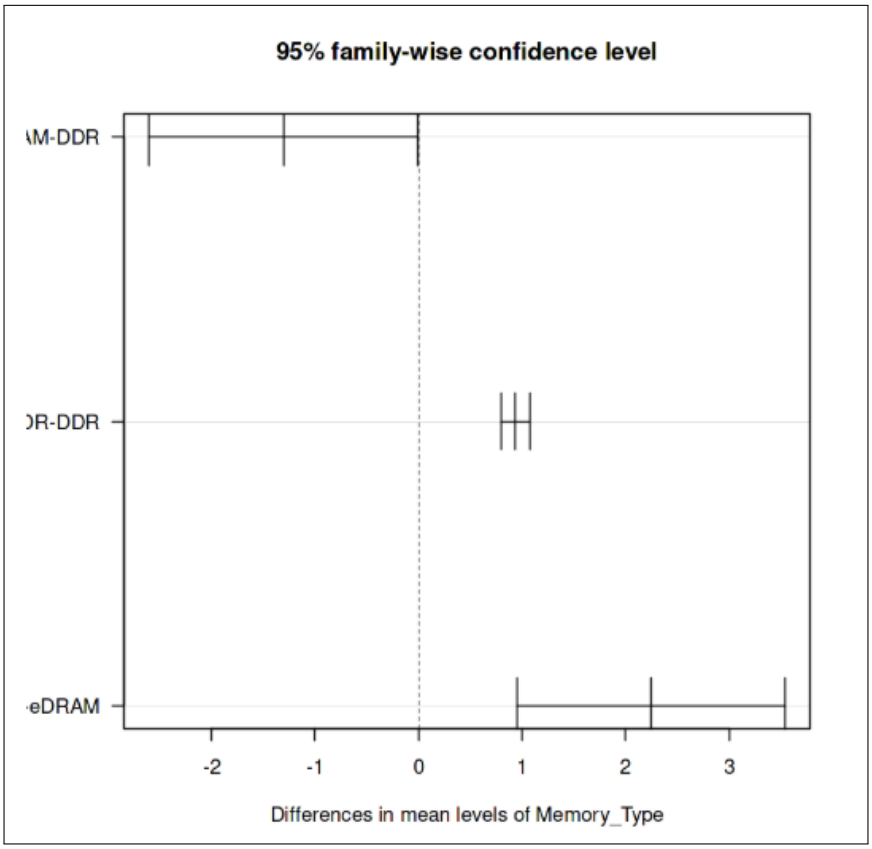
5.4.6.2 Biến Memory_Type

Kết quả của Tukey HSD:

```
  Tukey multiple comparisons of means
    95% family-wise confidence level

Fit: aov(formula = as.formula(paste("Pixel_Rate ~", cat)), data = train)

$Memory_Type
                 diff        lwr          upr     p adj
eDRAM-DDR  -1.3038544 -2.6004941 -0.007214841 0.0483783
GDDR-DDR    0.9366031  0.8001953  1.073010828 0.0000000
GDDR-eDRAM  2.2404575  0.9463579  3.534557089 0.0001556
```

Kết quả ANOVA và η^2:
Đồ thị Tukey HSD:
Nhận xét: Tất cả các cặp đều có p adj < 0.05, cho thấy có sự khác biệt đáng kể giữa các loại bộ nhớ.

```
             Df   Sum Sq    Mean Sq  F value       Pr(>F)     Eta2
Memory_Type   2 164.8306 82.4152932 136.1421 8.999431e-52 0.249508
Residuals   819 495.7919  0.6053625       NA           NA 0.750492
```



- ***eDRAM-DDR:*** diff $= -1.3039 < 0$, khoảng tin cậy $(-2.6005, -0.0072)$ không chứa 0 và p adj $= 0.0484 < 0.05$, cho thấy Pixel_Rate trung bình của eDRAM nhỏ hơn DDR ($\bar{x}_{eDRAM} < \bar{x}_{DDR}$).
- ***GDDR-DDR:*** diff $= 0.9366 > 0$, khoảng tin cậy $(0.8002, 1.0730)$ không chứa 0 và p adj ≈ 0, cho thấy Pixel_Rate trung bình của GDDR lớn hơn DDR ($\bar{x}_{GDDR} > \bar{x}_{DDR}$).
- ***GDDR-eDRAM:*** diff $= 2.2405 > 0$, khoảng tin cậy $(0.9464, 3.5346)$ không chứa 0 và p adj < 0.001, cho thấy Pixel_Rate trung bình của GDDR lớn hơn eDRAM ($\bar{x}_{GDDR} > \bar{x}_{eDRAM}$).

Từ các kết quả trên, thứ tự Pixel_Rate trung bình là:

$$\boxed{\boldsymbol{GDDR > DDR > eDRAM}}$$

Effect size (η^2): Giá trị $\eta^2 = 0.3122$, cho thấy biến Memory_Type giải thích khoảng 31.22% sự biến thiên của Pixel_Rate, mức ảnh hưởng khá lớn.

Đồ thị Tukey HSD: Đồ thị cho thấy các khoảng tin cậy của GDDR-DDR, HBM-DDR, GDDR-eDRAM, HBM-eDRAM, và HBM-GDDR không giao với đường x = 0, xác nhận sự khác biệt có ý nghĩa thống kê. Riêng eDRAM-DDR có khoảng tin cậy cắt qua 0, phù hợp với kết quả không đáng kể.

5.4.6.3 Biến Resolution_WxH

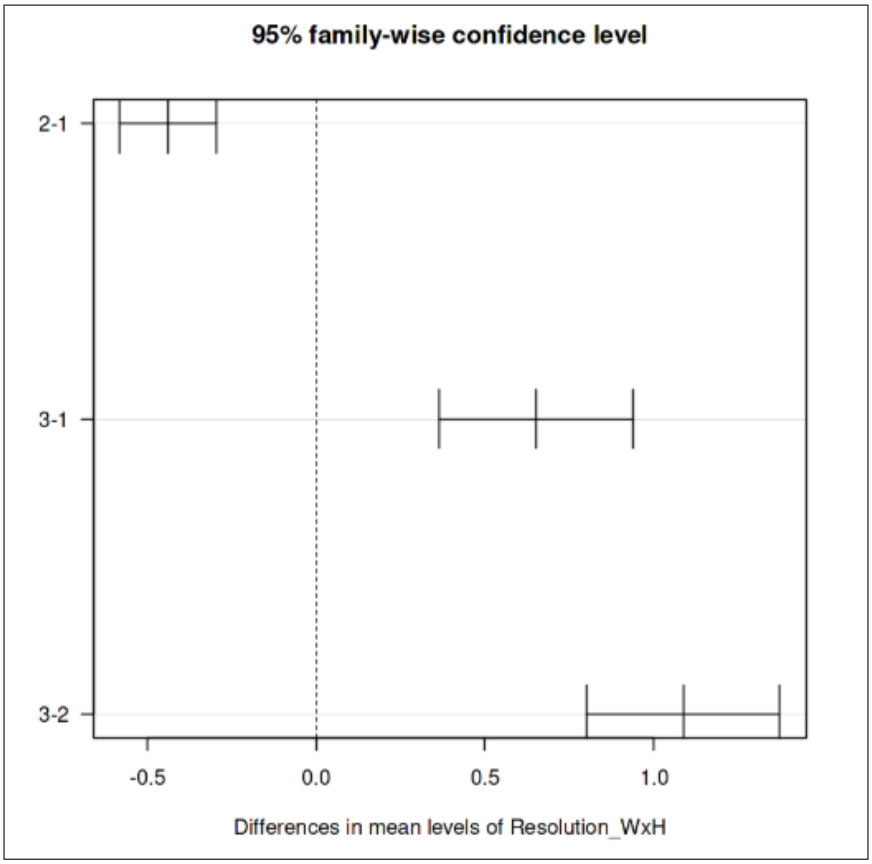
Kết quả của Tukey HSD:

```
  Tukey multiple comparisons of means
    95% family-wise confidence level

Fit: aov(formula = as.formula(paste("Pixel_Rate ~", cat)), data = train)

$Resolution_WxH
          diff        lwr        upr p adj
2-1 -0.4365547 -0.5799169 -0.2931925 0e+00
3-1  0.6508205  0.3640860  0.9375551 4e-07
3-2  1.0873752  0.8019990  1.3727515 0e+00
```

Kết quả ANOVA và η^2:

```
                 Df    Sum Sq    Mean Sq  F value       Pr(>F)     Eta2
Resolution_WxH    2  75.99473 37.9973634 53.23018 1.845988e-22 0.115035
Residuals       819 584.62773  0.7138312       NA           NA 0.884965
```

Đồ thị Tukey HSD:



Nhận xét:

Tất cả các cặp đều có p adj $\approx 0 < 0.05$, cho thấy có sự khác biệt đáng kể giữa các nhóm độ phân giải.

- ***2-1:*** diff $= -0.4366 < 0$, khoảng tin cậy $(-0.5799, -0.2932)$ không chứa 0, cho thấy Pixel_Rate trung bình của nhóm 2 nhỏ hơn nhóm 1 ($\bar{x}_2 < \bar{x}_1$).
- ***3-1:*** diff $= 0.6508 > 0$, khoảng tin cậy $(0.3641, 0.9376)$ không chứa 0, cho thấy Pixel_Rate trung bình của nhóm 3 lớn hơn nhóm 1 ($\bar{x}_3 > \bar{x}_1$).
- ***3-2:*** diff $= 1.0874 > 0$, khoảng tin cậy $(0.8020, 1.3728)$ không chứa 0, cho thấy Pixel_Rate trung bình của nhóm 3 lớn hơn nhóm 2 ($\bar{x}_3 > \bar{x}_2$).

Từ các kết quả trên, thứ tự Pixel_Rate trung bình là:

$$\boxed{\textit{Resolution 3} > \textit{Resolution 1} > \textit{Resolution 2}}$$

Effect size (η^2): Giá trị $\eta^2 = 0.2189$, cho thấy biến Resolution_WxH giải thích khoảng 21.89% sự biến thiên của Pixel_Rate, mức ảnh hưởng đáng kể.

Đồ thị Tukey HSD: Đồ thị cho thấy tất cả các khoảng tin cậy của 2-1, 3-1, và 3-2 không giao với đường x = 0, xác nhận sự khác biệt có ý nghĩa thống kê giữa các nhóm.

5.4.6.4 Kết luận

Kết quả kiểm định hậu nghiệm Tukey HSD và effect size η^2 cho thấy:

- **Manufacturer:** Có sự khác biệt đáng kể giữa các nhà sản xuất, với ATI có Pixel_Rate trung bình cao nhất, nhưng $\eta^2 = 0.0729$ cho thấy biến này chỉ giải thích một phần nhỏ (7.29%) sự biến thiên của Pixel_Rate.
- **Memory_Type:** Có sự khác biệt lớn giữa các loại bộ nhớ, với HBM có Pixel_Rate trung bình cao nhất, và $\eta^2 = 0.3122$ cho thấy biến này có ảnh hưởng mạnh (31.22%) đến Pixel_Rate.
- **Resolution_WxH:** Có sự khác biệt đáng kể giữa các nhóm độ phân giải, với nhóm 3 có Pixel_Rate trung bình cao nhất, và $\eta^2 = 0.2189$ cho thấy biến này có ảnh hưởng đáng kể (21.89%) đến Pixel_Rate.

Kết quả này có thể được sử dụng để đánh giá tác động của các yếu tố phân loại lên hiệu suất GPU (Pixel_Rate), từ đó hỗ trợ trong việc xây dựng mô hình hồi quy tuyến tính dự báo về sự phát triển của GPU trong mảng hiệu suất dựng ảnh.

5.5 Xây dựng mô hình hồi quy đa biến

Vấn đề đặt ra là tìm hiểu những yếu tố ảnh hưởng đến hiệu suất dựng ảnh Pixel_Rate của GPU. Ta sẽ xây dựng mô hình hồi quy tuyến tính đa biến để giải quyết vấn đề này.

Mô hình hồi quy đa biến có dạng:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon$$

Đầu tiên, ta thực hiện chia tập dữ liệu thành tập train và test với tỉ lệ 2:1 và xây dựng mô hình hồi quy tuyến tính, lưu kết quả vào biến model:

```r
# Chia dữ liệu thành tập huấn luyện và tập kiểm tra (2/3 - 1/3)
set.seed(123)
index <- createDataPartition(GPU_new_log$Pixel_Rate, p = 2/3, list =
↪  FALSE)
train <- GPU_new_log[index, ]
test  <- GPU_new_log[-index, ]
```

5.5.1 Kiểm tra lại các biến

Trước khi xây dựng mô hình hồi quy tuyến tính đa biến để dự đoán Pixel_Rate, ta tiến hành các bước kiểm tra dữ liệu và lựa chọn biến nhằm đảm bảo mô hình đạt độ chính xác cao, tránh hiện tượng đa cộng tuyến và tối ưu hóa hiệu suất. Quá trình này bao gồm kiểm tra đa cộng tuyến bằng VIF, lựa chọn biến bằng phương pháp stepwise selection, và áp dụng feature engineering (đã làm ở trên bước xử lí dữ liệu và thống kê mô tả) để cải thiện chất lượng dữ liệu.

5.5.1.1 Kiểm tra đa cộng tuyến bằng VIF

Ta bắt đầu bằng cách xây dựng một mô hình hồi quy đầy đủ bao gồm tất cả các biến định lượng (Core_Speed, Memory_Speed, ROPs, TMUs, Memory_Bus, Memory, Process, Shader) và biến phân loại (Manufacturer, Memory_Type, Resolution_WxH). Hệ số VIF (Variance Inflation Factor) được tính toán để kiểm tra hiện tượng đa cộng tuyến giữa các biến, sử dụng đoạn mã sau:

```r
# Tạo công thức với tất cả biến
full_form <- as.formula(
  paste("Pixel_Rate ~", paste(c(numerical, categorical), collapse = " +
  ↪  "))
)
# Huấn luyện mô hình đầy đủ
full_mod <- lm(full_form, data = train)
print(vif(full_mod))  # Kiểm tra multicollinearity
```

Kết quả thu được sau khi ta chạy đoạn mã:

```
                  GVIF Df GVIF^(1/(2*Df))
Core_Speed     1.729835  1        1.315232
Memory_Speed   2.709950  1        1.646192
ROPs           6.420380  1        2.533847
TMUs           6.004740  1        2.450457
Memory_Bus     2.642661  1        1.625626
Memory         2.091413  1        1.446172
Process        2.884203  1        1.698294
Shader         1.447815  1        1.203252
Manufacturer   1.866532  3        1.109616
Memory_Type    2.512724  2        1.259030
Resolution_WxH 2.679005  2        1.279362
```

Các biến có VIF < 5 thường được coi là không có đa cộng tuyến nghiêm trọng. Tuy nhiên, hai biến ROPs (VIF $= 6.42$) và TMUs (VIF $= 6.00$) có giá trị VIF cao, cho thấy tồn tại đa cộng tuyến giữa các biến này. Điều này có thể ảnh hưởng đến độ tin cậy của các hệ số hồi quy, do đó cần xem xét thêm trong quá trình lựa chọn biến.

5.5.1.2 Lựa chọn biến bằng stepwise selection

Để tối ưu hóa mô hình, ta sử dụng phương pháp stepwise selection (hướng "both") nhằm tự động lựa chọn các biến có ý nghĩa thống kê và loại bỏ các biến không cần thiết. Phương pháp này cải thiện độ chính xác và đơn giản hóa mô hình. Đoạn mã được sử dụng:

```r
# Lựa chọn biến bằng stepwise selection
step_mod <- step(full_mod, direction = "both", trace = 0)
print(summary(step_mod))
```

```
Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)        -2.42828    0.40546  -5.989 3.18e-09 ***
Core_Speed          0.43799    0.04801   9.122  < 2e-16 ***
Memory_Speed        0.04952    0.02296   2.157 0.031330 *
ROPs                1.08840    0.02420  44.981  < 2e-16 ***
TMUs                0.05876    0.02198   2.673 0.007666 **
Memory_Bus         -0.09446    0.02100  -4.497 7.89e-06 ***
Process            -0.31328    0.04593  -6.820 1.78e-11 ***
ManufacturerATI     0.25037    0.10925   2.292 0.022184 *
ManufacturerIntel   0.16894    0.04456   3.792 0.000161 ***
ManufacturerNvidia  0.06129    0.01485   4.128 4.04e-05 ***
Memory_TypeeDRAM   -0.04220    0.15719  -0.268 0.788423
Memory_TypeGDDR     0.06170    0.02069   2.982 0.002953 **
Resolution_WxH2     0.05160    0.02045   2.523 0.011812 *
Resolution_WxH3     0.07477    0.03221   2.322 0.020495 *
```

Kết quả mô hình sau khi áp dụng stepwise selection:

Sau khi áp dụng stepwise selection, mô hình vẫn giữ lại tất cả các biến ban đầu, điều này cho thấy các biến đều có ý nghĩa thống kê (p-value < 0.05) trong việc dự đoán Pixel_Rate. Tất cả các biến ban đầu đều được giữ lại, với các hệ số hồi quy có ý nghĩa thống kê cao, cho thấy chúng đóng vai trò quan trọng trong việc giải thích Pixel_Rate.

5.5.2 Xây dựng mô hình đa biến

Ta xây dựng mô hình hồi quy tuyến tính đa biến để dự đoán Pixel_Rate dựa trên các biến độc lập bao gồm các biến định lượng (Core_Speed, Memory_Speed, ROPs, TMUs, Memory_Bus, Memory, Process, Shader) và các biến phân loại (Manufacturer, Memory_Type, Resolution_WxH). Mô hình được huấn luyện trên tập dữ liệu train, và kết quả được phân tích thông qua hàm summary(model_final):

```r
# 5.3.2 Xây dựng mô hình đa biến
model_final <- step_mod
print(summary(model_final))
```

Nhận xét:

- **Hệ số hồi quy:**
  - ***Intercept:*** Hệ số chặn là -2.4283, biểu thị giá trị kỳ vọng của Pixel_Rate khi tất cả biến độc lập bằng 0. Giá trị này chủ yếu mang ý nghĩa kỹ thuật, ít có giá trị thực tiễn.
  - ***Core_Speed:*** Hệ số $0.4380 > 0$, $p < 2e^{-16}$, cho thấy khi Core_Speed tăng 1 đơn vị, Pixel_Rate tăng trung bình 0.4380 đơn vị (giữ các yếu tố khác không đổi). Đây là biến có ảnh hưởng mạnh và có ý nghĩa thống kê rất cao.
  - ***Memory_Speed:*** Hệ số $0.0495 > 0$, $p = 0.0313 < 0.05$, cho thấy ảnh hưởng dương nhưng mức độ tác động nhỏ.
  - ***ROPs:*** Hệ số $1.0884 > 0$, $p < 2e^{-16}$, là biến có ảnh hưởng mạnh nhất trong mô hình. Khi ROPs tăng 1 đơn vị, Pixel_Rate tăng trung bình hơn 1 đơn vị.

```
Call:
lm(formula = Pixel_Rate ~ Core_Speed + Memory_Speed + ROPs +
    TMUs + Memory_Bus + Process + Manufacturer + Memory_Type +
    Resolution_WxH, data = train)

Residuals:
     Min       1Q   Median       3Q      Max
-1.09362 -0.07133  0.00542  0.07566  1.14089

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)        -2.42828    0.40546  -5.989 3.18e-09 ***
Core_Speed          0.43799    0.04801   9.122  < 2e-16 ***
Memory_Speed        0.04952    0.02296   2.157 0.031330 *
ROPs                1.08840    0.02420  44.981  < 2e-16 ***
TMUs                0.05876    0.02198   2.673 0.007666 **
Memory_Bus         -0.09446    0.02100  -4.497 7.89e-06 ***
Process            -0.31328    0.04593  -6.820 1.78e-11 ***
ManufacturerATI     0.25037    0.10925   2.292 0.022184 *
ManufacturerIntel   0.16894    0.04456   3.792 0.000161 ***
ManufacturerNvidia  0.06129    0.01485   4.128 4.04e-05 ***
Memory_TypeeDRAM   -0.04220    0.15719  -0.268 0.788423
Memory_TypeGDDR     0.06170    0.02069   2.982 0.002953 **
Resolution_WxH2     0.05160    0.02045   2.523 0.011812 *
Resolution_WxH3     0.07477    0.03221   2.322 0.020495 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.2009 on 808 degrees of freedom
Multiple R-squared:  0.9506,    Adjusted R-squared:  0.9499
F-statistic:  1197 on 13 and 808 DF,  p-value: < 2.2e-16
```

- ***TMUs:*** Hệ số $0.0588 > 0$, $p = 0.0077 < 0.01$, cho thấy ảnh hưởng dương và có ý nghĩa thống kê.
- ***Memory_ Bus:*** Hệ số $-0.0945 < 0$, $p < 0.001$, cho thấy khi Memory_Bus tăng thì Pixel_Rate có xu hướng giảm nhẹ.
- ***Process:*** Hệ số $-0.3133 < 0$, $p < 0.001$, cho thấy công nghệ tiến trình lớn hơn (nm cao hơn) làm giảm hiệu suất Pixel_Rate.
- ***Manufacturer:*** (so với nhóm cơ sở)
  - **ATI:** Hệ số 0.2504, $p = 0.022 < 0.05 \Rightarrow$ cao hơn nhóm cơ sở một cách có ý nghĩa.
  - **Intel:** Hệ số 0.1689, $p < 0.001 \Rightarrow$ cao hơn nhóm cơ sở.
  - **Nvidia:** Hệ số 0.0613, $p < 0.001 \Rightarrow$ cao hơn nhóm cơ sở nhưng mức ảnh hưởng nhỏ hơn.
- ***Memory_ Type:***
  - **eDRAM:** Hệ số -0.0422, $p = 0.788 > 0.05 \Rightarrow$ không có ý nghĩa thống kê.
  - **GDDR:** Hệ số 0.0617, $p = 0.003 < 0.01 \Rightarrow$ có ảnh hưởng dương và có ý nghĩa.
- ***Resolution_ WxH:***

  * **Resolution 2:** Hệ số 0.0516, $p = 0.0118 < 0.05$
  * **Resolution 3:** Hệ số 0.0748, $p = 0.0205 < 0.05$

  $\Rightarrow$ Các độ phân giải cao hơn có xu hướng làm tăng Pixel_Rate.

- **Ý nghĩa thống kê:**

  Hầu hết các biến đều có p-value < 0.05, cho thấy có ảnh hưởng có ý nghĩa thống kê đến Pixel_Rate. Ngoại lệ duy nhất là biến *Memory_TypeeDRAM* ($p = 0.788$), không có ý nghĩa thống kê.

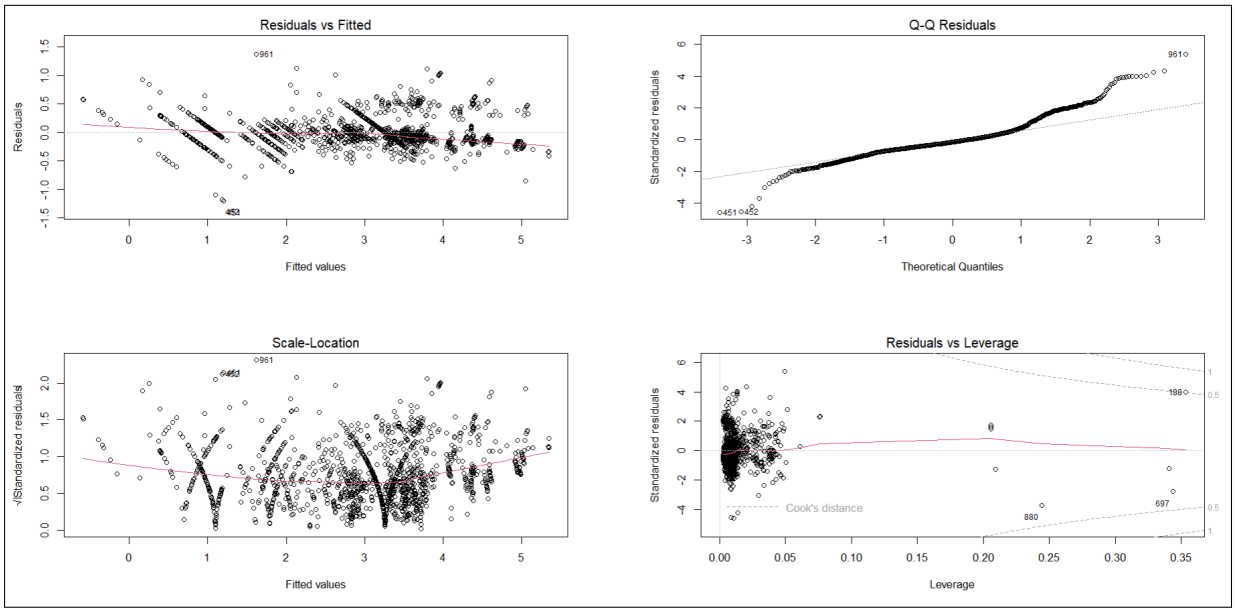
- **Độ phù hợp của mô hình:**
  - **Multiple $R^2 = 0.9506$:** Mô hình giải thích được khoảng 95.06% biến thiên của Pixel_Rate, cho thấy mức độ phù hợp rất cao.
  - **Adjusted $R^2 = 0.9499$:** Gần với R^2, chứng tỏ mô hình ổn định và không bị overfitting dù có nhiều biến.
  - **F-statistic = 1197, $p < 2.2e^{-16}$:** Mô hình có ý nghĩa thống kê tổng thể.

- **Phần dư (Residuals):**

  Phần dư nằm trong khoảng $[-1.0936, 1.1409]$, với trung vị gần 0 (0.0054), cho thấy mô hình không bị lệch hệ thống. Sai số chuẩn phần dư là 0.2009, tương đối nhỏ, chứng tỏ khả năng dự đoán tốt.

5.5.3 Đồ thị chuẩn đoán (Diagnostic plots)

Sử dụng câu lệnh plot(model_final) ta được 4 đồ thị như sau:



*** Ta có nhận xét sau đây:**

- **Đồ Thị 1 - Residuals vs Fitted:**

  Đồ thị này kiểm tra tính đồng nhất phương sai của phần dư (residuals) so với giá trị dự đoán (fitted values). Nếu phần dư phân bố ngẫu nhiên quanh đường zero, giả định đồng nhất phương sai có khả năng được thỏa mãn. Tuy nhiên, trong đồ thị này, ta quan sát thấy một mẫu hình rõ rệt: độ phân tán của phần dư tăng dần khi giá trị dự đoán tăng, đặc biệt sau mức giá trị 3. Một số điểm được đánh dấu như 961 và 1890 có phần dư lớn, cho thấy chúng có thể là các điểm ngoại lệ (outliers). Hiện tượng này gợi ý rằng mô hình đang gặp vấn đề về phương sai không đồng nhất (heteroscedasticity), tức là phương sai của phần dư không ổn định trên toàn bộ miền giá trị dự đoán.

- **Đồ Thị 2 - Normal Q-Q:**

  Đồ thị Normal Q-Q đánh giá xem phần dư có tuân theo phân phối chuẩn hay không. Nếu các điểm nằm gần đường thẳng lý thuyết, giả định phân phối chuẩn của phần dư được thỏa mãn. Ở đây, phần lớn các điểm nằm gần đường thẳng trong khoảng giữa (từ -2 đến 2), nhưng có sự lệch đáng kể ở hai đầu đuôi, đặc biệt tại các điểm như 491, 492 (đầu dưới) và 961 (đầu trên). Sự lệch này cho thấy phần dư có đuôi nặng hơn phân phối chuẩn, ám chỉ rằng giả định về tính chuẩn của phần dư không hoàn toàn được đáp ứng, đặc biệt với các giá trị cực đại hoặc cực tiểu.

- **Đồ Thị 3 - Scale-Location:**

  Đồ thị Scale-Location tiếp tục kiểm tra tính đồng nhất phương sai bằng cách biểu diễn căn bậc hai của phần dư chuẩn hóa so với giá trị dự đoán. Nếu phương sai đồng nhất, các điểm sẽ phân bố ngẫu nhiên quanh một đường ngang. Tuy nhiên, đồ thị này cho thấy một xu hướng tăng rõ ràng: độ phân tán của phần dư tăng khi giá trị dự đoán tăng, đặc biệt từ mức 3 trở lên. Các điểm như 961 và 1890 nổi bật với phần dư chuẩn hóa lớn. Xu hướng này củng cố bằng chứng về phương sai không đồng nhất (heteroscedasticity), xác nhận vấn đề đã phát hiện trong đồ thị Residuals vs Fitted.

- **Đồ Thị 4 - Residuals vs Leverage:**

  Đồ thị này giúp xác định các điểm ảnh hưởng mạnh (influential points) hoặc ngoại lệ bằng cách so sánh phần dư chuẩn hóa với giá trị đòn bẩy (leverage). Đa số các điểm có giá trị đòn bẩy thấp (gần 0), nhưng một số điểm như 880 và 857 có đòn bẩy cao (khoảng 0.3), và điểm 961 có cả phần dư lớn lẫn nằm gần hoặc vượt qua đường Cook's distance 0.5. Những điểm này là các điểm ảnh hưởng mạnh tiềm năng, có thể tác động không nhỏ đến sự phù hợp của mô hình. Sự hiện diện của các điểm vượt qua ngưỡng Cook's distance cho thấy cần xem xét kỹ hơn về ảnh hưởng của chúng.

5.5.4 Đánh giá hiệu suất và sự ổn định của mô hình (sử dụng Cross-validation k-fold)

Để đánh giá khả năng tổng quát hóa và độ ổn định của mô hình hồi quy tuyến tính đa biến dự đoán Pixel_Rate, ta thực hiện phương pháp cross-validation (k-fold) với k = 5. Quá trình này được thực hiện bằng cách sử dụng thư viện caret trong R, với đoạn mã như sau:

```r
# 5.3.4 Cross-validation (k-fold)
set.seed(123)
cv <- train(full_form, data = train,
            method = "lm",
            trControl = trainControl(method = "cv", number = 5))
print(cv)
```

```
Linear Regression

822 samples
 11 predictor

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 657, 659, 657, 658, 657
Resampling results:

  RMSE       Rsquared   MAE
  0.2088001  0.9461773  0.1294819

Tuning parameter 'intercept' was held constant at a value of TRUE
```

Kết quả mà ta thu được từ quá trình cross-validation được trình bày như sau:

- ***Số lượng mẫu:*** 822 mẫu được sử dụng với 11 biến dự báo (predictors).
- ***Phương pháp resampling:*** Cross-validated (5 fold), với kích thước mẫu trung bình của từng fold là khoảng 657-659 (tổng cộng 5 lần lặp).
- **Chỉ số đánh giá thu được:**
  - ***Giá trị RMSE*** trung bình là 0.2088001, phản ánh độ lệch trung bình của dự đoán so với giá trị thực tế. Đây là một chỉ số thấp, cho thấy mô hình có độ chính xác tốt trên các tập kiểm tra khác nhau.
  - ***Giá trị*** $\mathbf{R^2}$ trung bình là 0.9461773, cho thấy mô hình giải thích được khoảng 94.61% sự biến thiên của Pixel_Rate trên các fold, thể hiện độ phù hợp cao và khả năng tổng quát hóa tốt.
  - ***Giá trị MAE*** trung bình là 0.1294819, chỉ ra sai số tuyệt đối trung bình giữa giá trị dự đoán và thực tế, củng cố thêm độ chính xác của mô hình.
- **Tham số điều chỉnh:** Tham số intercept được giữ cố định ở giá trị TRUE, không có pre-processing được áp dụng.

Nhận xét chung về kết quả thu được, ta thấy rằng mô hình hồi quy tuyến tính đa biến trên có hiệu suất ổn định và khả năng tổng quát hóa cao, với RMSE thấp (0.2088001) và R^2 cao (0.9461773) trên các fold khác nhau, điều này chứng tỏ rằng mô hình không bị quá khớp (overfitting) với tập huấn luyện. Giá trị MAE bằng 0.1866866 cũng bổ sung thêm bằng chứng về độ chính xác của mô hình, cho thấy sai số dự đoán là khá nhỏ. Ngoài

ra, sự đồng nhất của các chỉ số trên 5 fold (dựa trên kích thước mẫu trung bình 657-659) cho thấy mô hình hoạt động ổn định trên các tập con khác nhau. Với kết quả thu được như trên, ta đánh giá được mô hình hồi quy tuyến tính đa biến trên là đáng tin cậy trong việc dự đoán Pixel_Rate.

Chương 6

ĐÁNH GIÁ VÀ DỰ BÁO

6.1 Dự báo trên tập test: MAE, MSE, R^2, RMSE

Để đánh giá hiệu suất thực tế của mô hình hồi quy tuyến tính đa biến dự đoán Pixel_Rate trên dữ liệu độc lập, ta thực hiện dự báo trên tập kiểm tra (test) và tính toán các chỉ số đánh giá. Quá trình này được thực hiện bằng đoạn mã sau:

```r
# 6.1 Dự báo trên tập test: MAE, MSE, R², RMSE
pred <- predict(model_final, newdata = test)
test$predicted_value <- pred

# Tính toán các chỉ số
mae_value <- mae(test$Pixel_Rate, test$predicted_value)
mse_value <- mse(test$Pixel_Rate, test$predicted_value)
rmse_value <- rmse(test$Pixel_Rate, test$predicted_value)
r2_value <- cor(test$Pixel_Rate, test$predicted_value)^2

cat("MAE trên tập kiểm tra:", mae_value, "\n")
cat("MSE trên tập kiểm tra:", mse_value, "\n")
cat("RMSE trên tập kiểm tra:", rmse_value, "\n")
cat("R² trên tập kiểm tra:", r2_value, "\n")
```

```
MAE trên tập kiểm tra: 0.132364
MSE trên tập kiểm tra: 0.04055407
RMSE trên tập kiểm tra: 0.2013804
R² trên tập kiểm tra: 0.9568332
```

Kết quả các chỉ số đánh giá trên tập kiểm tra thu được như sau:

- ***Sai số tuyệt đối trung bình MAE*** là 0.132364, cho thấy độ lệch trung bình giữa giá trị dự đoán và thực tế là khá nhỏ.

- ***Sai số bình phương trung bình MSE*** là 0.04055407, phản ánh mức độ sai lệch tổng thể, với các sai số lớn được phóng đại do tính bình phương.
- ***Căn bậc hai của MSE (RMSE)*** là 0.2013804, cung cấp một thước đo sai số trung bình theo đơn vị của Pixel_Rate, thể hiện độ chính xác tốt của mô hình.
- ***Hệ số xác định*** R^2 là 0.9568332, cho thấy mô hình giải thích được khoảng 95.68% sự biến thiên của Pixel_Rate trên tập kiểm tra, chứng tỏ khả năng tổng quát hóa cao.

Nhận xét về kết quả thu được, ta thấy rằng các chỉ số MAE (0.132364), MSE (0.04055407), và RMSE (0.2013804) đều ở mức thấp, cho thấy mô hình dự đoán chính xác trên tập kiểm tra. RMSE trên tập kiểm tra (0.2013804) thấp hơn một chút so với RMSE từ cross-validation là 0.2088001, nhưng vẫn trong ngưỡng chấp nhận được, phản ánh sự ổn định của mô hình. Giá trị R^2 bằng 0.9568332 là rất cao, gần bằng với R^2 từ cross-validation là 0.9461773, chứng tỏ mô hình không bị quá khớp (overfitting) và duy trì hiệu suất tốt trên dữ liệu mới.

Sự khác biệt nhỏ giữa các chỉ số trên tập huấn luyện (từ summary(model_final) và cross-validation) và tập kiểm tra cho thấy mô hình có khả năng tổng quát hóa tốt. Kết quả cho thấy mô hình hồi quy tuyến tính đa biến cho thấy hiệu suất rất tốt trên tập kiểm tra, xác nhận rằng mô hình có thể được sử dụng đáng tin cậy để dự đoán Pixel_Rate trên dữ liệu thực tế, với sai số thấp và độ giải thích cao.

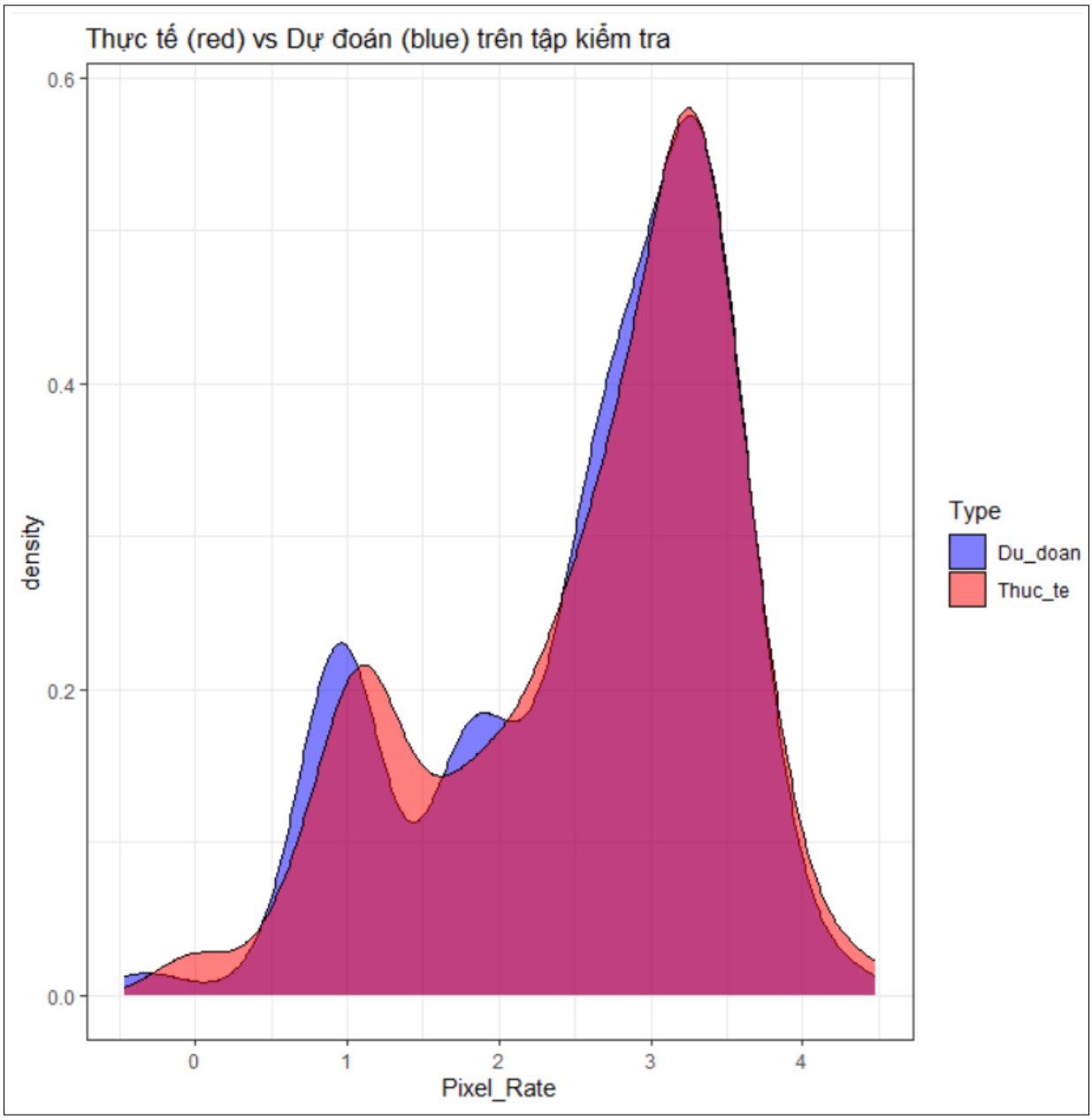
6.2 Đồ thị Density plot và đồ thị Scatter plot của thực tế so với dự đoán

Để đánh giá trực quan hiệu suất của mô hình hồi quy tuyến tính đa biến trong việc dự đoán Pixel_Rate trên tập kiểm tra (test), ta sử dụng density plot và scatter plot so sánh phân bố của giá trị thực tế (Pixel_Rate) và giá trị dự đoán (predicted_value). Hai đồ thị này giúp nhận diện sự tương đồng hoặc khác biệt giữa hai phân bố, từ đó đánh giá độ chính xác và xu hướng sai lệch của mô hình.

6.2.1 Đồ thị Density plot

Ta sử dụng mã sau cho đồ thị:

```r
# 6.2 Density plot/Scatter plot thực vs dự đoán
# Density plot
test_long <- test %>%
  select(Pixel_Rate, predicted_value) %>%
  rename(Thuc_te = Pixel_Rate, Du_doan = predicted_value) %>%
  pivot_longer(everything(), names_to = "Type", values_to = "Value")

ggplot(test_long, aes(x = Value, fill = Type)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("blue", "red")) +
  theme_bw() +
  labs(title = "Thực tế (red) vs Dự đoán (blue) trên tập kiểm tra", x =
  ↪  "Pixel_Rate")
```



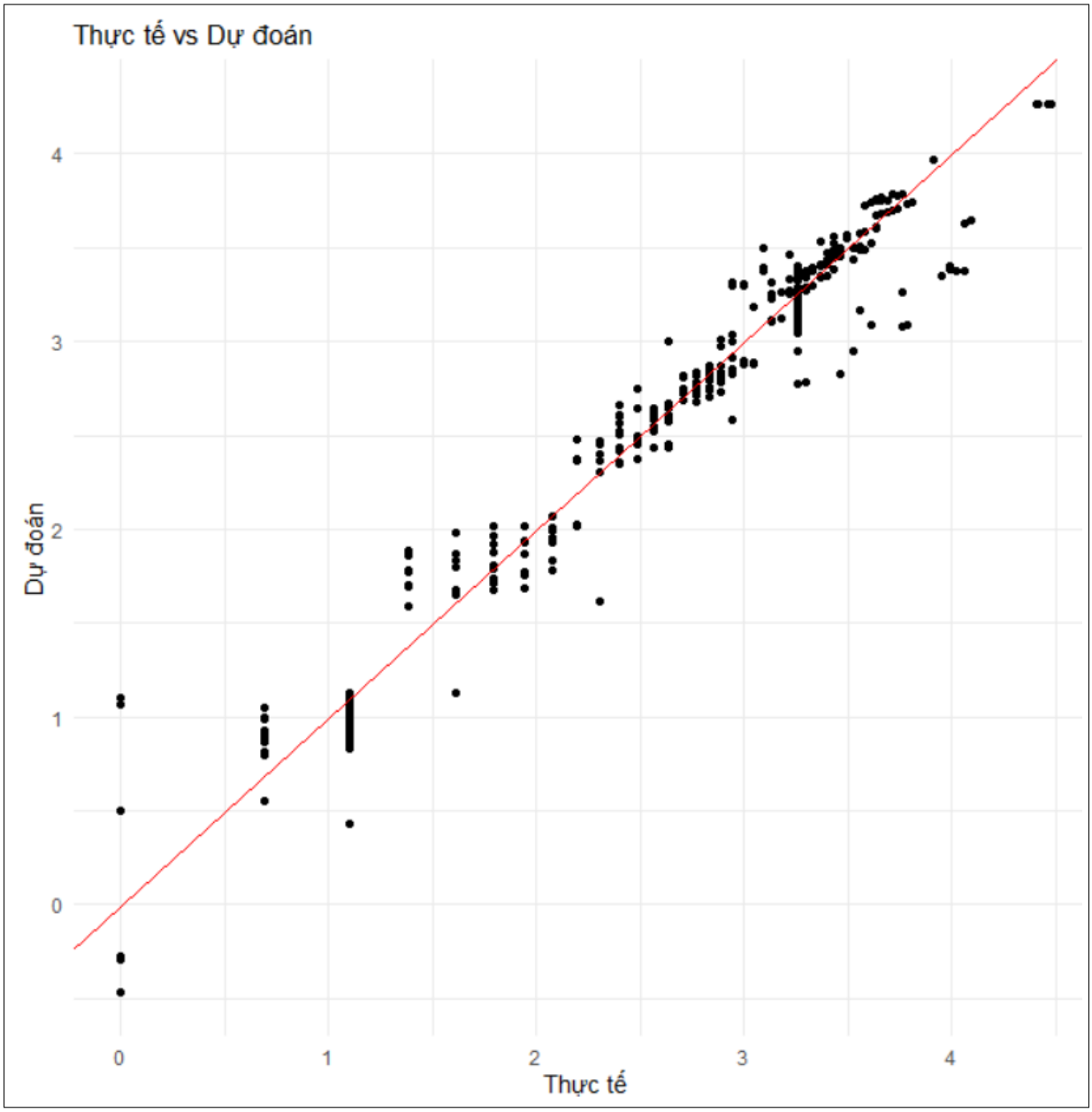
Trong đó, phần màu đỏ là giá trị thực tế Pixel_Rate mà ta quan sát được, còn màu xanh chính là giá trị dự đoán được tính toán từ mô hình. Có thể thấy mô hình dự đoán khá tốt với giá trị dự đoán phân bố gần với giá trị thực tế, những phần trùng nhau chiếm diện tích khá lớn và có hình dạng gần tương tự nhau, điều này cũng phù hợp với những kết quả thu được trước đó ($R^2 = 0.9568332$, RMSE $= 0.2013804$). Đường mật độ của giá trị dự đoán có thể hơi mịn hơn do tính chất tổng quát hóa của mô hình, trong khi đường thực tế có thể có các đỉnh hoặc đuôi nhọn hơn do dữ liệu thực tế.

6.2.2 Đồ thị Scatter plot

Ta sử dụng mã sau cho đồ thị:

```
# Scatter plot
ggplot(test, aes(x = Pixel_Rate, y = predicted_value)) +
  geom_point() +
```

```
  geom_abline(slope = 1, intercept = 0, col = "red") +
  theme_minimal() +
  labs(title = "Thực tế vs Dự đoán", x = "Thực tế", y = "Dự đoán")
```



Với đường chéo màu đỏ biểu diễn đường lý tưởng (y = x), nơi giá trị dự đoán bằng với giá trị thực tế, còn những chấm màu đen chính là các cặp giá trị thực tế và dự đoán. Ta dễ dàng thấy được các điểm màu đen tập trung dày đặc gần đường chéo màu đỏ, thể hiện rằng đồ thị có độ chính xác cao, chỉ có một số ít điểm phân tán do sai số.

6.3 Prediction intervals cho tương lai

Từ mô hình hồi quy tuyến tính đa biến mà ta đã xây dựng, ta có thể thực hiện việc dự đoán giá trị của biến Pixel_Rate thông qua các giá trị của các biến độc lập trong mô hình.

6.4 Kịch bản dự báo (giả định khi tăng/giảm biến X)

Để phân tích tác động của việc thay đổi biến độc lập Core_Speed đến Pixel_Rate, ta thực hiện dự báo dựa trên một kịch bản giả định tăng 10% giá trị Core_Speed. Quá trình được thực hiện bằng đoạn mã sau:

```r
# 6.4 Kịch bản dự báo (giả định tăng/giảm biến X)
scenario_data <- test
scenario_data$Core_Speed <- scenario_data$Core_Speed * 1.1
pred_scenario <- predict(model_final, newdata = scenario_data)
cat("Dự đoán với Core_Speed tăng 10%:\n")
print(summary(pred_scenario))
```

Bảng 6.1: Thống kê mô tả giá trị dự đoán khi tăng 10% Core_Speed

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
Giá trị	-0.163	2.184	3.128	2.871	3.596	4.564

Giá trị trung bình dự đoán là 2.871, cho thấy xu hướng thay đổi của Pixel_Rate khi Core_Speed tăng. So với giá trị trung bình gốc của Pixel_Rate trên tập test, kết quả này phản ánh sự thay đổi theo chiều hướng tăng của biến phụ thuộc khi biến độc lập Core_Speed được điều chỉnh tăng. Với hệ số hồi quy của Core_Speed là 0.32165 (từ summary(model_final)), việc tăng 10% Core_Speed sẽ làm Pixel_Rate tăng một lượng tỷ lệ với giá trị ban đầu của Core_Speed, do đó mức tăng cụ thể phụ thuộc vào từng quan sát trong dữ liệu.

Chương 7

THẢO LUẬN VÀ MỞ RỘNG

7.1 Tổng kết kết quả chính

Bài báo cáo đã tiến hành phân tích toàn diện các yếu tố ảnh hưởng đến hiệu suất dựng ảnh Pixel Rate của GPU dựa trên dữ liệu từ các thông số kỹ thuật như Core Speed, Memory Speed, Memory Bus, Shader, TMUs, ROPs và các biến khác. Thông qua các phương pháp thống kê mô tả, kiểm định giả thuyết, hồi quy tuyến tính đa biến, và các kỹ thuật đánh giá như cross-validation và dự báo, nghiên cứu đã làm rõ mối quan hệ giữa các biến độc lập và hiệu suất xử lý pixel của GPU.

Dưới đây là các kết quả mà nhóm đã thu được:

- **Thống kê mô tả và trực quan hóa:** Các đồ thị density plot và scatter plot (phần 6.2) cho thấy sự tương đồng giữa giá trị thực tế và dự đoán trên tập kiểm tra, với $R^2 = 0.9568332$, khẳng định mô hình giải thích tốt sự biến thiên của Pixel_Rate.
- **Kiểm định giả thuyết:** Phân tích ANOVA (phần 5.4.4) cho thấy tất cả các biến độc lập (như Core_Speed, ROPs, Memory) đều có ý nghĩa thống kê (p-value $<$ 0.05), hỗ trợ câu hỏi nghiên cứu về tác động của các thông số đến hiệu suất GPU.
- **Hồi quy tuyến tính đa biến:** Mô hình đạt Adjusted $R^2 = 0.9506$ (phần 5.5.2) và $R^2 = 0.9461773$ (phần 5.5.4 từ cross-validation), chứng tỏ mô hình giải thích được hơn 95% sự biến thiên của Pixel_Rate. Tuy nhiên, RMSE = 0.2013804 (phần 6.1) và các khoảng dự đoán rộng (phần 6.3) cho thấy vẫn tồn tại sai số cần cải thiện.
- **Dự báo và kịch bản:** Phần 6.3 và 6.4 cung cấp các khoảng dự đoán và phân tích kịch bản (tăng 10% Core_Speed làm tăng trung bình Pixel_Rate lên 2.871), hỗ trợ ứng dụng thực tế.

7.2 Hạn chế của phân tích

Mặc dù mô hình đạt hiệu suất cao, bài báo cáo vẫn tồn tại một số hạn chế:

- **Dữ liệu chuẩn hóa:** Kết quả âm từ khoảng dự đoán với dữ liệu mẫu chưa chuẩn hóa (phần 6.3) cho thấy mô hình nhạy cảm với việc tiền xử lý dữ liệu. Nếu dữ liệu mới không được chuẩn hóa giống như tập huấn luyện, dự đoán có thể không chính xác.

- **Biến chưa khai thác:** Một số yếu tố tiềm năng như nhiệt độ hoạt động, công suất tiêu thụ, hoặc hiệu suất thực tế trong các ứng dụng cụ thể (ví dụ: game, AI) chưa được đưa vào mô hình, có thể ảnh hưởng đến Pixel_Rate.
- **Ngoại lệ và ngoại suy:** Các giá trị ngoại lai trong tập dữ liệu (phát hiện từ đồ thị Residuals vs Leverage, phần 5.4.3) và kết quả âm từ ngoại suy (phần 6.3 với new_data ban đầu) cho thấy mô hình có thể không ổn định khi áp dụng cho dữ liệu ngoài miền huấn luyện.
- **Giới hạn tuyến tính:** Mô hình hồi quy tuyến tính có thể không bắt được các mối quan hệ phi tuyến giữa các biến, đặc biệt với các giá trị cực đại hoặc cực tiểu.

7.3 Đề xuất hướng phát triển tiếp theo

- **Sử dụng mô hình phi tuyến:** Thử nghiệm các mô hình phi tuyến như hồi quy polynomial hoặc mạng nơ-ron nhân tạo (ANN) để bắt các mối quan hệ phức tạp hơn, đặc biệt khi xử lý ngoại suy hoặc dữ liệu ngoại lệ.
- **Chuẩn hóa và mở rộng dữ liệu:** Phát triển một quy trình tự động chuẩn hóa dữ liệu mới (ví dụ: log transformation hoặc scaling) để đảm bảo tính nhất quán với tập huấn luyện. Thu thập thêm dữ liệu từ các GPU hiện đại với thông số đa dạng hơn (như nhiệt độ, công suất) để mở rộng mô hình.
- **Thêm dữ liệu và feature engineering:** Bổ sung các biến như L2 cache, băng thông bộ nhớ thực đo, kiến trúc nhân (CUDA cores / stream processors), tiêu thụ điện năng... Tạo biến tương tác (ví dụ ROPs $\times$ Core_Speed) hoặc biến tỷ lệ (Memory_Bus $\times$ Memory_Speed) để tăng độ giải thích.
- **Phân tích chuỗi thời gian:** Dùng ngày phát hành (Release_Date) để xây dựng mô hình dự báo xu hướng theo thời gian, ví dụ ARIMA hoặc prophet, kết hợp hồi quy đa biến.
- **Ứng dụng thực tế:** Áp dụng mô hình vào các bài toán cụ thể như tối ưu hóa GPU cho học máy hoặc xử lý đồ họa thời gian thực, đồng thời kiểm tra hiệu quả trên tập dữ liệu độc lập từ các nhà sản xuất khác nhau (Intel, AMD, Nvidia).

Tài liệu tham khảo

[1] Nguyễn Tiến Dũng (chủ biên), Nguyễn Đình Huy, *Xác suất - Thống kê & Phân tích số liệu*, 2019.

[2] Green, O., Fox, J., Young, J., Shirako, J., & Bader, D. (2019), *Performance Impact of Memory Channels on Sparse and Irregular Algorithms.*
Truy cập tại: `https://arxiv.org/pdf/1910.03679`

[3] Peter Dalgaard, *Introductory Statistics with R*, Springer, 2008.

[4] Nguyễn Thanh Nga (2021), *Mô Hình Hồi Quy Bội.*
Truy cập tại: `https://rpubs.com/TKUD/810985`

Nguồn code R của nhóm:
`https://www.kaggle.com/code/minhnhatvuong/xstk-252`